Humboldt-Universität zu Berlin
Mathematisch-Naturwissenschaftliche Fakultät
Institut für Informatik

Fuzzing Minimal Inputs for Interactive Programs

Masterarbeit

zur Erlangung des akademischen Grades
Master of Science (M. Sc.)

eingereicht von: Kai Patrick Münch

Gutachter/innen: Prof. Dr. rer. nat. Lars Grunske
Prof. Dr. Thomas Kosch

eingereicht am: June 6, 2025 verteidigt am: August 26, 2025

Abstract. Current fuzzing tools only apply to a small range of target applications, often restricted by programming language or platform. Additionally, they are primarily limited to non-interactive programs under test (PUTs). As a result, many approaches targeting interactive programs have been developed. Many of these are even more restrictive, requiring access to GUI data in addition to regular internal program data.

While black-box approaches exist, they are also constrained to specific platforms and operate with minimal guidance, often blindly searching for faults. Other existing approaches generate long input sequences containing superfluous user-events that are unnecessary to trigger the observed abnormal behavior. Traditional grey-box fuzzers already struggle to uncover issues hidden in deep program states and no black-box approaches currently target such states at all due to the lack of internal state information and the prevalence of irrelevant user-events.

This work introduces TimeFuzz, a novel fuzzing framework for targeted black-box fuzzing of generic interactive applications. It employs a generational approach to systematically generate program inputs that target deep program states. TimeFuzz incrementally extends inputs to reach deeper states while integrating input reduction techniques to minimize generated inputs. The framework uses a grammar based input generation approach that interprets program inputs as sequences of user-events, enabling the collection of unique time related data unavailable to traditional fuzzers. TimeFuzz supports extensive user customization, enabling communication with arbitrary PUTs and ensuring general applicability across platforms and programming languages. To explore deep states without instrumentation, TimeFuzz employs simple heuristics to analyze and select promising inputs for extension, ultimately exercising the targeted functionality.

TimeFuzz is evaluated against AFL and its extension for deep state fuzzing, IJon, in four experiments, applying variations of the Maze and Super Mario Bros. The fuzzing tools are compared in terms of runtime, number of executed tests, and the generated input's length.

The results demonstrate that TimeFuzz outperforms both AFL and IJon in runtime and number of executed tests across all experiments. However, the evaluation also reveals limitations in the proposed approach, identifying avenues for future improvement. While a large-scale evaluation is needed to fully assess the general applicability of TimeFuzz's generational approach, the current results show that TimeFuzz is a practical fuzzing framework. It is applicable to diverse PUTs across programming languages and platforms, and it demonstrates strong performance across a wide range of test execution times, an essential factor for practical fuzzing of both lightweight and computationally intensive applications.

Contents

1. Introduction **1**

2. Interactive Programs **6**
2.1. Program classifications 6
2.2. Class differences 7
2.3. Observable Information 9

3. Background **11**
3.1. Introduction to Fuzzing 11
3.1.1. Basics of fuzzing 11
3.1.2. Fuzzing Approaches 12
3.1.3. Coverage based Fuzzing 13
3.2. Input Generation 15
3.2.1. Mutation based Generation 15
3.2.2. An Introduction to Grammars 16
3.2.3. Grammar based Generation 18
3.3. Directed Fuzzing 19
3.4. Input Reduction 20

4. Related Work **22**

5. Framework **30**
5.1. Workflow of TimeFuzz 31
5.2. Implementation Details and Customization 33
5.3. Computational Architecture and Save Files 35
5.4. Executing tests and obtaining data 37
5.5. Exclusion Tree 38

6. Algorithms **41**
6.1. Input Generation 41
6.1.1. Time Grammars 41
6.1.2. Derivation, Expansion, and Extraction 42
6.1.3. Generating a Program Input 49
6.2. Delta Debugging 49
6.2.1. Standard Delta Debugging 51
6.2.2. Score Progress 53
6.3. Selecting Inputs to minimize and Sources for input Generation 56

7. Evaluation **59**
7.1. The Maze 60
7.2. Super Mario Bros C 65
7.2.1. Super Mario Bros Restricted Input 66
7.2.2. Super Mario Bros Unrestricted Input 69

7.2.3. Conclusion and discussion of resources 71
7.3. TimeFuzz on slow PUT 73
7.4. Comparing Delta Debugging Algorithms 75

8. Discussion **78**

9. Threats to Validity **83**
9.1. Internal Threats 83
9.2. External Threats 84

10. Limitations **85**

11. Future work **87**

12. Conclusions **90**

References **97**

Appendix **A1**

A. Exclusion Tree Sizes **A1**

List of Figures

1. OneNote Screenshot . . . 7
2. Zork Screenshot . . . 7
3. Movement of Mario in Super Mario Bros . . . 9
4. Example Derivation Tree . . . 17
5. Execution flow of TimeFuzz . . . 31
6. Inheritance structure of TimeFuzz's classes . . . 36
7. Exclusion Tree Example . . . 39
8. Example Derivation Tree Time Grammars . . . 43
9. The three Maze versions . . . 61

List of Tables

1. Results of the Maze Experiment . . . 62
2. Tests until best Input in the Maze Experiment . . . 62
3. Tests run in the Maze Experiment . . . 63
4. Length of generated Inputs in the Maze Experiment . . . 64
5. Results of experiment on Super Mario Bros C with restricted Input . . . 67
6. Results of experiment on Super Mario Bros C with unrestricted Input . . . 70
7. Results of the experiment on the Python based Super Mario Bros . . . 74
8. Total tests and length of inputs for the Python based Super Mario Bros . 74
9. Results of the experiment on the Python based Super Mario Bros . . . 76
10. Results of the Delta Debugging experiment . . . 76
11. Average values for the exclusion tree on the Maze using TimeFuzz Score. 80
12. Average values for the exclusion tree on the Maze using TimeFuzz Length. 80
13. Average values for the exclusion tree on the restricted SMB C. . . . A1
14. Average values for the exclusion tree on the unrestricted SMB C. . . . A2

List of Algorithms

1. Minimizing Delta Debugging Algorithm . . . 21
2. Handle running Tests . . . 38
3. Get Event Sequence from Derivation Tree . . . 44
4. Derive Word from Tequence Grammar . . . 45
5. Extract Subsequence from Derivation Tree . . . 47
6. Extend Input Sequence . . . 48
7. Generate Input . . . 50
8. Decide Subsets and Complements . . . 51
9. Generate Subsets and Complements . . . 52
10. Score Progress Helper Functions . . . 54
11. Score Progress . . . 55

1. Introduction

Automated software testing is a cornerstone of modern software engineering used to assist quality assurance [63]. Among automated techniques, fuzzing has gained widespread use to discover bugs, vulnerabilities and general abnormal program behavior, by testing programs with randomly generated inputs [69]. Fuzzers have been instrumental in discovering vulnerabilities and improving the robustness of software systems, ranging from Unix utilities [69] to communication protocols, with vulnerabilities such as the heartbleed bug being discovered.

Traditional fuzzers such as AFL [58] and libfuzzer [34] use mutation based strategies to generate program inputs for programs under test (PUTs) and combine it with program instrumentation to track execution paths and code coverage during test execution. The generated information is then used to guide the generation of new inputs with the goal of producing crashes or other identifiable abnormal program behavior. In addition to mutation based fuzzing, grammar based fuzzing has been used to limit generated inputs to valid program inputs improving efficiency for some applications [30]. Traditional fuzzing tools have achieved wide-spread success in discovering abnormal program behavior, as is evidenced by numerous studies [9][29][16][42][55]. While grey- and white-box fuzzing approaches that require access to program instrumentation or an application's source code have found great success, black-box approaches are only successful when supplied with good seed inputs that can be used as starting points for input generation [29]. In contrast, white- and grey-box fuzzers are typically limited to specific programming languages, such as AFL is limited to PUTs written in C [58]. Black-box fuzzers are often applicable to a wider variety of PUTs. Traditional fuzzers are particularly effective when applied to deterministic programs that process single inputs like files or strings, as they generate a single large input.

However, they struggle in addressing interactive programs, such as games, command-driven tools, or GUI based applications. These applications require continuous sequences of user-events as input, that interact with the program's internal state over time, and for which not only the order of events but also the timing is crucial. For example, completing a level in Super Mario Bros [52] requires sequences of button presses with timed and dynamic responses by the user to avoid obstacles and defeat enemies. Traditional fuzzers are poorly suited to this context as they cannot generate inputs that satisfy complex temporal properties [68].

To address this gap, new automated testing approaches have been developed that exclusively target interactive programs, particularly in domains like GUI testing [12], mobile application fuzzing [81], and browser and website fuzzing [6].

These tools, however, are limited in their applicability to programming languages like Java [33], platforms [46][60] or both [66], as they rely on internal program information provided by the environment [60], are limited to specific frameworks [66], or employ instrumentation. They lack generality and are difficult to adapt to other platforms, programming languages, or closed source applications that do not provide required information.

Notably, some fuzzing approaches are not available for interactive programs at all,

including directed fuzzing. Directed fuzzing, such as AFLGo [16], has shown significant success in generating inputs that trigger specific user defined abnormal program behavior or exercise particular lines of code [16]. Generating additional inputs for already known bugs can help developers locate bugs and/or understand the conditions under which they occur [49]. The generated inputs can also be used as test cases for patch development. In contrast, exercising particular lines of a program can be used for patch testing [16]. Yet this approach has not been extended to interactive programs. Instead, primary approaches favor broader exploration like model based testing or symbolic execution [81].

Similarly, black-box approaches for interactive software remain underdeveloped and are most often limited to specific platforms, such as with Monkey, the primary black-box testing tool for Android [59]. This is because of the respective tools' dependence on specific API functions that allow to dispatch user-events to the PUT. As APIs are not generally available cross-platform and for every application, black-box approaches suffer from the same limitations as grey-box approaches on interactive programs.

Furthermore, fuzzing interactive programs tends to generate long input sequences composed of a substantial amount of unnecessary user-events [46][36]. These sequences, are difficult for a developer to analyze. Long sequences may obscure the set of user-events responsible for crashes or abnormal behavior

This motivates the use of input reduction algorithms such as minimizing Delta Debugging (DD) to reduce input sequences helping developers understand why programs fail [97]. Input reduction algorithms systematically reduce the size of an input until it reaches 1-minimality, where no single event can be removed, without the abnormal behavior vanishing [97]. Input reduction techniques have been improved over the past decades [70][85] and have been applied to reduce event sequences for interactive programs [19][46], but have not been used integrated into input generation strategies.

While fuzzing tools for interactive programs implicitly recognize inputs as series of user-events, no black-box fuzzing tool has used the unique domain independent time related data only available for interactive programs to actively steer input generation. Incorporating time into the analysis of program behavior allows for unique domain independent data to be collected. This includes responsiveness to individual user-events and exact point of failure from a user perspective. While traditional fuzzers provide a PUT with a single user-event and have to guess where the input failed exactly, a fuzzer for interactive programs can dispatch individual user-events to the PUT and monitor whether the PUT has actually consumed the user-events. The moment the PUT exits a point of failure can be determined. Additionally, interactive programs offer a unique opportunity to retrieve internal program data without the need for instrumenting the PUT. Interactive applications generate visible and collectable information, such as command line output, visual output, and updates to displayed score values in games, all of which can be collected without the need to instrument an application. This information can be used to steer input generation. However, no interactive fuzzing tools make use of this behavioral feedback.

This work presents TimeFuzz, a novel framework for directed black-box fuzzing of interactive programs. TimeFuzz represents program inputs as time series of user-events, capturing runtime observations such as response times and points of failure. It employs a

grammar based input generation approach to ensure that input sequences are syntactically valid, and uses an iterative feedback loop to progressively explore the input space. Promising inputs are selected and systematically extended to form longer event sequences, allowing the fuzzer to reach deeper program states over time.

To decide whether a generated input satisfies the fuzzing objectives, TimeFuzz relies on a user defined test oracle. The oracle evaluates the collected data, decides test outcomes, and optionally provides a score based progress metric toward the objectives. This score enables TimeFuzz to effectively select promising inputs during iterations (in the following referred to as generations). The oracle operates on black-box data, including program outputs and response times, enabling the inclusion of domain specific data into the feedback loop without requiring instrumentation or access to the PUT's source code.

When an input satisfies the fuzzing objectives, TimeFuzz applies DD to minimize the input sequences while preserving the observed behavior. Moreover, if a score value is available, TimeFuzz uses DD to minimize the input while preserving its progress, enhancing the quality of source inputs for future generations. When the oracle offers fine grained score values, one value per user-event in the input, TimeFuzz applies the newly developed `Score Progress` DD algorithm, which only applies DD to those subsequences of the input that show stagnation.

This allows TimeFuzz to fuzz programs with time constraints or input length limitations, such as video games where winning strategies must be achieved within a time limit. For example, in Super Mario Bros, input sequences that fail to complete a level within the allowed time limit result in Mario's death, making short input sequences without superfluous user-events essential.

In addition, TimeFuzz builds an exclusion tree based on the observed points-of-failures. This tree stores information about executed passing and failing inputs and is used to predict the outcome of new inputs, preventing the execution of redundant inputs.

The framework is designed around the following core principles:

- **Generality** The framework is able to communicate with PUTs either directly or through an intermediary API, allowing it to fuzz a broad range of PUTs regardless of programming language or operating system.
- **Scalability** Input generation and test execution scale seamlessly to available computational resources.
- **Modularity** Design and implementation are modular, providing a framework that is easy to understand and maintain.
- **Extensibility** TimeFuzz can be easily extended to new input generation approaches, input reduction algorithms, and selection algorithms for promising inputs.
- **Customizability** TimeFuzz exposes configurable parameters, allowing users to tailor the frameworks functionality and algorithms to individual PUTs.

- **Robustness** TimeFuzz implements a backup mechanism that allows for fuzzing sessions to be paused and resumed without data loss, and provides robustness against issues that may arise within TimeFuzz or its environment.

Together these will provide a flexible and extensible framework for black-box fuzzing of interactive programs.

To evaluate the effectiveness of the proposed approach and TimeFuzz itself, the following research questions are being posed:

- **RQ1** How does TimeFuzz compare to the popular grey-box fuzzing tool AFL and its extension IJon, focusing on deep state fuzzing? That is, can TimeFuzz solve similar problems in comparable time, despite being a black-box fuzzer without access to internal program data?
- **RQ2** To what extend are winning strategies generated by TimeFuzz minimal, and how do they compare to those generated by AFL and IJon?
- **RQ3** How effective and efficient is the `Score Progress` delta debugging algorithm, and how does it compare to the standard delta debugging algorithm?
- **RQ4** How effective is the usage of an exclusion tree to reduce the number of executed tests and how much memory is needed to maintain it?
- **RQ5** To what extent does TimeFuzz adhere to the core principles outlined in the introduction?

To address the research questions, it will be analyzed whether fuzzing based on simple time based information is effective and how it compares to established fuzzing approaches, specifically to general mutation based fuzzing with AFL [58] and targeted fuzzing for deep state exploration using IJon [9]. It will be evaluated which fuzzers are able to complete the selected test cases, the Maze and Super Mario Bros, how much runtime is required to generate the first satisfying program input, and how many tests are executed in total before this input is found.

The remainder of this thesis is structured as follows. Chapter 2 defines interactive programs and specifies the time related data collected by TimeFuzz. Chapter 3 introduces relevant background and reviews related literature. Chapter 5 describes the core design of TimeFuzz and presents the general workflow. It also provides technical details about the implementation and outlines the computational architecture of the framework. Additionally, this chapter discusses the test execution and how response times and points-of-failure are obtained. The final part of the chapter introduces the exclusion tree which is used to store and predict test results.

Chapter 6 presents the implemented input generation and extension algorithms, the implementation of DD, and the new developed `Score Progress` algorithm. It also explains the strategy for selecting promising inputs at the end of generations.

Chapter 7 outlines the evaluation setup, describes the conducted experiments, and presents the experimental results. Chapter 8 interpretes these results in terms of the

research questions. Chapter 9 discusses internal and external threads to validity of the evaluation. Chapter 10 addresses the limitations of both the approach and the framework itself. Chapter 11 outlines possible avenues for improvement and future work. Finally, Chapter 12 presents the conclusions of the work.

2. Interactive Programs

The concept of software testing is trivial: Generate a test input, execute the PUT with the test input, and observe the PUTs behavior. Even though the concept is straightforward, the task itself is resource- and time-consuming and can easily make up a significant part of software development cost [37]. To reduce the cost of testing, a large number automated tools have been developed and continuously improved using a variety of input generation approaches, such as Fuzzing. The tools are not equally applicable to all PUTs. They work best on specific classes of programs that fit their design. In this section, a binary classification for interactive and non-interactive programs will be given.

2.1. Program classifications

There are many possible classifications of software relying on a variety of different attributes. In this work, the classification focuses on how programs receive and use input. There are two basic classes of programs in this approach: (1) programs that receive exactly one input at the beginning of the execution (PSI - program with single input), and (2) programs that receive continuous input during execution (PMI - program with multiple inputs).

The first class of programs is the traditional target of automated software testing tools such as large fuzzing suites like AFL [58] and libfuzzer [34]. At the time the PUT is executed, it immediately receives the input and does not further interact with the automated tool. How the input is provided to the PUT depends on the PUT itself, it may receive the input over the standard input, or it could be provided in a file the program reads. After the PUT receives the input it computes a result and exits. Some popular examples that belong into this category and are often used as test subjects in research, are compilers (e.g. gcc, clang, etc.) [85][87][94], language parsers and interpreters (e.g. XML, Json, csv, javascript, lua, etc.) [31][64], and Unix libraries and command line tools (e.g. libpng, find, grep etc.) [9][24][49].

The second class are interactive programs. They receive continuous inputs during their execution. The most representative and well-known subclass consists of programs with a graphical user interface (GUI). A program with a GUI receives sequence of user-directed interactive events in the form of pressed buttons, keyboard input, mouse-events, etc. The programs internal progress is dependent on these user-interactions. The program does not exit or progress unless the user performs specific interactions or abnormal behavior occurs. For instance, to exit the program `OneNote` [4], a user has to press the button with the label `X` in the upper right-hand corner of the GUI, as seen in 10.

Even though GUI based programs are the most representative interactive programs, there are a lot more programs that belong to the class of interactive software and do not possess a GUI. Another interesting and expansive sub-class are servers, as they receive interactive events from a remote client, which may be an interactive program itself, and respond to the events. The method of interaction may be different but the essence remains: They receive a sequence of externally generated interactive inputs. In addition to these large subclasses, there is a significant amount of other software that belongs

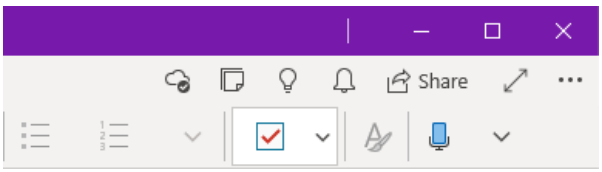


Figure 1: Screenshot of the upper right-hand corner of OneNote's GUI.

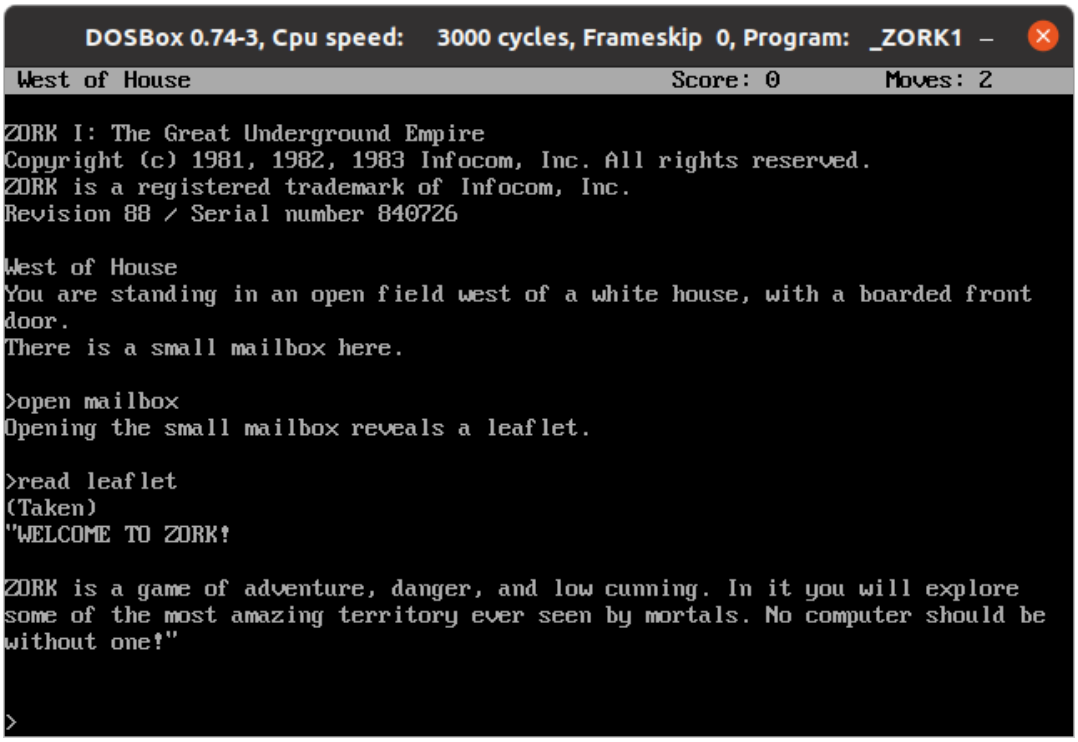


Figure 2: Screenshot of the command line adventure game Zork.[1]

to the class of interactive software. For instance, command-line tools such as ssh, sudo, passwd, or command-line based games such as Zork [86], as seen in Figure 4.

Due to the nature of binary classification both classes, interactive and non-interactive, are expansive and similarly significant are their differences.

2.2. Class differences

PSI as introduced in this chapter follow the most basic principle of data processing: input, process, and output. They compute a result and then exit while possibly providing an output. In contrast, PMI have to loop this behavior. Take Zork in Figure 4 as an example. The program begins by displaying the intro information and the beginning of the story:

[1]picture source: https://wiki.ubuntuusers.de/Spiele/Zork/ (last accessed 19.09.2024

```
West of House
You are standing in an open field west of a white house, with a
boarded front door.
There is a small mailbox here.
```

Afterwards, the program waits for the user to type the next action they want to take. In this case, `open mailbox`, after which the program responds with `Opening the small mailbox reveals a leaflet.` and waits for the next user command. This schema is common to all interactive software: Input - Process - Output and repeat. While there are differences between programs whereas some programs contain modules that do not adhere to this schema, for instance, background services that are not connected to user input, this schema applies to the input-handling parts of the software.

Another major difference between PSI and PMI exists in the interdependency of events within a program input. Trivially, all valid inputs for any program follow specific syntactic and semantic rules. A PUT designed to compute a result from an input must be able interpret that input correctly. For PSI, an input specified by the program's input language may contain some interdependent parts. For example, the IJON document used as input for vcpkg [20]:

```
{
  "name": "example",
  "version": "1",
  "dependencies": [
    "frozen",
    "lua",
    "magic-enum",
    "rsm-binary-io",
    "zycore",
    "zydis"
  ]
}
```

This input includes several interdependent elements, such as matching opening and closing brackets, quotation marks, and command separating elements. Additionally, the elements inside the `"dependencies"` array depend on the presence of `"dependencies"` itself. However, most of the elements are not interdependent; for example, `"dependencies"` does not relate to `"version"` and `"lua"` has no relation to `"name"`, `"frozen"`, or others. Although the number of interdependencies may seem large, each element depends on only a small number of others.

For PMI, the number of interdependent events and dependencies is often much higher. In extreme cases, every event in the input sequence depends on all prior events. Retro video games are the prime example of this behavior. When playing Super Mario Bros [52] every player action changes the character's location and thus the outcome of the interaction. Consider the current situation as shown in Figure 3a. The player character jumps to the right, resulting in the new state shown in Figure 3b. If the player then



(a) Initial position of Mario.

(b) Situation after Mario jumps to the right from 3a.

(c) Situation after Mario moved right from 3b

Figure 3: Example for the movement of Mario in Super Mario Bros and the dependency of interaction results on prior user-interactions.

moves to the right again, the state shown in Figure 3c is reached. If the first action does not occur, the second action produces a completely different result, and the third state is never reached. A sequence of actions for Super Mario Bros without unnecessary moves consists only of events that depend on all prior actions.

The order of events is not the only important time related factor for interactive programs, especially retro video games. The timing of specific user-events is equally critical. For example, in Super Mario Bros, the timing of Mario's jump can determine whether he continues his quest to save Peach or gets eaten by a plant.

2.3. Observable Information

Simply observing a program from the outside and its interactions with the environment, while providing it with the input it requires, allows for the collection of detailed information. While only few observations can be made on executions of PSI, for instance, e.g. results and the runtime.

As program inputs for PMI are sequences of user-events and all events are provided one-at-a-time, the response time of the PUT for each individual input can be observed by measuring the time taken for the program to respond. In case of Zork, this information could be obtained by simply measuring the time between when the input is provided and when the program writes to the standard output. For general applications, the response time can be measured by observing the time difference between when the input is provided and when it is collected by the PUT (e.g., when the PUT reads from the standard input). Through this approach, statistical analysis of soft requirements such as reaction times and responsiveness can be performed, and fine-grained performance bugs can be identified. This analysis could be further enhanced by associating performance bugs with features of the input language [31], enabling domain independent, fine-grained performance evaluation.

Whenever a bug is encountered (i.e. when certain constraints over the program's behavior are satisfied), the user-input that triggered the abnormal behavior can be identified. This allows the generated input to be automatically reduced by the testing tool to the time series of events that were executed by the PUT.

For deterministic interactive programs, the result observed can be categorized into three meaningful categories: (1) the test run ends in a result which satisfies some constraints and is considered a success, (2) the test run ends in a result which is either an abnormal behavior we did not want to observe or the result does not satisfy some constraints and is considered a fail, or (3) the PUT does not exit after being handed the last user-input and the test is neither considered a success nor a fail but unfinished. While the first two categories are consistent with single input programs, the third is unique to interactive programs. The latter occurs in case the input given to the program can be considered incomplete, i.e. it can be extended to result in (1) or (2) respectively. This categorization does not consider whether the PUT halts on a given input, as failing to halt can simply be considered in violation of some timeout constraint.

This categorization of test results is used by TimeFuzz to determine the point of failure of generated inputs and to decide which inputs can be extended and how they should be extended. Inputs that successfully trigger abnormal behavior do not need to be extended, as they already represent the behavior that is intended to be reproduced.

3. Background

This chapter introduces key foundational concepts. It begins with a general overview of fuzzing, explaining its basic principles, common approaches, and their respective advantages and disadvantages. This is followed by a discussion of coverage based fuzzing, the most widely used progress metric for evaluating the exploration of a PUT.

The next section focuses on input generation strategies. It begins with a brief overview of input generation, followed by a short explanation of mutation based techniques. This leads into a formal introduction to grammars and related concepts, which are essential as this work proposes a new grammar type and corresponding algorithms. Finally, grammar based input generation is discussed in detail, as TimeFuzz uses this approach to produce program inputs.

After introducing fuzzing and input generation, the chapter presents the concept of directed fuzzing, as TimeFuzz is a directed fuzzing approach. It concludes with an introduction to input reduction, including an explanation of the most widely used algorithm, which is employed in this work to minimize generated inputs.

3.1. Introduction to Fuzzing

Fuzz-testing (or Fuzzing) is an automated software testing technique originally developed by B.P.Miller et al. [69] in 1990. They tested 90 programs with randomly generated input strings and observed that over 24% of the programs crashed [69]. Since then, fuzzing has become a staple of automated software testing [55][100]. This subchapter introduces the fundamentals of modern fuzzing while the next subchapter introduces input generation approaches, as described by Zhu et al. in [100], along with an overview of relevant tools related to this thesis.

3.1.1. Basics of fuzzing

Fuzzing has been developed as an automated technique to find abnormal program behavior, such as software crashes, security violations, and violations of non-functional requirements. It is one of the most widely used and easy-to-deploy testing techniques.

A fuzzing tool (called a fuzzer) generates inputs for a program under test (PUT) automatically, executes the PUT with these inputs, and observes the program's behavior. If the fuzzer detects abnormal behavior, the input is saved and either reported to the user for manual investigation or fed directly into specialized tools to determine the nature of the abnormal behavior automatically, without user involvement.

The fuzzing process consists of three major steps: (1) input generation, (2) PUT execution, and (3) behavior observation, categorization, and reporting. The most basic fuzzing technique, random fuzzing, strictly follows this process. The fuzzer generates inputs completely at random, executes the PUT, and observes and reports the results in an infinite loop. Although this approach seems simple and effective, in reality, fuzzing with random input generation has a very low success rate in detecting bugs.

This is due to three major concerns that fuzzing is facing. First, bugs are rare. Contrary to what modern software sometimes leads us to believe, encountering software bugs is an infrequent occurrence. In other words, random generation is unlikely to produce inputs that cause abnormal program behavior. While some bugs can be found relatively quickly, many others require long and complex inputs, which are unlikely to be generated at random.

Second, most programs have a constricted and complex input language defined by large grammars with numerous syntactical and semantic restrictions on what constitutes valid input. Due to this constriction of the valid input space of many programs, random input generation is unlikely to produce inputs that satisfy all constraints. While testing a PUT with unexpected, invalid inputs can detect critical vulnerabilities, such as the Heartbleed bug [24], it is equally important to expose PUTs to valid inputs, as these exercise the functionalities users expect to work correctly.

The third concern applies to automated testing in general. Software is incredibly diverse: some software runs on a variety of systems, such as the Unix tool `grep` [26], while other software runs only on specific devices requiring dedicated hardware—for example, Intel's CPU microcode, which operates only on specific Intel CPUs. Additionally, software like `grep` is easily automated, as it requires only command-line arguments and files, whereas testing other software, such as APIs, often requires additional effort to automate execution.

In order to address these concerns, many approaches have been proposed over the past decades. These include a wide variety of tools targeting specific platforms and programming languages, as well as widely applicable tools such as AFL [58] and libFuzzer [34]. Generalized input generation and program behavior categorization techniques have also been developed, which can be applied almost universally across applications.

In the following subchapters a number of these approaches will be discussed in greater detail, including fuzzing approaches, input generation techniques and behavior categorization.

3.1.2. Fuzzing Approaches

Fuzzing tools can be categorized into the three main approaches white-, black-, and grey-box fuzzing based on the kind of information that they observe during fuzzing, and whether they require access to a PUTs source code.

All three approaches have unique strengths and weaknesses. While black-box fuzzing is the most universally applicable, it suffers from a lack of information to guide input generation algorithms. White-box fuzzing, often considered the most effective approach, cannot be applied to closed-source applications, rendering it unsuitable for research on a wide range of software. Grey-box fuzzing offers a compromise between the two. Although many popular tools require source code access, libraries like AFL provide modes that can instrument a program without source code access (see QEMU[2]) [58].

White-box fuzzing is the most restricted approach, requiring access to a PUT's source

[2]https://github.com/qemu/qemu (last accessed 14/01/2025 at 13:17)

code. This source code is used to extract information about internal program states, their response to tests, and the control and data flow. It provides the fuzzing tool with extensive data for static code analysis and precise dynamic analysis during PUT execution. The information obtained guides input generation algorithms. Because of its insights into a program's internal workings, white-box fuzzing is usually considered the most effective fuzzing approach. However, it requires more effort to set up and may suffer from performance issues, as the PUT's source code is typically heavily instrumented by the fuzzing tool. This instrumentation usually requires recompiling the PUT with custom libraries provided by the fuzzer, which can cause additional overhead if incompatibilities with the PUT's source code arise.

Black-box fuzzing can be seen as the complete opposite of white-box fuzzing. It only requires the compiled PUT and does not collect any internal program data at all. Instead, it simply observes the program's behavior, for instance, based on the PUT's output, or system events. Due to the relatively small amount of information available to the fuzzer, steering the input generation effectively is difficult. This becomes apparent when the goal of fuzzing is to generate inputs that cover all possible execution paths of a program. The biggest advantage to black-box fuzzing is its ease of use. Because black-box fuzzing does not require source code, it can be set up quickly for virtually any application, needing only instructions on how to run tests and how to identify bugs.

The last approach is gray-box fuzzing. It is a middle-ground between white-box fuzzing and black-box fuzzing. Gray-box fuzzing collects internal program data through analysis of the compiled PUT or though instrumentation that allows access to program internals. Most fuzzers collect information about the control flow of the PUT and use the edge coverage or block coverage of the PUT's `Control Flow Graph (CFG)` to track internal execution states. Even though, the theoretical approach does not require access to a PUT's source code, some popular gray-box fuzzing tools like AFL and libfuzzer require the PUT to be compiled with special instructions such that coverage data can be easily collected.

3.1.3. Coverage based Fuzzing

Steering input generation is not an easy task. The goal of fuzzing is to cover as much execution paths of the PUT as possible. A range of approaches have been proposed that seek to understand which part of a program is exhibited by a test case and what generated test cases are interesting and can used for future generations. These techniques include code-coverage, state machines for protocol implementations [21][28][79], alias coverage for concurrency implementations [93], etc. This paper discusses code-coverage, as other techniques are specialized approaches that can only applied in limited environments.

Coverage-based fuzzing instruments the PUT to track the execution path of each test case. The execution paths are mapped to the PUT's CFG, storing information about which edges are executed and how many times each edge is executed by a test. The CFG is typically constructed by analyzing a program's branches or statement blocks. Depending on the chosen method, coverage is computed as either edge coverage or block coverage, with both methods found in research. Coverage goals vary based on the graph type and

can include `coverage of all edges`, `coverage of all blocks`, or more specific goals such as covering all functions. The CFG not only accounts for edge or block coverage but also provides metrics on the number of times an edge or block is covered by a single test input. This is typically implemented using a bucketed system where inputs with similar edge or block counts are assigned to the same bucket.Since edge coverage is more precise and sensitive than basic block coverage [91] and is used by widely adopted fuzzing suites like AFL, this work will refer exclusively to edge coverage.

Irrespective of the method used to construct the CFG and the specific coverage goal chosen, the goal of the fuzzer is to generate inputs in a way that the control flow graph is explored completely by the different execution paths of the generated inputs. Thereby, inputs which explore new, prior unseen, paths in the CFG are considered interesting and used to steer input generation. This is fulfilled in case it either covers edges that have not been covered so far, or it exhibits edge repetitions which have not been produced by other inputs. The exact strategy varies based on implementation, as well as, supported bucket sizes for edge repetitions and the granularity of the CFG.

Coverage based fuzzing can be applied in both gray-box and white-box fuzzing, and is used by many fuzzing tools as a measure of completeness. The higher the CFG coverage, the less bugs the PUT is assumed to have. Using coverage based fuzzing ensures that many security issues and other bugs are found during fuzzing. Additionally, coverage also helps in preventing the fuzzer from getting stuck on specific execution paths by accidentally generating inputs that exercise paths repeatedly.

Even though coverage based tools often find large numbers of bugs, the generated inputs can still fail to cover some scenarios due to the way interesting inputs are being chosen. Selecting only inputs that cover new paths in the CFG can result in failing to find rare bugs or situations that only occur when certain conditions are met or a certain sequence of edges is covered [9]. For example, in some cases, it might be possible for a bug to only manifest under the condition that a loop is executed a fixed amount of times. Should the situation occur that a different input has already executed the corresponding edges often enough, this places both input into the same edge count buckets. Fuzzers may then fail to rightfully classify the new input as interesting and, instead, choose to exclude it from input generation. This could lead to unintentionally missing bugs.

3.2. Input Generation

The primary goal of automated testing is to generate program inputs that reveal abnormal behavior. Three common approaches to input generation are used: (1) random generation, (2) mutation-based fuzzing, and (3) grammar-based fuzzing. The first approach generates inputs entirely at random. In practice, this method has a low success rate in detecting bugs and is typically used to generate initial inputs or in combination with more advanced strategies. Mutation-based and grammar-based approaches are more sophisticated. The former generates new inputs by altering existing ones, while the latter uses a grammar of the program's input language to generate inputs. Although both build upon random generation and employ fundamentally different strategies, they are not mutually exclusive and can be combined.

3.2.1. Mutation based Generation

Mutation based fuzzers take a set of initial seed input. The fuzzer maintains a set of interesting inputs, that are to be mutated, at all times. At the beginning of the fuzzing this set of interesting inputs is entirely made up of the initial seed inputs. During the fuzzing process this set of inputs is modified by adding newly generated, more interesting inputs, and removing older inputs that are not interesting enough for mutation any more. Inputs are considered interesting if they achieve high code coverage or trigger unique execution paths in the program. Interesting inputs are mutated to generate new program inputs. Mutation is performed through a variety of operations, including random changes to bits or bytes, insertion or removal of bytes, reordering of byte sequences, splicing data from other inputs, and dictionary based substitutions [25][76][54]. Validity of inputs is not strictly required; instead, what matters is their ability to explore new or deep parts of the code. While this approach generates many invalid inputs, this is actually a strength, as it allows testing how the program handles unexpected or malformed data.

Mutation based fuzzing is a commonly used approach for input generation. It can produce varied inputs that usually represent a mix of valid and invalid program inputs testing a PUT on a wide variety of input, resulting in high code-coverage and large quantities of bugs being discovered. Despite its effectiveness in discovering bugs and achieving high code coverage, mutation based fuzzing has limitations. It often struggles with complex input formats, where structural constraints or interdependencies between parts of the input must be respected. In such cases, generating valid inputs that exercise deeper program logic becomes unlikely. This is especially problematic when fuzzing structured data like communication protocols or file formats. As a result, while mutation based fuzzing excels at general robustness testing, it may fall short in scenarios that require consistent production of valid, structurally correct inputs.

3.2.2. An Introduction to Grammars

A grammar is a formal concept that is used to describe languages. Languages are possibly infinite sets of sentences of finite length. Whether the languages are natural languages such as the English language or artificial languages such as C++, any language can in theory be described by a corresponding grammar [47][18].

According to [53], a grammar is formally defined as a 4-tuple G = (V, T, P, S), where:

- V is a finite set of non terminal symbols,
- T is a finite set of terminal symbols with $V \cap T = \emptyset$,
- $P \subseteq \Gamma^+ \times \Gamma^*$ is a finite set of productions, where $\Gamma = V \cup T$ and $l \to r$ describes the derivation from l to r,
- $S \in V$ is the start symbol,

and the language L that the grammar describes as $L(G) = \{w \in T^\star | S \xrightarrow{*} w\}$, where $\xrightarrow{*} \subseteq \Gamma^+ \times \Gamma^*$ and $S \xrightarrow{*} w$ describes that there is a sequence of derivations that derive w from S.

The set of valid inputs for a computer program, i.e. its input language, can be described by a formal grammar of the above form. Take the following simplified representation of an input grammar for a limited calculator program as an example:

```
Grammar(
    'start := 'function ~ "(" ~ 'numbers ~ ")",
    'function := "sqrt" | "tan" | "sin" | "cos",
    'numbers := 'number | 'numbers ~ 'number,
    'number := "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
)
```

The formal definition would be: Grammar $G = (V, T, P, S)$, where:

- $V = \{'start, 'function, 'numbers, 'number\}$
- $T = \{(,), 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, sqrt, tan, sin, cos\}$
- $P = \{'start \to' function \circ (\circ 'numbers \circ), 'function \to sqrt, 'function \to tan, 'function \to sin, 'function \to cos, 'numbers \to number, 'numbers \to' numbers \circ' number, 'number \to 0, 'number \to 1, 'number \to 2, 'number \to 3, 'number \to 4, 'number \to 5, 'number \to 6, 'number \to 7, 'number \to 8, 'number \to 9\}$
- $S =' start \in V$

These two definitions are equal: (1) any non-terminal symbol that is found in the simplified form can be found in V, (2) any terminal symbol that is found in the simplified form can also be found in T, (3) the start symbol $'start$ which is the root of the grammar is S and can be found in V, and (4) all productions that are described in the simplified form can also be found in P. The same holds the other way round: (1) Any non-terminal

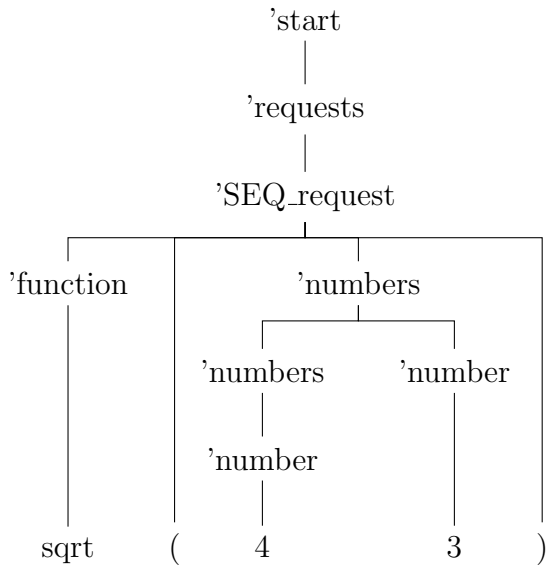


Figure 4: A derivation tree for the word "sqrt(43)" of the calculator language.

symbol from V occurs as the left side of a production in the simplified form, (2) any terminal symbol from T can be found on the right side of a production rule in the simplified form, (3) the start symbol S is the left side of a production rule and can be seen as the root of the simplified grammar, and (4) all production rules found in P have an equivalent in the simplified form.

In the following, the simplified form will be used over the formal definition to improve readability.

Grammars that describe input languages of programs can be very diverse, from simple grammars for simple calculator programs as seen above, to grammars with complex syntactical structure such as the grammar for the C++ programming language. C++ grammar definition spans 19 pages, while leaving out a significant amount of features, and only defining a superset of the language [15].

Any word of an input language can be derived from the start symbol of the grammar by applying a certain number of production rules. This process can be represented by a derivation tree. A derivation tree contains all non-terminal nodes that production rules are applied to as well as all nodes that are produced by the production rules. A derivation tree for the word `sqrt(43)` of the calculator language defined above could look like the tree shown in Figure 4

A derivation tree is an accurate representation of the applied production rules and symbols they were applied to. It contains all terminals that make up the produced word and all symbols along the way. The derivation tree is often called parse tree. A parser can be used to parse a word of the input language and construct the derivation tree (parse tree) on demand, as long as a specification of the language is available [23][45]. Parse trees are typically used by parsers for languages like json or xml that store information in treelike structures. Another use is by programs for lexical analysis like gcc and clang. Using algorithms [23][45] to construct a parse tree for a given word is a common way to

check whether the given word is part of the input language. If the word belongs to the input language, at least one parse tree must exist. If no parse tree exists, the given word cannot be part of the input language.

In computer sciences, the most commonly used grammars are type 2 and type 3 languages as defined by Chomsky [18]. Regular languages (type 3) are heavily restricted in their expressiveness, but they can be parsed and evaluated relatively easily. A lot of different parse and evaluation libraries exist for regular expressions, many of those integrated into programming languages such as std::regex for C++ [3]. Context free languages (type 2) are more complex than regular languages and require complex algorithms with high runtimes for parsing such as the early parser [23] which has a cubic worst case complexity in the length of the input word. Context sensitive (type 1) languages and in extension type 0 languages are rarely used in computer sciences, as these languages are well-known to be human unfriendly and all parsings algorithms for type 0 and type 1 languages have exponential complexity [35].

3.2.3. Grammar based Generation

Grammar based fuzzing approaches use a grammar to generate program inputs. The fuzzer uses the grammar to derive valid program inputs and uses them as test cases for the PUT. This approach aims at verifying that the PUT behaves correctly on valid inputs, validating the presences and correctness of features.

Grammars allow a user to define what exactly a fuzzer is generating and thus how a program is tested. Most often, fuzzers use the grammar of the PUT's input language, but this is not strictly necessary. In principle, any grammar can be used. For example, a grammar that generates only a subset of the input language can help test specific features of a PUT. Consider the command-line tool `x83d`, which accepts several command-line arguments. In its latest version update, the developers introduced a new argument without changing any other functionality. To verify that this new argument behaves as expected, a user can define a grammar that consistently produces only this specific argument. The fuzzer would only test `x83d` on this new option. Using a grammar in this fashion allows users to target their testing without the fuzzer generating large amounts of otherwise uninteresting inputs.

When combined with coverage-based fuzzing or similar approaches, grammar-based input generation often achieves higher code coverage earlier in the fuzzing process than mutation-based fuzzing. This is mainly because grammars generate inputs that actively exercise a program's features, while mutation-based approaches tend to produce large volumes of inputs that fail to trigger meaningful behavior. As a result, mutation-based fuzzers run more test cases, consuming more time without necessarily increasing coverage.

However, the benefit of grammar-based fuzzing depends heavily on the PUT and the test environment. In some cases—such as when the PUT accepts nearly all inputs as valid or operates on loosely structured data—grammar-based fuzzing may offer no speedup at all. In fact, the computational cost of deriving inputs from a grammar and performing

[3] https://en.cppreference.com/w/cpp/regex

related operations can make mutation-based fuzzing significantly faster in practice.

3.3. Directed Fuzzing

Directed fuzzing focuses on generating inputs that trigger specific, user defined, abnormal program behavior, or exercise particular lines of code [16].For example, when a PUT crashes, directed fuzzing can generate new inputs that reproduce the crash by reaching the same code path. Developers can also use directed fuzzing to exercise specific lines of code for purposes such as patch testing [16].

Generating more inputs capable of triggering already known bugs is useful. When encountering bugs that occur in a location that relies on a lot of internal program data, it may not be possible to correctly identify the problem from just one program input and the internal program data that is associated with it. As different inputs may lead to different internal states of the PUT, having multiple inputs that exhibit the same bug may help in differentiating between data that is relevant to the abnormal behavior and data that is not. This may help developers to identify the source problem much faster. This not only applies to internal program data that has to be analyzed manually but can be extended to the program input itself, which may contain elements that are necessary to reproduce a bug and elements that are not. Several approaches address this identifying failure circumstances using patterns in program inputs [24][49].

Having more inputs that exhibit abnormal behavior may improve the patch quality and reduce overfitting and underfitting of patches as there are more test cases that can be used to verify the patch. An overfitting patch resolves the exact test case that it was made for, while failing to cover all theoretically possible inputs that result in this abnormal program behavior. In contrast, an underfitting patch resolved the issue for all affected inputs, but also affects inputs that do not exhibit the abnormal behavior. Thus, while overfitting patches fail to resolve the bug, underfitting patches introduce new bugs. Both cases are undesired and may be avoided by providing more test cases to develop a patch that fixes the problem precisely [49].

A specific strength is the coverage of deep program states [9]. Deep program states are usually understood as program states that are only exercised when certain constraints over the program input are fulfilled. The more complex the constraints, the more difficult it is for coverage based fuzzers to reach them. Deep states may not only require complex inputs to be reached, but may hide behind repetitive code execution, only reaching the desired deep state when a certain number of actions / repetitions has been performed. This results in coverage based fuzzers not being able to differentiate program paths that reach a certain deep state and those that do not [9]. One such example for programs with deep states that cannot easily be differentiated by code-coverage are video games, for instance, Super Mario Bros. With the goal to complete a level, the input itself and state the transitions caused by them, are much more important than the source code covered. Traditional fuzzers such as AFL find it difficult to generate program inputs that complete a level [9].

Addressing this issue is complex as code-coverage is the primary factor that decides whether an input is interestinge. Without code-coverage, other measures are necessary to

guide the fuzzing. IJon [9] is an extension to AFL that seeks to guide fuzzing towards exploring deep states by having a human developer add special commands into an applications source-code. These commands, when executed by an input, alter what AFL considers interesting. For example, inserting an instruction into Super Mario Bros' source code that maximizes the players coordinates in the world, makes AFL favor inputs that achieve high x coordinates over those that achieve low x coordinates.

In this work a different approach is taken. Instead of relying on instrumentation and annotations, deep states are found and exercised by iteratively generating longer inputs, that reach deeper states. The approach of this work is a black-box approach, in contrast to other directed fuzzing approaches who are build upon grey-box and white-box fuzzers.

3.4. Input Reduction

Input reduction deals with the task of reducing a given set of elements to a minimal subset while preserving a set of properties. Input reduction minimizes a given program input while preserving certain aspects of the PUTs behavior. The minimizing delta debugging algorithm (minDD) developed by Zeller et al. [38][98] is a simple, universally applicable technique that has seen major research and development over the past decades. For the sake of simplicity the minimizing delta debugging algorithm will be referred to as delta debugging (DD) in this work.

The DD algorithm treats every program input as a list of elements, which are typically characters or lines [40]. It iteratively reduces this list to a 1-minimal subset, where removing any single element causes the abnormal behavior to disappear during test execution [97]. The algorithm is shown in Algorithm 1. The algorithm starts by splitting the list into two equal parts and testing the PUT on these subsets. If one subset exhibits the abnormal behavior, it becomes the new starting point for reduction and is split into two parts in the next iteration. If neither subset reproduces the abnormal behavior, their complements are computed and tested. If any complement exhibit the abnormal behavior, it is chosen as the new starting point, and the granularity parameter `n` which defines the number of subsets is either decreased by one or set to the length of the chosen subset. If no subset or complement reproduces the abnormal behavior, the granularity for the next iteration is doubled. The total granularity depends on all prior iterations since the last subset that exhibited the abnormal behavior.

Algorithm 1: Minimizing delta debugging algorithm [51], where $Test(...) = X$ symbolizes that a test exhibits the desired program behavior.

```
   Input: Program input input
   Output: Reduced program input
1  Function minDD(input):
2      n ← 2;
3      while true do
4          for i ← 1 to n do
5              if Test(subset_i) = X then
6                  input ← subset_i;
7                  n ← 2;
8                  continue while;
9          for i ← 1 to n do
10             if Test(complement_i) = X then
11                 input ← complement_i;
12                 n ← max(n − 1, 2);
13                 continue while;
14         if n < length(input) then
15             n ← min(length(input), 2 ∗ n);
16         else
17             break;
18     return input;
```

4. Related Work

In this chapter, an overview of related literature and existing tools is presented to provide necessary context.

Grey-box Fuzzing

Grey-box fuzzers have achieved widespread success in discovering vulnerabilities, crashes, and other abnormal behaviors as is evidenced by the number of studies done and tools developed [55][58][100][25][34][79]. These tools operate by randomly mutating seed inputs and collecting internal program data via instrumentation. This internal data is then used to evaluate test executions and adapt the seed inputs. In this way, grey-box fuzzers identify interesting inputs and generate new similar inputs with the goal of exercising new program paths, and covering more source code.

Although some grey-box tools provide support for binary-only PUTs [5][25][58], most require access to the PUT's source code for instrumentation. This often imposes narrow restrictions on the PUT's programming language and may enforce constraints on the language standard. For example, AFL version 2.51b cannot be applied to PUTs written in C++20, as it predates the standard and does not support it.

Additionally, grey-box fuzzers typically target non-interactive programs, as they often rely on mutation based input generation, which generally cannot generate sequences of user-events. There are some tools that target interactive programs, however, they are often limited to specific applications and are not available for generic interactive applications (see related work for `Fuzzing interactive Applications`).

In contrast to grey-box fuzzers which rely on internal program data and source code access, TimeFuzz is a black-box approach that primarily relies on domain independent data obtained by observing the PUT or evaluating its output.

Grammar based Fuzzing

Grammar based approach have emerged as a critical advancement in automated testing for when input formats follow complex syntactical rules. One of the earliest tools to emerged is LangFuzz [42], using code fragments extracted from existing test cases new inputs are generated. Other tools include Superion [89] which integrates grammars into AFL and Grammarinator [41] which uses ANTLR grammars [1] to generate tests.

As discussed in chapter 3.2.3, bugs are rare and generating program inputs that exercise those bugs randomly is difficult. Using a grammar to randomly generate inputs partially alleviates this issue, as only desired program inputs are being generated, but it does not change the fact that randomly generating inputs has low efficiency in finding bugs [55]. To improve grammar based fuzzing, a significant number of tools developed in recent years. Some approaches [8][22][32][42][89][96] combine grammar based fuzzing with mutation based approaches. These techniques use parsers and parse trees to understand the structure of initial inputs and then mutate them, for example, by replacing nodes in one parse tree with similar nodes from another. The specific mutations applied vary between approaches and are not limited to this example. The effectiveness of these

approaches varies, with one common limitation being the quality of the initial inputs and their parse trees. For example, the usefulness of subtree replacement depends on how well the initial parse trees represent the input space of the PUT [22].

The biggest problem with grammar-based approaches is the grammars themselves. Grammars are formal definitions of a program's input space and are often fairly complex. They also evolve during development, as new features are introduced or modified and old ones are removed. Developing and maintaining a formal definition of valid inputs is time-consuming and requires in-depth knowledge of the program [32]. Consequently, for the vast majority of programs such a formal specification does not exist which limits the applicability of grammar-based approaches. Even for programs that once had accurate grammars for research purposes, years of updates often render these grammars outdated and inaccurate.

Several approaches have been proposed to address the lack of proper input grammars. These aim to automatically infer (probabilistic) context-free grammars for a program's input language based on samples of valid or invalid inputs [14][43][88]. Most tools then use the learned grammars to fuzz the PUT, applying random generation, mutation based techniques, or other specialized strategies to generate inputs. Although these tools can be effective, the quality of the generated grammars heavily depends on the quality of the initial inputs. This impacts not only the grammar's accuracy but also its completeness, that is, how much of the actual input language it can reproduce.

Although the approaches discussed above address specific challenges in grammar-based testing, they are not considered in this work. While TimeFuzz does use grammars, the grammars used in the evaluation are intentionally simple. They do not contain complex syntactic or semantic features, instead, they generate streams of user events that correspond to button presses on a console, with no dependencies between events encoded in the grammar. As such, grammars are easy to create as they simply map to the buttons available on a console or a subset thereof. Additionally, existing tools typically use standard grammars, which cannot easily generate sequences of user events in a way that allows the tool to distinguish between them, as required for TimeFuzz. Although integrating TimeFuzz with tools that support more complex grammars is a promising direction for future work, it is beyond the scope of this study.

Symbolic Execution

According to a survey conducted by Baldoni et al. [11], symbolic execution has been developed to test whether a program can violate certain properties. Rather than relying on random input generation to test a PUT, symbolic execution systematically explores all possible execution paths in parallel without using concrete input values. Inputs are instead represented as abstract symbols, each of which stands for a class of concrete values that lead to a specific execution path. These abstract symbols are used to construct logical formulas that describe the conditions satisfied by each branch along an execution path. These formulas, along with symbolic expressions and associated data, are then processed by a satisfiability modulo theories (SMT) solver, which determines whether program properties are violated on those paths and whether the paths themselves are

realizable (i.e., whether the formulas are satisfiable). Due to the complexity of this analysis, symbolic execution requires full access to the PUT's source code in order to examine branch conditions, analyze control flow, and construct the corresponding formulas. This makes symbolic execution a white-box technique.

In addition to requiring source code, symbolic execution must also track all exercised execution paths and their associated symbolic values, which consumes significant memory resources [11]. As a result, exhaustive symbolic execution is unlikely to scale beyond small applications [11]. While the strategy of storing all discovered execution paths resembles the exclusion tree used by TimeFuzz, the underlying approaches differ fundamentally: TimeFuzz is a directed black-box method, whereas symbolic execution is a white-box approach that attempts to exhaustively explore the PUT.

Directed Testing

Directed fuzzing tools have undergone significant development over the past decades. Their origins can be traced back to symbolic execution, particularly to attempts to reduce the high resource consumption of symbolic execution by narrowing the areas of code explored. For example, Zhang et al. [99] proposed `regular property guided DSE`, a derivative of symbolic execution designed to find program execution paths that satisfy specified regular properties. Other approaches to targeted symbolic execution have since emerged, including techniques that rely on fitness functions [92]. A fitness function evaluates how close a particular execution path is to achieving a specific target or coverage goal [11]. In general it can be understood as a measure of closeness between an executed test case and a given target.

Building on the concept of targeting specific program parts, Directed Grey-box Fuzzing (AFLGo) was introduced by Böhme at al. [16] ith the goal of steering input generation toward predefined target program locations. AFLGo, a directed grey-box fuzzer, extends AFL by leveraging its instrumentation capabilities to focus fuzzing efforts on these target locations. It employs a distance metric that quantifies how close program locations exercised by a given input are to the target locations, and prioritizes mutations of inputs that minimize this distance.

Since the development of AFLGo, numerous newer directed fuzzing tools have been introduced such as IJon [9], SDFuzz [56], LibAFL-DiFuzz [75], or UAFuzz [72] and others. While these tools improve upon the basic directed fuzzing approach and some are even applicable to generic binary files, most are limited to specific programming languages of the PUTs and and rely on white- or grey-box techniques. In contrast, this work focuses on developing a black-box directed fuzzing approach. Because existing directed fuzzers depend on instrumentation, their methods are incompatible with black-box fuzzing.

Despite the differences between this work's approach and the current state-of-the-art directed fuzzing methods, the evaluation will compare this approach with IJon [9] to assess the performance of traditional grey-box directed fuzzing against this work's black-box method under comparable conditions. IJon is an extension of AFL that guides fuzzing toward exploring deep program states by allowing a human developer to insert special commands into an application's source code. These commands, when triggered by an

input, influence what AFL considers interesting. For example, inserting an instruction into Super Mario Bros' source code that maximizes the player's coordinates causes AFL to favor inputs that achieve high x-coordinates over those that achieve low x-coordinates.

IJon is not typical automated testing tool. Instead, it relies on a developer to provide annotations directly within the source code. This approach requires detailed knowledge of the program and the conditions necessary to exercise specific execution paths, thus demanding expert domain knowledge. While this dependency on human input can be seen as a limitation, the human-in-the-loop design offers developers fine-grained control over the fuzzing process and enables precise targeting. This increases their ability to analyze the PUT and address challenges that automated fuzzers often cannot overcome [9]. Additionally, IJon supports a variety of annotation functions beyond simple maximization, including hashing, value differences, and more, making it a versatile tool that assists developers effectively.

IJon is not the most modern approach to deep state fuzzing that identifies important source code locations and annotates them to guide fuzzing. SandPuppy [74] improves upon IJon's method by automating the annotation process. It identifies state-representative variables in a program and inserts annotations similar to those used by IJon, but without requiring human intervention. However, since the source code for SandPuppy was not available, it could not be used as an evaluation target in this work.

Search-based Testing

According to Silva et al. [84] search-based testing techniques use meta-heuristics to generate and select program inputs as test cases. These techniques employ fitness functions to assign values to individual test cases and apply meta-heuristics to guide the selection process. The most prominent meta-heuristics used are Simulated Annealing [50], inspired by concepts in metallurgy and Genetic algorithms [44], based on biology and genetics. Meta-heuristics are mathematical methods applicable to a wide range of problems. In software engineering, they have been used for test data generation, test case selection, test case prioritization, and more [84].

In search-based testing, meta-heuristics are used both to generate new inputs from known inputs and to select which inputs to generate from. Depending on the specific meta-heuristic applied, the strategies and results can vary widely. Nonetheless, these methods are effective in reducing the number of test cases compared to purely random testing.

Although meta-heuristics are mathematical approaches for general optimization problems, their applications in automated testing are typically white- or grey-box methods [65]. These approaches track internal program data and execution paths and are often directly involved in input generation [33][61].

While TimeFuzz also uses optional score values that fit the description of a fitness function, these scores are not directly connected to input generation. Instead, they serve to select promising inputs. Although applying meta-heuristics to this selection process could be possible, it will not be pursued in this work for the sake of simplicity.

Input Reduction

Since the introduction of the original DD algorithm by Zeller et al., numerous derivative algorithms have been developed. Some specialize in targeting specific program input languages or inputs with particular structures. Examples include C-Reduce [80], Perses [85], Hierarchical Delta Debugging (HDD) [70], and others. While C-Reduce focuses on reducing C programs used as input for C compilers, Perses and HDD concentrate on inputs with tree-like parse trees specific to the PUT's input language. Other derivatives, such as ProbDD [87], aim to reduce the number of tests executed by DD by introducing stochastic methods into test selection and execution.

Although a variety of specialized algorithms exist, there are no known approaches that significantly improve upon the original DD algorithm for general domains. EEven algorithms like ProbDD, which use stochastic methods to identify important input elements and remove irrelevant ones early in the reduction process, have notable limitations. While ProbDD can achieve substantial speedups compared to the basic DD algorithm, it assumes dependencies between elements of the program input [87]. As the authors note, applying ProbDD to inputs with a large number of interdependent elements may lead to worse performance than the original DD.

Input languages for interactive programs are typically defined by the possible user actions at any given moment, making many of these languages quite permissive. While detailed grammars can sometimes be created for certain interactive applications, inputs for video games are often represented simply as sequences of button presses available on the respective console. This simplification is due to the complex internal state and the time-dependent relationships between interactive events. Because of these interdependencies and the generally broad input grammars, none of the approaches mentioned above can be universally applied to interactive programs.

There are other input reduction algorithms not based on Delta Debugging that specifically target the reduction of input sequences for interactive programs. Yan et. al. [95] propose an input reduction approach for test sequences in GUI applications. Their method categorizes events or event sequences based on their impact on the PUT's GUI, removing those that do not contribute to reproducing abnormal program behavior. They identify two types of removable events: (1) nop-events, which are unrelated to any interactable GUI element or are invalid for that element, and (2) redundant events, which do not contribute to testing such as selecting multiple radio buttons in succession, where all but the last selection can be removed. Although this approach is effective for structured GUI applications with clear, tree-like interfaces and could be adapted to input languages instead of GUI elements, many interactive programs do not have a GUI. This absence makes it difficult, if not impossible, to identify ineffective or redundant events, since immediate program behavior may be unobservable or unanalyzable without access to internal program data.

Other input reduction tools targeting GUI-based programs often focus on associating user events with GUI states [46], while some approaches adapt Delta Debugging to handle non-determinism [19]. The former, and similar methods, require access to GUI state data, which may not be available on all platforms or for all applications. The latter specifically addresses non-deterministic applications which TimeFuzz does not target.

Overall, the basic DD algorithm remains the only universally applicable input reduction technique. Other algorithms cannot be applied to generic applications and thus cannot be included in TimeFuzz, although the work by Clapp et al. [19] may remain for future work.

Fuzzing interactive Applications

Fuzz testing interactive applications has been a major topic for a long time, resulting in many specialized automated testing tools. Examples include network protocol fuzzers like SNOOZE [13], server fuzzers such as STATEAFL [71], and database fuzzers like DynSQL [48], among others.

While these tools are effective within their domains, their implementations and approaches are often tightly tailored to their specific target applications. In contrast, TimeFuzz aims to provide a general fuzzing framework adaptable to a wide range of applications through the use of custom input grammars, oracles, and APIs where needed.

GUI Testing

Graphical User Interfaces (GUI) are unavoidable in modern software applications, as they facilitate interactions between users and the program [81]. GUI testing is an active field of research within software engineering, focused on verifying that the interface displays correct information and that user interactions lead to the expected program and GUI states [81]. Since traditional fuzzing tools cannot directly interact with GUIs, dedicated GUI testing tools are often essential for automated testing of user interfaces.

While GUIs simplify user interaction, they also add complexity to software testing [67]. Beyond testing core functionality such as file operations and communication protocols, the GUI itself must be validated, as its correctness significantly impacts overall software quality [12]. The field of GUI testing primarily concentrates on verifying GUI behavior, although some tools integrate additional testing and analysis techniques.

GUI-based applications are a subclass of interactive programs, taking sequences of interactive events as their program input. As such, GUI testing ca be defined as the testing of a GUI based application solely by performing a sequence of interactive events on the GUI [17]. The key distinction between GUI-based software and non-interactive programs, is an extension of the basic architectural differences described in Chapter 2. Providing GUI-based software with interactive inputs is non-trivial. There is no universal API to access GUI elements, dispatch targeted events (e.g., button presses), or observe and interpret changes in the GUI. This lack of standardization is reflected in the wide variety of GUI testing tools tailored to specific platforms or programming languages. For example, many tools are designed exclusively for the Android platform [10][57][59][62][90], leveraging the Android SDK's capabilities to expose GUI structure and elements. Similarly, other tools have been developed for desktop applications, typically targeting specific languages such as Java [33][66] or focused on particular domains, such as browsers and webpages [6].

GUI testing tools are typically limited in their applicability to specific programming languages and platforms. Nonetheless, they are the closest in spirit to this work's approach

of representing program inputs as time series. While no tools known to the author explicitly model inputs as time series, many acknowledge the importance of event timing and interdependencies between user events. These concepts are often used implicitly. For instance, Monkey [59] generates a stream of random user events, which can be considered a time series. However, since Monkey lacks a feedback loop in its fuzzing process, it ignores time related information that could otherwise be used to steer fuzzing more effectively. Other tools, such as DynoDroid [60], incorporate timing information in a more structured way. DynoDroid monitors when an application registers for specific user events, using this data to infer which events are available in a given GUI state and to decide which event to generate next. However, even in such cases, the focus is not on the time series nature of inputs but rather on state-aware event selection.

One GUI testing tool that explicitly interprets program inputs as time series of user events is ComboDroid [90]. ComboDroid generates program inputs by concatenating short sequences of user events, referred to as use cases, to form longer input sequences. A use case is defined as a sequence of user events that transitions the PUT from one idle GUI state to another. Each use case has a designated start state and end state. when ComboDroid generates program inputs, it concatenates use-cases where the end state of one use-case is the starting state of another use-case. This approach takes advanced information into account such as data-flow dependencies between events. By enumerating valid combinations of use cases, ComboDroid generates new program inputs.

ComboDroid is capable of generating long program inputs that can exercise application functionality which may not be reachable using other tools. However, its effectiveness depends on the availability of use cases, which must either be manually provided by the user or extracted automatically from execution traces of existing use cases. While this requirement may not appear overly restrictive, both approaches introduce limitations. The first requires users to manually supply as many use cases as possible, which not only demands considerable effort but also assumes that the user has sufficient knowledge of the PUT to capture all relevant functionality. The second approach relies on the user providing execution traces that sufficiently cover the basic behavior of the application. If key functionality is not exercised in these traces, ComboDroid may fail to generate inputs that reach those areas of the program. Furthermore, ComboDroid operates in a feedback loop, analyzing execution traces of previously generated inputs to discover new use cases. This design requires the availability of executions traces, and especially that execution traces may be collected for any executed test case. As such, the approach is limited in its applicability to systems that provide such information. In practice, this restricts ComboDroid's applicability to the Android platform, as most other systems do not expose the level of information necessary to support this kind of feedback-driven use case discovery.

ComboDroid generates program inputs by concatenating smaller input sequences and is, in its basic approach, similar to the one proposed in this work. However, while the approach is similar, ComboDroid relies on a pre-defined set of short input sequences (use cases) to construct longer inputs. In contrast, TimeFuzz extends prior generated program inputs randomly. As a result, ComboDroid is limited to generating input sequences that can be composed from known use cases, whereas TimeFuzz can create arbitrary input

sequences without such constraints.

Other tools related to TimeFuzz include SimplyDroid [46], which applies Hierarchical Delta Debugging (HDD) to reduce event traces. However, this approach depends on access to detailed GUI state information, which is typically only available on Android. Moreover, as discussed previously, HDD is not suitable for the goals and constraints of this work and cannot be applied here.

While many tools developed for GUI testing implicitly treat program inputs as time series, since user interactions are inherently sequential, no prior work explicitly uses this to improve their input-generation algorithms. Similarly, no prior work investigates the effects of simple generation improvements based on time series. There is currently no GUI testing tool known to the author that focuses on generating program inputs specifically aimed at reproducing particular program behavior. Instead, most tools emphasize maximizing code coverage or inducing program crashes [81].

Testing and solving Video Games

There are relatively few prior works that directly address the fuzz testing of video games. One such study introduces an experimental framework for exploring software fuzzing techniques for roguelike games [27], but it does not present actual fuzzing approaches. Other works apply reinforcement learning and autonomous agents to test video games [7], focus on domain-specific challenges such as game balancing [77], or examine specific sub-genres like adventure games [78]. While some general fuzzing tools, such as IJon and SandPuppy, have been evaluated using video games, they are not specifically designed for them.

Overall, few research efforts explicitly target video games as a testing domain, including simple and often deterministic retro games such as Super Mario Bros. [52] or Sonic the Hedgehog [83]. While scientific work on fuzzing video games remains limited, there are AI-based approaches that aim to automatically play or solve video games. One notable framework supporting such efforts is OpenAI Gym-Retro [73], a Python-based emulation environment capable of running a wide range of retro video games, including the aforementioned titles. In combination with NEAT [3], an evolutionary algorithm for the creation of artificial neural networks, this allows for video games to be solved. However, these approaches typically require access to internal game data (e.g., game state variables or memory content) in order to evaluate user-events and their outcomes.

All currently available techniques either rely on internal program data or are limited to specific subsets of video games, often with strong platform or domain constraints. TimeFuzz follows a more general approach targeting general interactive applications. It focuses on evaluating the effectiveness of the basic approach of generating iteratively longer inputs, instead of relying on learning which can often be complex and opaque [82].

5. Framework

In this work a new fuzzing framework for directed deep state fuzzing of interactive programs is presented. Even though there are other effective directed fuzzing approaches, those approaches either belong into the white-box or grey-box fuzzing categories. In this work however the focus lies on black-box fuzzing.

Even though black-box fuzzing can be considered the least effective and most time-consuming fuzzing method, most applications cannot be fuzzed with white- and grey-box approaches as their source code is either unavailable or fuzzing tools are not available on the applications target platform. Black-box fuzzing is the only approach that can fuzz arbitrary target applications, including software executed on emulators, without modifying the application or the emulator. As black-box fuzzing does not have access to a programs internal data, many fuzzing approaches cannot be applied in a black-box context, including directed fuzzing approaches. Additionally, there are no fuzzing tools known to the author that apply black-box fuzzing to interactive programs without being limited to specific target platforms such as Android.

This work presents a black-box fuzzing framework which collects time related data about tests and uses them in conjunction with a domain specific oracle to steer input generation. TimeFuzz generates inputs that satisfy the oracles constraints while considering the length of the generated program inputs and optionally a primary and secondary score value computed by the oracle.

TimeFuzz is designed to be applicable to a broad spectrum of applications as PUTs are not constricted to a specific programming language or platform as long as TimeFuzz can be implemented for the platform. The goal is not to provide a tool that can communicate with all interactive programs itself, but to provide a framework that handles input generation and analyzes collected time related data, while allowing intermediary APIs for communication with the PUT when necessary.

TimeFuzz implements a new generational fuzzing technique that takes the test evaluation of the user defined oracle and collected time related data into account and generates input sequences based on a given input grammar, the user defined oracle, and the PUT itself. Additionally, TimeFuzz allows a user to specify parameters regarding thread management, generation sizes, and DD in a custom configuration file.

This chapter presents the general approach of TimeFuzz. The first part of this chapter gives an introduction to the workflow of TimeFuzz and the overall generation strategy. Further, the foundational details of the implementation as well as the input expected by the framework and the output provided after a fuzzing session are discussed. After the basic details have been introduced, the chapter then provides an explanation of the frameworks computational architecture and data storage capabilities and continues with the technical details of test execution and the collection of response times and point of failure s of tests. The chater will finish by outlining TimeFuzz's use of an exclusion tree as a measure of reducing the total amount of tets which are executed during fuzzing sessions. While this chapter focuses on the technical aspects of the framework, the following Chapter 6 will discuss the input generation algorithms and DD algorithms used to generate and minimize program inputs.

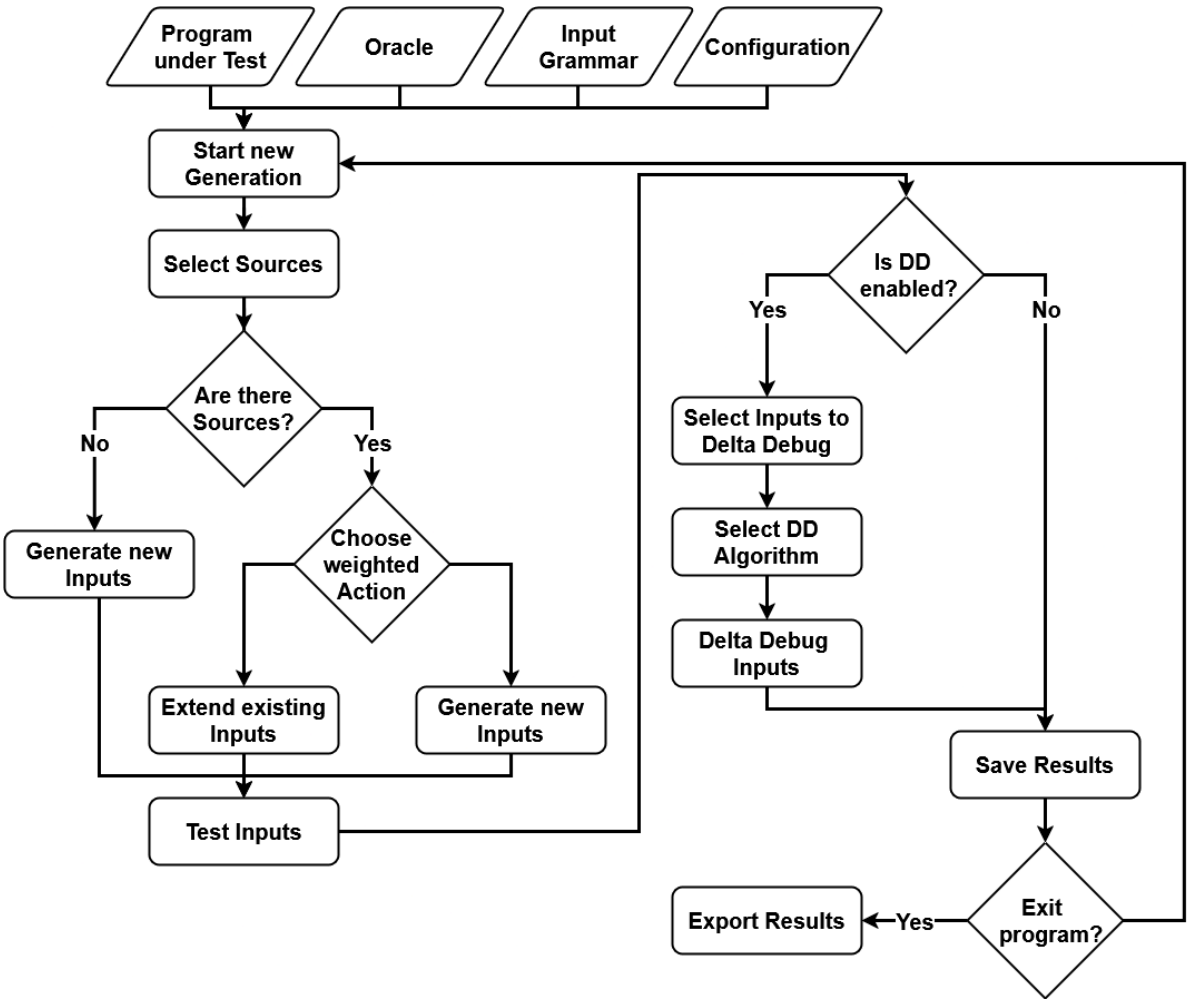


Figure 5: Execution flow of TimeFuzz

5.1. Workflow of TimeFuzz

The overall workflow of TimeFuzz is shown in Figure 5. The program begins by starting its first `generation`. A generation consist of mainly two phases: (1) Generating new inputs and testing them, and (2) applying DD to the best inputs generated in this generation.

The first phase always begins by selecting a user defined number of source inputs that will be extended in this generation. The inputs are either chosen according to their score values based on their length. The user can decide which value to maximize by adapting the configuration file. No sources exist in the first generation.

After the source inputs have been determined, new program inputs are generated. If there are no sources, which happens only in the first generation, new inputs are generated from the PUTs grammar completely at random. If there are sources, the generation of new inputs randomly alternates between generating new inputs from the grammar and extending one of the source inputs, with the probabilities for either action being user defined. After an input has been generated, the PUT is executed on the input. The

output, exit code, response times, and runtime are observed and handed to the oracle which computes the result (passing, failing, or unfinished) and optionally a score value. If the test result is passing, the input is minimized using DD, and will be exported once TimeFuzz exits.

The second phase begins after a user defined amount of inputs has been generated. If the oracle provides a score value for the tests, a user defined number of generated inputs will be chosen for minimization. The selection proceeds the same way as sources are chosen at the beginning of a new generation. These input will then be minimized with DD, while preserving their score values. If the user has deactivated DD or the oracle does not provide a score value that can be used to measure an inputs progress, the second phase ends immediately.

After both phases have been completed and all instances of DD have finished, the results so far are saved. A custom data format is used to store all inputs generated as well as their test and ancestry information. The savefile can later be used to continue the fuzzing session from the current point in time. This is useful in case TimeFuzz crashed, the environment crashed, or TimeFuzz was run with a timeout and was not able to produce satisfying results within the allotted amount of time. Once the results have been stored, TimeFuzz checks its end conditions and decides whether it should stop at the point reached. If it ends, results are exported, including (1) information about DD instances, such as size, runtime, and results, (2) information about all generations that have been run, including the number of generated inputs, inputs produced by DD, and the runtime, and (3) information about passing inputs and the top K inputs generated, including the inputs themselves, command line arguments, and test results. If the program does not end at this stage, a new generation is begun and new sources are being selected.

The selection of source inputs for any generation but the first is determined by the outcome of the second phase of the previous generation. If no inputs were minimized with DD, sources are selected as mentioned before. However if inputs were minimized, the sources are not directly selected according to the users settings. Instead, they are selected equally among the inputs generated by the DD instances. If all source inputs are selected from one DD instance because the score of the generated inputs is much better than any other input generated in the last generation, there is a risk of restricting the search space to one particular program path. Since inputs generated by the same instance of DD tend to be similar, they might follow similar or identical program paths. In the worst case this might lead to path constriction as sources from one specific execution path may be selected in each generation. This could result in insufficient exploration of the PUT or may lead to a dead end. As this is a black-box approach, there is no way to detect such issues directly, and therefore precautions have to be taken. To mitigate this, when selecting sources after minimizing with DD, either inputs generated or extended using the PUTs grammar are chosen or the best inputs generated by DD are selected equally among the different instances. Once the source inputs haven been selected, a new generation is begun.

5.2. Implementation Details and Customization

TimeFuzz has been implemented in C++ using the modern C++20 standard and has compiler support for gcc, clang, cl, and clang-cl. It is available for both the Windows and Linux operating systems, even though the supported functionality varies slightly because of the differences in the operating systems. TimeFuzz was designed as a scalable general fuzzing framework that can not only scale input generation to arbitrary amounts of threads but can also interact with a wide variety of different PUTs. Additionally, TimeFuzz is able to save the current progress of a session at any point in time and resume the session from there at a later date. incorporating this, a computational architecture has been designed and implemented which will be introduced in subChapter 5.3 The source code of TimeFuzz can be found on the authors GitHub page[4].

To develop a tool that can interact with a variety of PUTs, a user must be able to customize any interactions between the framework and a PUT. TimeFuzz provides this functionality in the form of a build-in lua [2] engine that can read and execute arbitrary scripts, any by providing an extensive configuration file that specifies enabled features and customizes their runtime parameters.

TimeFuzz requires six files as input, including the configuration file.

- An **executable** file, which can either be the PUT itself or an AI that forwards generated inputs to the PUT and returns the PUT's output to TimeFuzz. The PUT must be deterministic, i.e. if the PUT is executed with the same input multiple times, it must always produce the same result.

- A **grammar** that either describes the input language of the PUT or that generates inputs that can be translated into valid inputs for the PUT.

- A **configuration file** specifying the path to the PUT, execution customization, file paths for the other required files, and settings for input generation, DD, and savefiles to stor progress. It also includes options to configure the number of computation threads and concurrently executed tests.

- A lua script that returns the **command line arguments** the PUT is executed on. This allows for dynamic adaption of command line parameters based on the generated input.

- A lua script that computes **script arguments** that optionally adapts generated inputs to the PUT's input language. This is only used, when inputs are passed to an API or written to the PUT's standard input, rather than being delivered as individual user-events.

- A lua script that implements the **test oracle** to evaluate observed data, decide the test outcome, and calculate the optional score values.

[4]https://github.com/muenchk/TimeFuzz

The oracle used by TimeFuzz does not entirely adhere to the common definition of an oracle: A function that decides whether a set of constraints over the PUT's behavior is satisfied or not. Instead, the oracle is given access to all information collected about the generated input and the associated test execution, including output, exit code, exit reason (no issues, timeout, start problems, excessive memory consumption, etc.), command line arguments, reaction times to individual user-events (if available), and the generated input sequence itself. From this information, the oracle is expected to compute the result of the test case, which can be passing, failing, unfinished, or undefined, if the result cannot be determined or there was an issue with the test execution. Additionally, the oracle can decide to repeat the test case if necessary, decide the score of the PUT, and can set the number of sequence elements that have been executed by the PUT, for instance, if the PUT was started using an API. TimeFuzz supports a primary and a secondary score value that are used to steer source selection and DD. The oracle can provide neither of those scores, one of them, or both or them. Additionally, it can provide an individual score value for each user-event in the generated input. This allows TimeFuzz to use apply the `Score Progress` for DD algorithm, which will be introduced in 6.2

TimeFuzz collects and stores a significant amount of information about generated program inputs and tests. This information is also provided to the user at the end of a session. TimeFuzz exports results to a configurable directory. The results include general information such as the number of generations, the total amount of generated tests, the total amount of tests generated during DD, the number of tests that were generated but not run (see subChapter **??**), the average number of tests per minute, and the internal runtime of TimeFuzz. The internal runtime is not the wallclock time of a session but instead excludes all bookkeeping activities such as saving the session to allow for an exact measurement of the time spent generating and executing tests. In Addition to general information, TimeFuzz exports statistics for all individual instances of DD and the individual generations. For generations, TimeFuzz also exports information about the best inputs generated in this session. Finally, TimeFuzz exports information about the top 100 inputs in the respective categories of primary score, secondary score and overall input sequence length, as well as command line arguments, script arguments, and a complete dump to of the entire generated input, so that the user can run the tests outside of TimeFuzz for evaluation or visualization. All this information, excluding the parameters needed to execute the exported tests, is provided in the form of csv tables.

The functionality implemented to provide additional data and to allow restarting the tests from a given point is supporting the research even in cases when long running studies are needed and interruptions like maintenance operations to the test environment are due. This is a unique feature of TimeFuzz, at least, as the author is not currently aware of any other fuzzing tool providing a similar functionality.

5.3. Computational Architecture and Save Files

During the development of TimeFuzz a major focus has been on developing a performant and scalable framework that would be able to seamlessly scale to large or computation intensive tasks. This was maintained during the work for both to the computation itself as well as the simultaneous execution of tests. Another focus of this work was to create regular backups of the generated results in the form of savefiles. These savefiles contain any inputs generated and information collected and can be loaded to resume a prior session. Loading a savefile allows for the continuation of the work at the exact point of time were the savefile was created without any loss of data.

Save files are useful in a multitude of situations. They can be used to evaluate the exact progress at a specific point in time, provide data loss prevention, in case the tool itself crashes or if there is an issue with the machine the tool runs on. A savefile allows a user to perform additional data analysis after the session has ended in case a more detailed analysis of certain results is necessary.

The savefiles store not only session information, inputs, and test results, but also any function that is currently waiting to be executed. When creating a savefile, TimeFuzz waits until all running functions have finished and then saves all data and all waiting functions to the savefile. On loading the savefile, the session continues with running the waiting functions and the user does not notice any difference between running TimeFuzz continuously and resuming an interrupted session.

This requires that not only all data must be serializable but also that the functions and algorithms used in TimeFuzz must be not be executed sequentially from a top-level function. Instead, functions must be split into reasonable work packages that are serializable, thread safe, and are executed using a task queue instead of a top-level function.

To implement a modular and extensible architecture that allows for the introduction of new data classes and functions without major code changes, TimeFuzz implements all data classes as inheritors of a common interface which provides functions for serialization and deserialization in the form of Read and Write methods. The class structure of TimeFuzz is illustrated in Figure 6. As the figure illustrates, all data classes inherit the `ReadData` and `WriteData` functions from the interface `Form`. This includes any and all classes in TimeFuzz including Inputs, Tests, the TaskController that controls task execution and scheduling, the ExecutionHandler that executes the tests, the Grammar of the PUT, the information about generations, and general session data, such as the top K generated inputs, the total number of inputs and any other type of generic internal information. The interface `Form` provides additional common functionality such as memory management functions, unique object identifiers.

All instances of classes derived from `Form` are stored in a single large hash table, which allows fast access to objects as well as a simple implementation of saving and reading savefiles. Instead of gathering all data for the savefile from multiple locations, the algorithm writing a savefile can simply iterate over all objects in the hash table and write them to disk. Similarly, the algorithm reading a savefile can simply read the objects from disk and add them to the hash table. The links between data objects, such as between

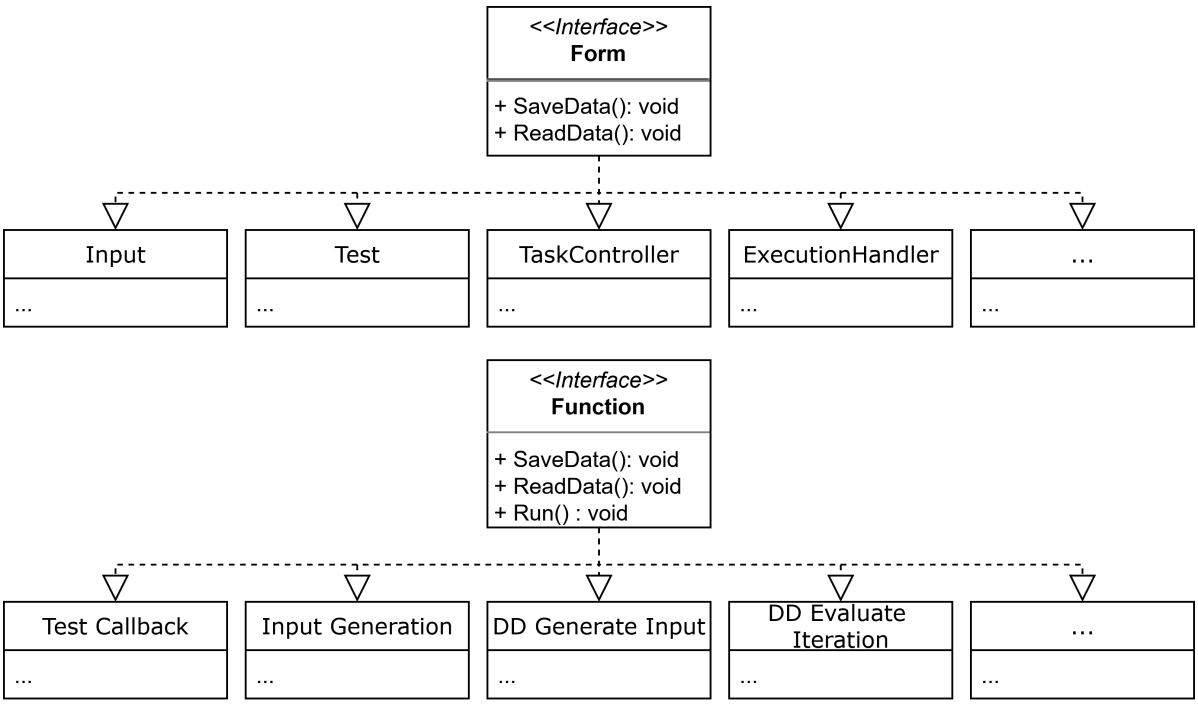


Figure 6: Inheritance structure of TimeFuzz's classes

inputs and corresponding tests, are implemented as references to the unique ID provided by `Form`. To create new data classes, a developer only has to derive a new class from `Form` and implement its functions; anything else will be taken care of automatically.

A similar design is in place for functions. There are two types of functions in TimeFuzz. Functions on data objects that provide data access or some operation on the data and are implemented directly on the objects, and functions that represent major algorithms or parts thereof. Each of the latter functions is implemented as function `Run` on a unique class. These unique classes store function parameters and are derived from the interface `Function` as seen in Figure 6. The interface provides read and write functions that save function parameters and allows the function to be stored in the savefile. Functions implemented as runnable classes are used for test case evaluation, input generation, DD input generation, evaluation of DD iterations, and any other computation intensive tasks.

To execute one of these major functions, a new instance of the respective class is created and added to a task queue on the task controller. The task controller manages a thread pool which can be configured via the configuration file of TimeFuzz. Each thread waits for new tasks to arrive. The tasks are categorized into three categories at compile time: `light tasks`, `medium tasks`, and `heavy tasks` according to the time required to run them. While `light tasks` include test callbacks, `medium tasks` include input generation for DD and source selection at the end of a generation, and `heavy tasks` refer to computation intensive operations such as input generation. The tasks of the thread pool execute tasks in order of their estimated time consumption with `light tasks` being executed first.

Once a task has been started it cannot be interrupted until it has been finished. Consequently, the algorithms implemented in TimeFuzz are split into smaller work packages which can complete within a reasonable amount of time. If there are follow up tasks to perform, a new task can simply be added to the task queue. This segmentation of functions allows TimeFuzz to pause the task controller within a reasonable amount of time to save the ongoing session and allows the algorithms to scale to large thread pools. An example to this segmentation is this works implementation of DD. DD is separated into four parts, (1) deciding which inputs to generate, (2) the individual generation of the inputs, (3) the individual evaluation of the inputs after a test has been executed, and (4) the decision on which input should be divided into subsets in the next iteration (see Chapter 6.2).

When saving the current session, TimeFuzz freezes the task controller and waits for all running tasks to be completed. After the task controller has been frozen, the savefile is written, with the waiting tasks being saved as data on the task controller. Upon loading a savefile, the tasks are added back to the task queue and the task controller resumes execution.

Using this architecture, the current status can be saved and loaded at any point in time without loss of data. This allows to provide a user a flexible way to save the work being done. Available options include periodic saves at the end of each generation, after a certain amount of time passed, or after a certain amount of tests executed.

5.4. Executing tests and obtaining data

TimeFuzz collects a large amount of data about tests. This data is obtained either from the test oracle or by observing the PUT during its execution. TimeFuzz implements an execution handler as a dedicated class that handles test execution and data collection.

The execution handler handles all test execution and consists of a queue of waiting tests and an array of running tests. To facilitate testing, the class uses three major functions: (1) a function that adds new tests to the waiting queue, (2) a function that starts tests, and (3) a function that writes input to running tests, reads output from running tests, and monitors their ressource usage and runtime. The third function also handles aborting any tests that violate resource and time constraints, and wraps up finished tests.

TimeFuzz starts tests by using either `CreateProcess` on Windows, or using `fork` or `posix_spawn` on Linux to create a child process. When a child process is created, its standard input, standard output, and standard error are redirected to input and output pipes opened by TimeFuzz. On Windows, the read end on the child process's input pipe and the write end of the output pipe are automatically closed by the system. On Linux, however, the ends of a pipe must be closed explicitly by TimeFuzz. Accordingly, the write end of the child process's output pipe is closed, while the read end of the input pipe is kept open. As both ends of the input pipe are accessible, input can be written to the child process and the pipe can be monitored to determine whether it had been emptied. In this way, it can be observed whether the PUT has read the input that was written to the pipe.

The basic algorithm used to monitor running tests is shown in Algorithm 2. It relies

Algorithm 2: Handle running Tests

```
Input: TaskController controller, Timeout timeout, Event Timeout eventtimeout, Enable
       Events eventsenabled
1  Function HandleTests(controller, timeout, eventtimeout, eventsenabled):
2      while abort == false do
           Calculate time difference between iterations to calculate accurate
           timeouts
3          if size(runningtests) == 0 then
               wait for new tests to start
4          for i ← 1 to size(runningtests) do
5              test ← runningtests[i];
6              test→output.append(ReadOutput(test));
7              if HasOverusedRessources(test) or GetRuntime(test) > timeout or
                GetRuntimeEvent(test) > eventtimeout then
8                  KillProcess(test);
9                  controller→AddTask(test→callback);
10                 continue;
11             if eventsenabled == true then
12                 if HasReadInput(test) then
13                     WriteNextEvent(test);
14             if HasFinished(test) then
15                 runningtests→remove(test);
16                 controller→AddTask(test→callback);
```

on a number of OS-specific helper functions to monitor resource usage, read output, and write input. The helper function `HasReadInput` is only available on Linux and only when user-events are written individually to the PUT. It monitors whether the PUT has read the user-event. It takes advantage of the Linux-specific behavior that permits multiple processes to shared read access to a pipe. This feature allows the time between writing a user-event and the PUT reading it to be monitored. This does not necessaruly represent the PUT's actual reaction time, as no domain independent method exists to measure it directly. Instead, a relatively accurate approximation of the PUT's reaction time can be obtained by measuring the it takes the PUT to read the next user-event written.

The ability to write user-events to the PUT individually, combined with confirmation that each event has been read, allows the precide point of failure in a generated input sequence to be identified. The number of input events written to the PUT before termination can be tracked, enabling the generated input to be trimmed to its executed length. If TimeFuzz is running on Windows or if the PUT does not support reading individual user-events, this information must instead be provided by the oracle.

5.5. Exclusion Tree

Obtaining the executed length of program inputs combined with the knowledge that the PUT is deterministic, provides with the information that if the result of a program input

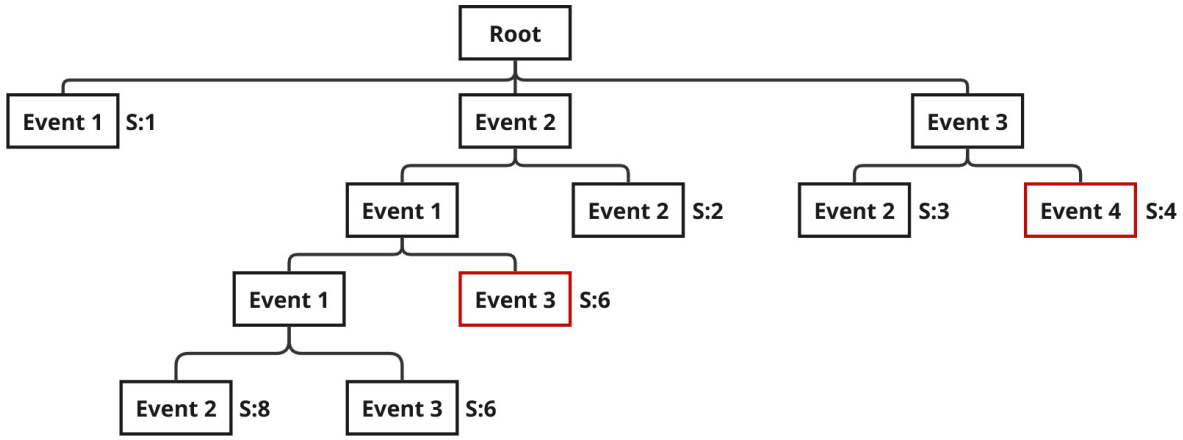


Figure 7: Example exclusion tree used in TimeFuzz. Black-bordered nodes represent normal nodes, while red-bordered nodes indicate final nodes. For each leaf node which corresponds to an executed input the associated score is displayed.

is passing or failing (i.e. absolute) any other input that has this input as a prefix, must also yield the same test result. Similarly, if the result of an input is unfinished, then any extension of that input must be executed to a length at least one user-event longer than the original input.

This absoluteness of test results is used to determine whether newly generated inputs must be executed or if their outcomes are already determined. To this end, an exclusion tree is implemented in which the results of all generated inputs are stored. This includes passing, failing, and unfinished inputs. Only undefined inputs are excluded as their result may not be absolute.

The exclusion tree contains one path for each generated input. Each path contains normal nodes for every user-event in an input and a final node for the last executed user-event in either a passing or failing input. A final node represents an absolute result whereas a normal node does not. Whenever a new program input is generated using the PUT's grammar or DD, the exclusion tree is checked for the longest path that is a prefix of the input. Take the exclusion tree shown in Figure 7 as an example. Nodes with black borders are normal nodes, for which either no input has been executed so far or the input's result was unfinished. Nodes with red borders are used to represent final nodes. For each final node an already executed input exists that was either passing and failing. Each node in the tree represents a specific user-event. A path in the exclusion tree, from the root to a leaf node, contains all user-events in a generated input, ordered from the first to the last executed user-event.

A path contains normal nodes for each user-event in an input and a final node for each last executed user-event in a passing or failing input. A final node represents an absolute result while a normal node does not. Whenever we generate a new program input from the PUTs grammar or with DD, we check the exclusion tree for the longest path that is a prefix to the input. Take the exclusion tree shown in Figure 7 as an example. Nodes with black lining represent normal nodes, for which either no input has been executed so far,

or the inputs result was unfinished. The nodes with red lining represent final nodes. For each of these final nodes there exists an input already executed that was either passing or failing. The nodes of the tree each represent one specific user-event. A path in the exclusion tree from the root to a leaf node contains all user-events in a generated input in order from the beginning to the last executed user-event.

If there is not path in the exclusion tree that is a prefix to a newly generated input, the input is tested because its result cannot be determined. In the exclusion tree shown in Figure 7, this applies to any input that begins with `Event 4`. If there is a path in the extrusion tree that is a prefix to the newly generated input, and the last node in the path is a final node, the result of the input must be passing or failing and the input does not need to be tested.

Contrarily, if the input is a prefix to any path in the exclusion tree, its result must be unfinished because there exists a viable extension. If the path matches the input exactly, an identical input has already been generated and does not need to be tested again. If the path does not match exactly and the input was generated from the grammar, it is tested, despite the result being know, because a score may be assigned by the oracle, and the input could become relevant for generational source selection.

An example OF such an input in the exclusion tree shown in Figure 7 is the input sequence `Event 2`, `Event 1`. Although this input must be unfinished, it might receive the same score as its extension `Event 2`, `Event 1`, `Event 1`, `Event 3`, and could therefore be shorter while preserving the score.

If the input was generated by DD instead, the handling differs slightly. when DD minimizes an input while preserving its result, the input does not need to be tested, as it cannot reproduce the original outcome. If the DD algorithm minimizes an input while preserving a score value (target value), any input for which an extended input with a lower score than the target value exists can be excluded from execution. For example, consider the input sequence `Event 2`, `Event 1`, `Event 1`, and `Event 2`. DD minimizes this sequence while preserving the score value 8. The input `Event 2`, `Event 1` is generated and compared against the exclusion tree. As there is no path that exactly matches the input, it could be tested. However, the shortest extension in the exclusion tree `Event 2`, `Event 1`, `Event 3` has a score of only 6. Since the score value is monotonic, the score value for the generated input has an upper bound of 6 and cannot meet the target score of 8. As a result, the input can be excluded from execution. This type of exclusion will be referred to as `exclusion by approximation` as opposed to `prefix exclusion`, in the remainder of this work.

Whenever a generated input has been tested and its result has been determined, it is inserted into the exclusion tree by extending the longest matching path. As more inputs are generated and tested the size of the exclusion tree grows. This can become problematic for PUTs that require long input sequences, as the number of nodes in the exclusion tree may grow exponentially with input length in the worst-case. To mitigate this, TimeFuzz enforces a user defined maximum depth, enabling a trade-off between then amount of information stored and the total memory consumption of the exclusion tree.

6. Algorithms

In the previous chapter, the framework itself and its technicalities were introduced. This chapter provides an overview to the algorithms that were implemented on top of the basic architecture. The chapter begins by discussing the grammar for input generation and the custom algorithms for deriving inputs from the grammar and for extending inputs. The implementation of the DD algorithm and the newly developed `Score Progress` DD algorithm will be explained. The chapter finishs with illustrating the specifics of selecting inputs to minimize after each generation and the source selection for the next generation.

6.1. Input Generation

As already discussed in Chapter 3, input generation is probably the most important part of a fuzzer. Fuzzing tools can use different approaches to input generation. The implementation of TimeFuzz applies a grammar based approach to generate valid program inputs. The grammar for the PUT's input language, must be able to generate sequences of user-events easily and reliably. In order to support the explanation of the approach selected for the framework, the basic formal concepts of grammars will be discussed and the definition of `time grammars` gets established. It will be the foundation of the description of the algorithms for input generation adopted in TimeFuzz.

6.1.1. Time Grammars

Grammars as defined in section 3.2.2 can reliably define input languages for all known computer programs. There are grammars of the above form that describe input languages to both non-interactive and interactive programs. However, such a grammar can only produce whole words which are to be interpreted by the target program. TimeFuzz cannot use such a grammar, as there is not enough information to identify individual user-events in the generated words.

This work defines an extension to standard grammars, incorporating information about user-events directly into the grammar, and defines `sequence grammars` as the quintuple $GS = (V, SEQ, T, P, S)$, where

- V is a finite set of non terminal symbols,
- $SEQ \subseteq V$ is a finite set of sequence symbols,
- T is a finite set of terminal symbols with $V \cap T = \emptyset$,
- $P \subseteq \Gamma^+ \times \Gamma^*$ is a finite set of productions, where $\Gamma = V \cup T$ and $l \to r$ describes the derivation from l to r,
- and $S \in V$ is the start symbol.

The sequence symbols are special non-terminal symbols that define individual elements of the input sequence, i.e. each sequence symbol in the derivation tree of a word of the language described by GS, corresponds to one user-event in the input sequence.

Illustrating the viability of the proposed changes to the grammar, the following discussion exemplary highlights the approach taken as a powerful way of addressing the need of understanding sequences as a component of input generation. A calculator program should allow a user to calculate multiple functions one after another providing an iteractive calculator program. A standard grammar could try to implement it like:

```
Grammar(
    'start := 'requests,
    'requests := 'requests ~ 'request | 'request,
    'request := 'function ~ "(" ~ 'numbers ~ ")",
    'function := "sqrt" | "tan" | "sin" | "cos",
    'numbers := 'number | 'numbers ~ 'number,
    'number := "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9",
)
```

The `'requests` symbol allows the user to chain multiple function calculations together, while the `'request` symbol represents a single request. It's intuitive that `'request` is the symbol that produces a single user-event, but the computer cannot determine which non-terminal symbols produce user-events. An equivalent sequence grammar would be:

```
Grammar(
    'start := 'requests,
    'requests := 'requests ~ 'SEQ_request | 'SEQ_request,
    'SEQ_request := 'function ~ "(" ~ 'numbers ~ ")",
    'function := "sqrt" | "tan" | "sin" | "cos",
    'numbers := 'number | 'numbers ~ 'number,
    'number := "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9",
)
```

In this grammar, the former `'request` symbol has been replaced by the symbol `'SEQ_request`. The equivalency of these two grammars is trivial, as only the name of one non-terminal symbol is changed. Even though the difference is simple in terms of the simplified grammar form, in terms of formal definition this would mean that `'SEQ_request` is an element of `SEQ`. This change to the name then allows a parser following the suggested new approach to identify non-terminal symbols that generate individual user-events.

6.1.2. Derivation, Expansion, and Extraction

The formal definition of a sequence grammar does not impose any restrictions on what other symbols may be derived from a sequence symbol. For implementing simple and straightforward algorithms for deriving words from the grammar and for transforming a derivation tree into an event sequence, the following restrictions are imposed: (1) A sequence symbol may not produce any other sequence symbols, (2) any terminal symbols must be derived from a sequence symbol, and (3) any non-terminal symbol that can be derived from a sequence symbol must not produce any sequence symbols. This implies that no sequence symbol can be derived from any other sequence symbol in any number

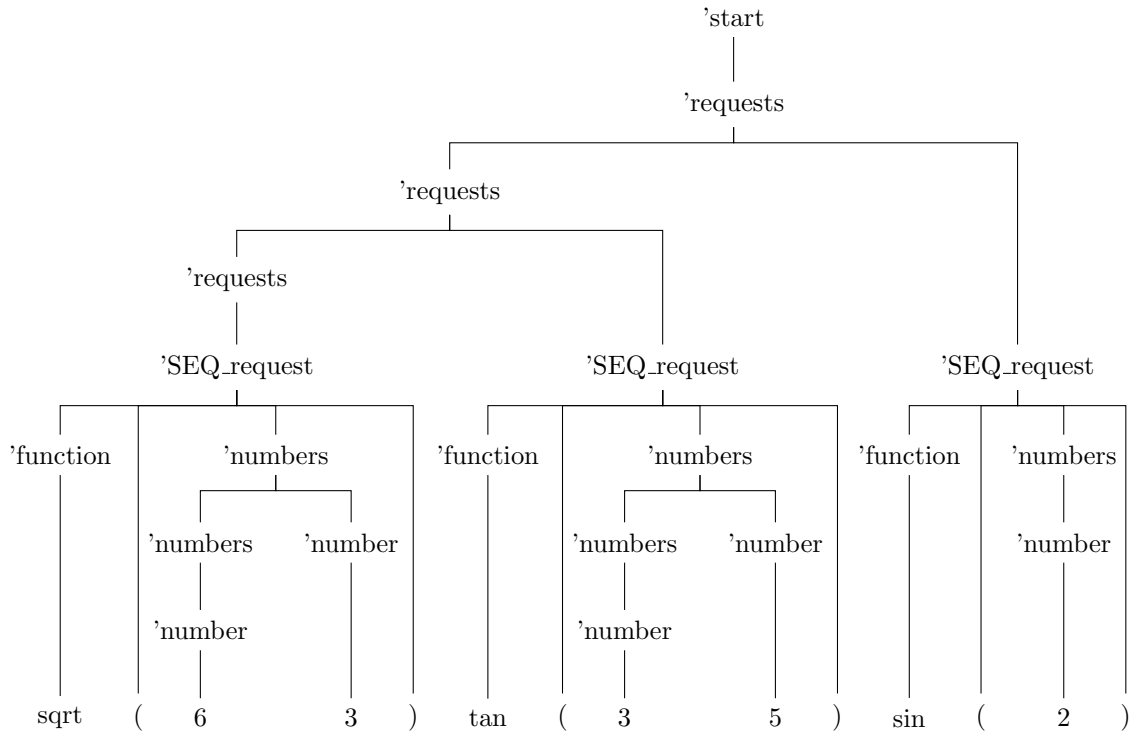


Figure 8: Derivation tree for the word "sqrt(63)tan(35)sin(2)"

of steps and that every terminal symbol has at least one sequence symbol ancestor. These restrictions result in derivation trees that can be separated into two sections. The upper section that only contains the start symbol and non-terminal symbols and the lower section that contains sub-trees that have a sequence symbol as root and contain non-terminal and terminal symbols. This "split" is shown in Figure 8. In this example, the sequence symbols (in this case, only `'SEQ_request`) separate the root and the non-terminals that generate sequence symbols, from the terminal-producing non-terminal symbols, which can be sequence symbols itself.

The following algorithms have been implemented to derive a word from a given sequence grammar respecting the imposed restrictions:

GetEventSequence

Consider the word `"sqrt(63)tan(35)sin(2)"` that can be derived from the above grammar. The derivation tree for this word is available in Figure 8.

Using the algorithm shown in Algorithm 3, the event sequence from this tree is collected by traversing the tree from left to right in a depth-first manner. If the root of the derivation tree is a terminal symbol, it's assumed that the event sequence has only a single element and it's returned. If the root is not a terminal symbol, traversal of the tree is started and all the sequence nodes are collected. Afterwards, the subtrees of all sequence nodes are traversed. For each subtree, a new event string is created, and whenever the algorithm traverses a terminal node, its content is appended to the

Algorithm 3: Get Event Sequence from Derivation Tree

```
   Input: Root r
   Output: array of string events
1  Function GetEventSequence(r):
2      if Type(r) is TerminalNode then
3          return [string(r)];
4      else
5          events ← [];
6          sequence_nodes ← [];
7          stack ← [r];
8          while stack is not empty do
9              node ← pop(stack);
10             if Type(node) is SequenceNode then
11                 sequence_nodes.append(node);
12             else
13                 for i ← 1 to size(node→children) do
14                     if type(node→children[i]) is not Terminal then
15                         push(stack, node→children[i]);
16         for i ← 1 to size(sequence_nodes) do
17             event ←″;
18             stack ← [sequence_nodes[i]];
19             while stackisnotempty do
20                 node ← pop(stack);
21                 if Type(node) is TerminalNode then
22                     event ← event + string(node);
23                 else
24                     for i ← 1 to size(node→children) do
25                         push(stack, node→children[i]);
26             events.append(event);
27         return events;
```

corresponding event string. Applying the algorithm to the above example returns the event sequence `["sqrt(63)","tan(35)","sin(2)"]`.

Derive

To use Algorithm 3, first a derivation tree is needed. For getting such a derivation tree, a word from the grammar is derived and the corresponding derivation tree is constructed on-the-fly by recording the productions applied. A simplified version of the algorithm used in this work is shown in Function `Derive` in Algorithm 4. The algorithm begins by initializing the tree's root, followed by handing over the derivation process to the function `DeriveFromNode` which will be reused later. Once the function `DeriveFromNode` has been called, the derivation process begins by expanding the given root node until at least `targetlength` sequence symbols have been produced. To get the maximal number of sequence nodes in minimal time, the algorithm chooses production rules that produce

Algorithm 4: Derive Word from Tequence Grammar

Input: Derivation tree *tree*, number *targetlength*, grammar G
Output: Derivation tree *tree*

Function `Derive`(*tree, targetlength, G*)**:**
    $dtree \rightarrow root \leftarrow new(Node)$;
    $dtree \rightarrow targetlength \leftarrow targetlength$;
    $dtree \rightarrow seed \leftarrow randomnumber$;
    `DeriveFromNode`(*tree, dtree→root, G*);

Input: Derivation tree *tree*, non-terminal node *rnode*, Grammar G
Output: Derivation tree *tree*

Function `DeriveFromNode`(*tree, rnode, G*)**:**
    $count \leftarrow 0$;
    $sequence_stack \leftarrow [rnode]$;
    $nonterm_stack \leftarrow []$;
    while $count < tree \rightarrow targetlength$ *and sequence_stack is not empty* **do**
        $node \leftarrow pop(sequence_stack)$;
        if *exists production* $p \in G$ *with* $p(node) \neq \emptyset$ *and* p *produces SequenceNode* **then**
            $node \rightarrow children \leftarrow p(node)$;
        else
            $p \leftarrow$ *production* $p \in G$ *with* $p(node) \neq \emptyset . \forall q \in G . q(node) \neq \emptyset . size(p(node)) \geq size(q(node))$;
            $node \rightarrow children \leftarrow p(node)$;
        for $i \leftarrow 1$ **to** $size(node \rightarrow children)$ **do**
            if $node \rightarrow children[i]$ *is not* $TerminalNode$ **then**
                if $node \rightarrow children[i]$ *may be derived into* $SequenceNode$ **then**
                    $push(sequence_stack, node \rightarrow children[i])$;
                else
                    $push(nonterm_stack, node \rightarrow children[i])$;
    while *sequence_stack is not empty* **do**
        $node \leftarrow pop(sequence_stack)$;
        if *exists production* $p \in G$ *with* p *does not derive into* $SequenceNode$ **then**
            $node \rightarrow children \leftarrow p(node)$;
        else
            $p \leftarrow$ *production* $p \in G$ *with* $p(node) \neq \emptyset . \forall q \in G . q(node) \neq \emptyset . size(p(node)) \leq size(q(node))$;
            $node \rightarrow children \leftarrow p(node)$;
        for $i \leftarrow 1$ **to** $size(node \rightarrow children)$ **do**
            if $node \rightarrow children[i]$ *is not* $TerminalNode$ **then**
                $push(nonterm_stack, node \rightarrow children[i])$;
    while *nonterm_stack is not empty* **do**
        $p \leftarrow$ *random production* $p \in G$ *with* $p(node) \neq \emptyset$;
        $node \rightarrow children \leftarrow p(node)$;
        for $i \leftarrow 1$ **to** $size(node \rightarrow children)$ **do**
            if $node \rightarrow children[i]$ *is not* $TerminalNode$ **then**
                $push(nonterm_stack, node \rightarrow children[i])$;
    return *tree*;

the most number of children. During this process, any non-terminal node that cannot be derived into a sequence node is not further expanded for now.

After the targeted number of sequence nodes has been produced, the algorithm shifts from a maximal expansion policy to a minimal expansion policy. It then applies production rules that produce the least amount of new nodes. Once there are no more nodes that can be derived into sequence nodes, the algorithm enters stage 2.

Due to the constraints imposed on sequence grammars, any non-terminal symbol that can be derived from the start symbol of the grammar but cannot be derived from any sequence symbol must itself derive sequence symbols. In other words, as there are no more nodes left that can be derived into sequence nodes and as sequence nodes cannot be derived into other sequence nodes, the remaining nodes must be sequence nodes. In this second stage, all subtrees for all sequence nodes are derived by randomly choosing production rules and applying them until there are no more non-terminal nodes to be expanded. Once this stage is finished, the derivation tree is complete and can be used to generate event sequences.

Extraction

Deriving a word from a given grammar and constructing the corresponding derivation tree is straightforward. Extracting a subword from a given word of a grammar and constructing the corresponding derivation tree is more complex. In this work, extraction is only applied at the sequence level, i.e. when extracting, a subsequence of the sequence of user-events represented by the original word is created.

Extracting a subword from a given word and constructing the corresponding derivation tree is important for two reasons: (1) When applying input reduction to a given word, it must be possible to remove parts of that word and use the produced subword as input for the given PUT, and (2) when trying to extend an already existing word it's desired to remove some parts at the end of the word to be able to change the starting point for expansion, i.e to perform backtracking.

For general context-free grammars, extracting subwords and parsing derivation trees is time consuming. When extracting, all sequence nodes in the derivation tree of our original word are collected. Afterwards, the sequence nodes are treated like leaves and a parsing algorithm to reversely construct the upper half of the subwords derivation tree is applied. The Earley Parser [23] gets implemented for this purpose. The most important problem with using context-free grammars is the complexity of parsing. The Earley Parser has a cubic worst-case complexity in the number of leaf nodes. As all PUTs used in this work's evaluation have `simple grammars`, a simple extraction algorithm has been implemented that does not need a parser.

Algorithm 5: Extract Subsequence from Derivation Tree

Input: Derivation tree *sourcetree*, array of pairs of begin—length *segments*, number *maxsrclength*, boolean *complement*, grammar G
Output: Derivation tree *desttree*

```
1  Function Extract(sourcetree, segments, maxsrclength, complement):
2      seqnodes ← array of all SequenceNode in sourcetree;
3      newseqnodes ← [];
4      if complement is true then
5          index ← 0;
6          segmentindex ← 0;
7          while index < maxsrclength and sequenceindex < size(segments) do
8              while index < maxsrclength and index < segments[segmentindex].begin do
9                  newseqnodes.append(copy(seqnodes[index]));
10                 index = index + 1;
11             index = segments[segmentindex].begin + segments[segmentindex].length;
12             segmentindex = segmentindex + 1;
13         while index < maxsrclength do
14             newseqnodes.append(copy(seqnodes[index]));
15             index = index + 1;
16     else
17         for i ← 1 to size(seqnodes) do
18             for c ← segments[i].begin to segments[i].begin + segments[i].length do
19                 newseqnodes.append(copy(seqnodes[c]));
20     if G is not a simple grammar then
           Use Earley Parser to construct the derivation tree. If a derivation
           tree cannot be constructed, the extracted word does not belong to the
           PUT's input language
21         return emptyderivationtree;
22     else
23         desttree ← empty derivation tree;
24         desttree→root ← new(Node);
25         desttree→root→children ← newseqnodes;
26         return desttree;
```

Definition 6.1 (Simple Grammar)**.** A simple grammar is a grammar G that can be transformed into a grammar G* that produces all sequence nodes from the start symbol S with a regular expression S+ or S*.

The implemented algorithm is shown in Algorithm 5. The algorithm takes the derivation tree of the original word, the `sourcetree`, a number of `segments` that specify sequence elements in the original word, and whether the new subword should consist of the sequence elements referred to in `segments` or of all sequence elements not specified in `segments`. If the algorithm can construct a derivation tree for the new subword, it returns that derivation tree; otherwise it returns an empty derivation tree which indicates that the subword is not part of the given input language.

Algorithm 6: Extend Input Sequence

Input: Derivation tree *sourcetree*, number *targetlength*, grammar G, number *backtrack*
Output: Derivation tree *tree*

```
1  Function Extend(sourcetree, targetlength, G, backtrack):
2      if backtrack > 0 then
3          desttree ←Extract(stree, [pair(0, size(stree) − backtrack)], false);
4      else
5          desttree ← empty derivation tree;
6          desttree→root ← copy(sourcetree→root);
7      if G is not a simple grammar then
           Traverse the derivation tree from right to left and find a node that
           can be expanded to new sequence nodes. Remove the subtree anchored at
           this node and set the node as root
8      else
9          root ← desttree→root;
10     DeriveFromNode(desttree, root, G);
11     return desttree;
```

Extension Similarly to extraction, extending an already existing word and corresponding derivation tree into a longer word that is still part of the language of the given grammar is not simple.

To expand an already existing derivation tree, first a node is to be found that can be expanded. In this work, two approaches to find such a node have been implemented. The first approach can be applied to context-free grammars in general. It traverses the derivation tree from right to left and looks for a non-terminal node that can be expanded into more sequence nodes. The second approach solely works on `simple grammars`. As all PUT's used in this works evaluation have `simple grammars` that describe their input language, the characteristic of a simple grammar can be applied for expanding a given derivation tree by generating new sequence nodes directly from the root of the grammar.

Extending an input needs to observe two cases: (1) The input to be extended is an unfinished input and can be extended directly, and (2) the input to be extended is a failing input and backtracking must be applied before it can be extended. The algorithm is shown in Algorithm 6. The algorithm takes the derivation tree of the input to be expanded, the number of new sequence nodes to generate, the grammar, and the number of sequence nodes to remove from the right-most end of the tree (i.e. the amount of user-events to backtrack). If backtracking is set to greater than 0, the extraction algorithm is applied to remove sequence nodes from the source derivation tree and the result is used as the starting point for expansion. Alternatively, the source derivation tree is copied and used as the starting point. If the grammar is simple, the root of the derivation trees copy is chosen as node for expansion. Alternatively, the right-most non-terminal node that can be expanded into new sequence nodes gets selected as node for expansion. Once the node for expansion has been decided, `DeriveFromNode` can be used, which has been introduced for general derivation, to expand the derivation tree.

6.1.3. Generating a Program Input

With the derivation, extraction, and expansion algorithms defined, the complete input generation algorithm can be put together. Algorithm 7 shows an abstracted version of the input generation flow implemented in TimeFuzz. TimeFuzz does not generate all inputs, run in a generation, at once. Instead, a definable amount of inputs are generated at the beginning of a generation. Whenever a test has been executed, a new input is generated, until the maximum number of inputs for the generation has been reached.

The function `GenerateInput` is called in two situations, either when starting a new generation to generate an initial set of tests or when a test case has finished. It begins by checking whether the current generation has source inputs. If there are source inputs, it is calculated whether one of the sources should be extended or if a new input should be generated. If a source input should be extended, the algorithm chooses a source input to extend. A source inputs is chosen in a way that at the end of each generation each source input has been extended for an equal amount of times. If the source input's test result was failing, the new inputs derivation tree is generated by calling `Extend` with a user defined number of events to backtrack. If the test result was unfinished, `Extend` is called without backtracking. If it is the first generation and there are no source inputs, or the probability check fails, a new derivation tree is generated from the PUT's grammar using `Derive`.

Once a derivation tree has been generated, the function `GetEventSequence` is used to extract the sequence of user-events. Afterwards, the input is checked against the exclusion tree and added as a test case to the execution handler's waiting queue. If the input has been excluded by the exclusion tree, input generation is simply repeated until an input is generated that is not excluded.

While the abstract version of the top-level input generation algorithm is straightforward, the implementation accounts for niche scenarios, such as backtracking on unfinished inputs and others. Furthermore, TimeFuzz is able to delete any generated derivation trees and event sequences at runtime, and regenerate them on-demand, for memory management purposes. The regenerability of derivation trees is being accomplished by using random seeds that are stored instead of the derivation trees and event sequences. To account for the case that a source input is currently not generated, the algorithm performs an additional check for this case and regenerates the source input before extending it.

6.2. Delta Debugging

TimeFuzz does not only generate new inputs by derivation from the grammar or by extending existing inputs. Inputs are also generated via DD. DD is not considered a separate unit as with many other fuzzing tools such as AFL. Instead, it is integrated into the fuzzing process itself. TimeFuzz applies DD in two situations: at the end of a generation, with the goal of minimizing the best inputs generated in the generation, and whenever an input's test result is passing. In both cases, TimeFuzz provides two algorithms that can be used for minimization. These are the standard DD algorithm developed by Zeller et al. and a new algorithm based on DD, `Score Progress`. The

Algorithm 7: Generate Input

```
Input: currently active generation generation, grammar G, user-settings settings, execution
       handler exechandler
Output: Derivation tree tree
1  Function GenerateInput(generation, G, settings, exechandler):
2      input ← [];
3      if generation→HasSources() is true and
         RandomChance() > settings→chanceToGenerateNewInput then
4          source ← generation→GetRandomSourceInput();
5          if source→result is failing then
6              input→derivationtree ←Extend(source→derivationtree,
                 settings→generationlength,G,settings→backtracking);
7          else
8              input→derivationtree ←Extend(source→derivationtree,
                 settings→generationlength,G,0);
9      else
10         input→derivationtree ← [];
11         input→derivationtree ←Derive(input→derivationtree,
             settings→generationlength,G);
12     input→events ←GetEventSequence(input→derivationtree→root);
13     if input is false then
14         exechandler→AddTest(input);
```

algorithms differ in the events that DD is applied to. While the standard DD algorithm is applied to the entire input up to the last executed event, the `Score Progress` algorithm applies DD only to subsequences of events that do not result in a score increase. This also affects the algorithms availability. The standard DD algorithm is available for any passing inputs and for minimization at the end of a generation as long as there is a score value available. The `Score Progress` algorithm is only available if and only if the oracle provides an individual score value for each user-event in a generated input.

Both the standard algorithm and `Score Progress` are available for two minimization strategies. They can be used to either minimize an input with a passing test result or to minimize an input while preserving either the primary score or the secondary score of the input. When minimizing to reproduce the test result, only inputs with a passing test result are considered valid results of minimization, conforming with the typical usage of DD. When minimizing while preserving a score value, inputs that reproduce the score are considered valid results. The score value does not have to be reproduced entirely. The user may specify a relative and an absolute loss in score value that generated inputs may show in order for them to be considered preserving the property. This is based on the understanding that in some situations a loss in score value may be acceptable if the input's length is drastically reduced in return. The choice whether to use this feature or to reproduce the score value completely is left to the user.

The first section of this chapter will present the implementation of the standard DD algorithm and the general workflow of input minimization used in TimeFuzz. As the implementation implements the well-known base algorithm, the section will focus

Algorithm 8: Decide Subsets and Complements

```
Input: original input input, grammar G, level level, task controller controller
Output: Derivation tree tree
Function DecideInputs(input, G, level, controller):
    splits ← [];
    position ← 0;
    length ← size(input)/level;
    while count < size(input) do
        splits.append([position, length]);
        position ← position + length;
    complements ← [];
    for i ← 1 to size(splits) do
        position, length ← splits[i];
        complements.append([[0, position], [position + length, size(input)]]);
    for i ← 1 to size(splits) do
        task ← DDGenerateSubsetCallback();
        task→segments ← splits[i];
        controller→AddTask(task);
        task ← DDGenerateComplementCallback();
        task→segments ← complements[i];
        controller→AddTask(task);
```

primarily on the workflow of DD in general. The second section will present the custom `Score Progress` algorithm, which follows the general input minimization workflow and implements improvements over the standard DD algorithm.

6.2.1. Standard Delta Debugging

DD is the base of many input reduction algorithms. It splits an input to minimize (also referred to as original input) into subsets and tests whether the subsets still exhibit some property. This property can be virtually anything that an oracle can decide. In this work, delta debugging (DD) is used to minimize inputs according to two properties: (1) to minimize passing inputs and (2) to preserve a score value reported by the test oracle. The implementation of these two strategies is virtually identical, except for the evaluation of test results.

As already mentioned in Chapter 5.3, the implementation of DD is split into four parts. Each of those parts represents a major step in the DD algorithm, which can be seen in Algorithm 1. DD begins by generating a number of subsets of the original input that are then executed as tests. If any of these subsets preserve the property, the subset is used as the new original input and divided into smaller subsets in the next iteration of dd. If none of the subsets preserve the property, DD generates the complements to the subsets and evaluates them the same way.

The first of the four parts of DD in TimeFuzz takes care of the decision how to split the original input into subsets and which complements are to be generated. The number of subsets generated depends on the `level` of the current iteration. The level begins in the

Algorithm 9: Generate Subsets and Complements

```
Input: original input orig_input, grammar G, segments segments, execution handler
       exechandler
1  Function GenerateSubset(orig_input, G, segments, exechandler):
2      newinput ← [];
3      newinput→derivationtree ← Extract(orig_input→derivationtree, segments,
         size(orig_input), false);
4      if CheckAgainstExclusionTree(newinput) is false then
5          exechandler→AddTest(newinput);
6  Function GenerateComplement(orig_input, G, segments, exechandler):
7      newinput ← [];
8      newinput→derivationtree ← Extract(orig_input→derivationtree, segments,
         size(orig_input), true);
9      if CheckAgainstExclusionTree(newinput) is false then
10         exechandler→AddTest(newinput);
```

first iteration as 2 but increases or decreases based on whether a subset or complement preserves the property. The algorithm used to decide the splits is straightforward and is shown in Algorithm 9. It decides the subsets based on the length of the input divided by the level and the complements based on the subsets. For each subset and complement, a new task is created and added to the task queue. Those tasks make up part two of the dd implementation and are illustrated in Algorithm **??**. They generate either subsets or complements and compare them against the exclusion tree as described in Chapter 5.5.

If the user defines that all tests should be executed before an evaluation is made, all created tasks are performed and the generated inputs are tested. Alternatively, the user can enable a batch mode. The batch mode allows the user to set a maximum amount of tests that may be generated and executed by a single instance of DD at any time. In batch mode, TimeFuzz generates a set amount of inputs and tests them. Whenever a test has been finished, it is evaluated. If it is found to preserve the property, all remaining ungenerated tests are discarded and the iteration is ended. If the test does not preserve the property, a waiting test is generated and executed. The non-batch mode of TimeFuzz generates all tests and executes them regardless of result, while the batch mode is a multi-threaded version of the algorithm shown in Algorithm 1.

The evaluation of an individual test after it has been executed is part three of the DD implementation. The oracle is used to obtain the test results and evaluate whether the property is preserved. This is performed every time a test has been completed and replaces the test callback used by inputs generated from the grammar.

If batch mode is not enabled, all tests are generated and executed. This also happens when batch mode is enabled but no subset or complement preserves the property. Once all tests have been executed or execution has stopped after an input has been found to preserve the property in batch mode, the iteration is evaluated in the fourth part of dd. All tested inputs are once again checked for their results and compared with one another. If DD is used to reproduce a passing test result, the test with the shortest executed length is chosen among those that have a passing result. The input is then set as the

new original input. If the chosen input is a subset, the level is reset to 2 for the next iteration. If it is a complement, the level is adjusted to $max(level - 1, 2)$. If DD is used to reproduce a score value, the test with the score of at least the target score is chosen. If there are multiple tests that share the same score, the one with the shortest executed length is chosen as new original input and the level is adjusted to $max(level - 1, 2)$.

If there is no input that can be chosen as new original input, the level is either adjusted to $min(level * 2, size(original\ input))$, or if the current level is already identical to the original inputs length, DD ends.

The batch mode implementation of DD is roughly consistent with the improvements to the original algorithm proposed by Hodován et al. in [39]. They were the first to propose parallelizing DD and show that the parallelized implementation also produces 1-minimal results.

The choice between batch mode and executing all tests in each iteration is not trivial. As already discussed in Chapter 2, a few changes to the order and timing of events in an event-sequence may completely alter the results of a test run. When reproducing the test result with DD, this might not have substantial consequences, but when reproducing a score value, this may result in some of the tests reproducing the score while others may even improve the score significantly. This phenomena also occurs during some test cases in the evaluation of this work (see Chapter 7.4).

6.2.2. Score Progress

One of the greatest problems of DD is the resource usage of the algorithm [97] in relation to the significant number of tests that need to be performed. While the implementation of the standard DD algorithm introduced above speeds up DD by leveraging parallel test generation and test execution, there is still room for improvement.

In addition to simple parallelization, Hodován et al. [39] describe other improvements in their paper. One of the improvements mentioned is to only execute the complements and not the subsets themselves. This is based on the idea that a significant reduction to a subset may not be achievable considering the format of a program input. The paper refers to input formats like C programs, for which it is unlikely to find a subset that reproduces a fault, as large necessary parts of the program might be removed in the process. This also applies to the input of interactive programs, as the user-events in the input are largely interdependent. Testing a subset from the middle of the original sequence may not be able to pass necessary hurdles and may be superfluous.

The approach taken in this work uses the idea of not testing subsets at all and takes it a step further. Instead of computing all subsets and then only testing their complements, this work implements `Score Progress`. Score Progress uses an oracle's score value to identify parts of a generated input that do not show an increase in score value. It then applies the DD algorithm to compute subsets that are limited to the identified sections of the generated input and executes their complements. The goal of this algorithm is to reduce the number of tests that have to be run significantly by keeping the parts of a generated input that actually do result in progress while trying to remove parts that do not contribute. The algorithm focuses mainly on the score value, but is available both for

Algorithm 10: Score Progress Helper Functions

```
   Input: original input input, grammar G, level level, execution handler exechandler
   Output: vector of pairs of begin and length ranges
 1 Function ExtractRanges(input):
 2     ranges ← [];
 3     lastvalue ← input→scores[1];
 4     begin ← 0;
 5     length ← 0;
 6     for current ← 2 to size(input→scores) do
 7         if input→scores[current] is lastvalue then
 8             length ← length + 1;
 9         else
10             if length > 0 then
11                 ranges.append([begin, length]);
12             length ← 0;
13             begin ← current;
14         lastvalue ← input→scores[current];
15     return ranges
16 Function GetSubsets(ranges, size):
17     subsets ← [];
18     for i ← 1 to size(ranges) do
19         if ranges[i].length > size then
20             position ← 0;
21             while position < ranges[i].length do
22                 if ranges[i].length − position ≥ size then
23                     subsets.append([ranges[i].begin + position, size]);
24                 position ← position + size;
25     return subsets
```

test result reproduction and score reproduction as both goals do not affect the algorithm itself but simply the reproduced property.

The implementation of the algorithm has a strong dependency on the functionality of the oracle that is must report a score value for each individual user-event in the generated input. The algorithm identifies the subsets of the input that do not show progress and proceeds to minimize them. An abstracted version of the implemented algorithm is shown in Algorithm 11. The implementation follows the structure outlined in Chapter 6.2.1 and the separation into deciding the subsets and complements, generating them, evaluating them, and evaluating the iteration.

The algorithm begins by extracting all ranges (subsets) that do not show an increase in score value, with the function `ExtractRanges` shown in Algorithm 10. This is done straightforward by iterating over the list of score values. Any range without score increase of at least length 1 is returned.

The algorithm proceeds by calculating the subsets for which complements are generated. They are chosen by the function `GetSubsets` (also shown in Algorithm 10). The function

Algorithm 11: Score Progress

```
   Input: original input input, grammar G, level level, execution handler exechandler
   Output: vector of pairs of begin and length ranges
1  Function ScoreProgress(input, G, exechandler):
2      while true do
3          ranges ← ExtractRanges(input);
           Step 1: Calculate subsets
4          size ← GetTotalLength(ranges)/level;
5          subsets ←GetSubsets(ranges, size);
           Step 2: Generate complements and run them
6          inputs ← [];
7          for i ← 1 to size(ranges) do
8              segment ← ranges[i] newinput ← [];
9              newinput→derivationtree ← Extract(input→derivationtree, [segment],
                 size(input), true);
10             if CheckAgainstExclusionTree(newinput) is false then
11                 exechandler→AddTest(newinput);
12                 inputs.append(newinput);
           Step 3: Evaluate finished tests. Either stop the above iteration on the
           first satisfying input found in batch mode, or run all tests.
           Step 4: Choose new input and level.
13         if there is a satisfying input then
14             input ← choose input with the highest score from inputs;
15             level ← 2;
16         else
17             if level = GetTotalLength(ranges) then
18                 break;
19             level ← min(level ∗ 2, GetTotalLength(ranges));
```

takes the extracted ranges and a maximum size for the subsets. The size is calculated as the quotient of the combined length of all ranges and the current level. `GetSubsets` iterates over every range. If the range is longer than `size`, the first subset starting at the beginning of the range with length of `size` is added to the list of subsets. If the range is a multiple of `size`, the range is split into `n` subsets of length `size`, with `n` being the truncated quotient of the length of the range and `size`. This approach guarantees that we only attempt to remove the longest allowed subsets in each iteration of `Score Progress`, ensuring that the longest stagnant subsets are removed first. Even though some ranges and potentially parts of ranges are ignored in earlier iterations, they are tested in later iterations of the algorithm.

Once the subsets have been identified, the complements are generated identically to the standard dd algorithm and tested. If batch mode is enabled, tests are only executed until a satisfying test is found, otherwise all tests are run and evaluated. After test execution has finished, all tested inputs are once again checked for their results and compared with one another. If the algorithm was set to reproduce the original input's test result, the new input with the shortest executed length, among those that reproduce the result, is

chosen. Otherwise, the test with the highest score value is selected. If there are multiple tests with the same score value, the one with the shortest executed length is chosen. If there are tests that are identical in length and score, the first one found is used as the new input to minimize.

If no satisfying input has been found, the level is increased to the lower value between `level*2` or the combined length of all identified ranges. In case a new input to minimize has been chosen, the level is reset to 2. This is not in line with the standard DD algorithm. It was chosen to ensure after any subset had been removed from the input, the algorithm tries to remove the largest ranges first, instead of attempting to remove subsets similarly in size to the removed one. The reasoning for this is: Even removing a subset of length 1 may result in significantly different behavior by the PUT (see Chapter 2), which might allow for the removal of one of the larger sections, resulting in fewer tests having to be executed. Similarly, as the behavior of the PUT on the new input might have been vastly different from the behavior on the original input, the algorithm proceeds by recalculating the ranges without score progress. The algorithm ends if the level is equal to the combined length of all identified ranges and no satisfying input has been found in the iteration.

Additionally to focusing on ranges in the input to minimize, the algorithm's implementation contains another optimization. TimeFuzz is a generational framework that uses prior generated inputs to generate new longer inputs. Any input that has been extended from an input generated as the result of DD with `Score Progress`, contains a number of subsets that do not result in progress but have already been minimized by DD. The Score Progress algorithm may use this information and does not apply DD to any sections that have already been minimized, if defined by the user.

While this might lead to some tests that could have increased the score value not being found, the reduction in the number of tests that need to be run might be significant enough to justify this trade-off. The decision is left to the user.

6.3. Selecting Inputs to minimize and Sources for input Generation

Input generation and DD are central topics of this work. However, even more important is the selection of inputs to minimize and of sources for a new generation. The right selection will allow for the generation of better extended inputs, while the wrong selection might result in hitting a dead end.

The decision which inputs are considered good and which bad cannot be made in a universal way for all PUTs. PUTs are diverse and some oracle may provide score values while other oracles might not. Both the algorithm selecting inputs to minimize as well as the algorithm for source selection support different customizability selection strategies. TimeFuzz provides the user with three different choices regarding the optimization strategy used in a fuzzing session.

First, the length of generated inputs can be maximized. In this mode, any score value provided by the oracle is ignored and only the length of an input is considered in any decisions made by the frameworks algorithms. As the goal is to generate the longest possible inputs, DD is completely disabled, except for minimizing passing inputs. This also affects source selection, as the algorithm simply selects the longest inputs among

those generated from grammar in the last generation and uses them as sources for the new generation.

Second, either the primary score or the secondary score can be maximized. When maximizing one of the score values, DD at the end of a generation is available. The selection algorithm will choose the inputs with the respectively highest score and minimize them using DD. Once DD has been finished, the source selection algorithm will once again choose the inputs with the highest score. If DD has successfully generated inputs that preserve the score value of the original inputs to minimize while reducing the length of the inputs, the source selection algorithm will prefer those inputs over the longer original ones. As already discussed in Chapter 5.1 if inputs generated by DD are selected, they are selected evenly among the instances of DD to avoid path constriction as much as possible. If DD is used to reproduce the respective score value completely, this will result in the source selection algorithm only choosing the shortest inputs generated by each instance of DD. However if the user has specified an acceptable loss greater than zero, the inputs generated by DD might not be chosen at all.

To address this issue, TimeFuzz implements the third option available. While the inputs to minimize are chosen according to their highest primary (or secondary) score identical to option two, the source selection algorithm will choose the inputs with the highest ratio of length per primary (or secondary) score. In turn, the source inputs for each new generation will only be comprised of the shortest inputs with the highest score, guaranteeing even with steady progress that the generated inputs are minimal in regards to their length. The primary purpose of this option is to allow DD to trade score value for the length of minimized inputs. As the number of new user-events extended in each generation is set by a user settings, the inputs generated in each generation will be as small as possible, which in turn will reduce the size of inputs that will need to be minimized both for end-of-generation minimization as well as minimizing passing inputs.

While selection for the first two options can simply select the inputs with the respectively highest value, this might result in problems with the third option. The simple ratio of length per primary score does not take into account whether the score value is high to begin with. This might result in inputs being chosen that have a significantly lower score than other inputs generated in the generation because their length efficiency is marginally higher. To mitigate this potential behavior, the algorithm splits the generated inputs into buckets according to their primary (or secondary) score values, and then sorts them by the ratio of input length per score. The buckets are traversed until the set number of sources have been chosen. Even though this might result in inputs with a lower length per score ratio, all inputs are guaranteed to have high scores.

The descriptions provided above give an insight into the idea of source selection pursued in this work. The implementation is slightly different. If DD is used at the end of a generation, the source selection begins by selecting the results of the different DD instances first, only then it proceeds to select inputs that have been generated directly from the grammar, as it is unlikely for them to be better than the result of DD and simpler to implement.

In the evaluation of this work, option one will be used to demonstrate the effectiveness of TimeFuzz when no score values are available. All other experiments will use score

values to apply DD and source selection while allowing for a marginal loss during DD at the end of each iteration and will use the third option for source selection.

7. Evaluation

The evaluation of this work focuses on the following research questions as defined in the introduction:

- **RQ1** How does TimeFuzz compare to the popular grey-box fuzzing tool AFL and its extension IJon, focusing on deep state fuzzing? That is, can TimeFuzz solve similar problems in comparable time, despite being a black-box fuzzer without access to internal program data?
- **RQ2** To what extend are winning strategies generated by TimeFuzz minimal, and how do they compare to those generated by AFL and IJon?
- **RQ3** How effective and efficient is the `Score Progress` delta debugging algorithm and how does it compare to the standard delta debugging algorithm?
- **RQ4** How effective is the usage of an exclusion tree to reduce the number of executed tests and how much memory is needed to maintain it?
- **RQ5** To what extent does TimeFuzz adhere to the core principles outlined in the introduction?

To answer these questions, this work compares TimeFuzz to the popular fuzzing framework AFL and its extension for deep state fuzzing, IJon. The version of IJon used in all experiments is available on GitHub[5]. Unfortunately the version originally released by the developers on their GitHub page[6]isnot able to compile on any reasonably modern platform. The version used in this work is a GitHub fork that includes additional functionality and bug fixes. AFL was also used in version 2.51b[7], which IJon is based on.

To evaluate TimeFuzz, four different experiments were conducted. The experimental setups and results are publicly available on GitHub[8]. In the first experiment, TimeFuzz, AFL, and IJon are applied to the Maze, a common benchmark originally used to evaluate systematic execution approaches. The Maze was used in the original IJon paper by Aschermann et al. [9] to compare IJon to various fuzzing approaches, including AFL. This work uses the Maze to compare TimeFuzz to IJon and, representative of general grey-box fuzzing approaches, AFL.

The second and third experiments replicate another experiment originally conducted by Aschermann et al. in which IJon and AFL are tasked with solving the levels of a C based Super Mario Bros (SMB C) implementation. This work repeats this experiment to compare AFL, IJon, and TimeFuzz. The third experiment expnds the second by modifying the input accepted by Super Mario Bros, comparing the three fuzzers on a version that requires significantly longer input sequences.

[5]https://github.com/AlwaysEden/ijon-experimental
[6]https://github.com/RUB-SysSec/ijon
[7]https://lcamtuf.coredump.cx/afl/releases/?O=D
[8]https://github.com/muenchk/TimeFuzz_Evaluation

The final experiment compares TimeFuzz's fuzzing strategy with random input generation. It evaluates the generational approach described in this work against a version that generates only random inputs, highlighting the improvement achieved by the proposed method. TimeFuzz is applied to a Python based implementation of Super Mario Bros executed using the OpenAI Gym-Retro framework [73]. The primary difference between this experiment and experiments two and three lies in their test runtimes and their communication between TimeFuzz and the PUT. This experiment offers an insight into TimeFuzz's performance on slow PUTs, and in scenarios where no interaction with the PUT is possible.

After the 4 experiments have been introduced and evaluated, the chapter finishes with a comparison between the standard DD algorithm and the `Score Progress` algorithm, analyzing the number of tests generated under different configurations.

7.1. The Maze

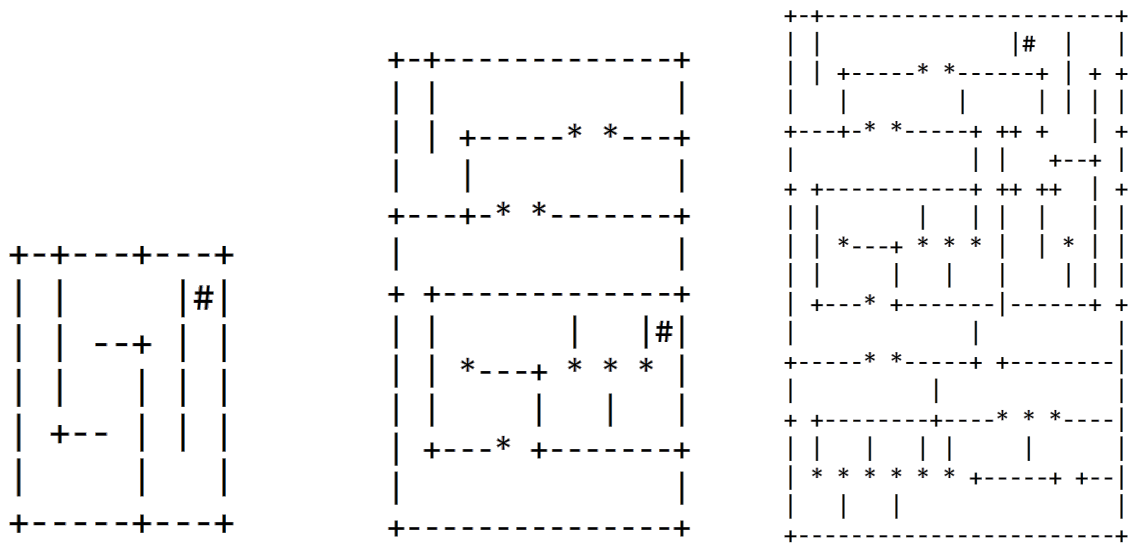
The Maze is a well-known benchmark often used to demonstrate the power of symbolic execution [9]. It has also been used to demonstrate the capabilities of directed fuzzing. Aschermann et al.'s original IJon paper uses it to compare IJon's directed fuzzing approach to other state-of-the-art fuzzers. While it compares IJon to both KLEE and AFL, this evaluation omits the white-box tool KLEE. It instead focuses on the comparison between the grey-box tools AFL, its extension IJon, and this works black-box tool TimeFuzz.

The base version of the Maze is a simple game where the player navigates from a fixed starting point to a goal using left, right, up, and down commands. In the original version, the player must take the correct path. Walking into a wall or backtracking results in failure. This work uses a modified version of the Maze. The player still cannot walk into walls but backtracking is now allowed[9]. Aschermann et al. also introduce a larger version of the Maze with dead-ends in their evaluation of IJon. Additionally, they introduce a hard mode that allows the player to walk into a wall without loosing the game. Figure9a shows the original Maze, Figure 9b shows the larger version. This work adds a new super large version of the Maze shown in Figure 9c. This version is even larger and introduces more and longer dead-ends. It also increases the number of paths to the goal to two.

The experiment compares AFL and IJon with TimeFuzz. TimeFuzz is run in two different configurations. Each Maze prints the full layout and the player's position to the standard output whenever a new user-event is processed. The first configuration of TimeFuzz (TimeFuzz Score) uses an oracle to analyze the output and count the number of unique positions found. This count serves as the primary score. The second configuration of TimeFuzz (TimeFuzz Length) does not use a score value, instead it only maximizes the length of the generated inputs. Both configurations use the standard DD algorithm for input minimization as tests are inexpensive.

All tools are run on a virtual machine with Ubuntu-24.04, kernel version 6.11.0-21-generic, and a 2 GHz CPU. Each experiment was run for a maximum of 1h. AFL and IJon

[9]This modification was not intended, but is the result of an implementation error, which was only found after the evaluation had concluded.



(a) The small Maze (b) The large Maze (c) The super large Maze

Figure 9: The three Maze versions AFL, IJon, and TimeFuzz are evaluated on. In all three mazes the player begins in the top left-hand corner and has to make their way to the treasure located at #.

were run for the full hour, TimeFuzz was allowed to exit early if it found and minimized a passing input. Each instance of AFL and IJon used a single CPU core. TimeFuzz was afforded a maximum of 3 cores for input generation and test evaluation, and a maximum of 5 concurrently executed tests.

In the remainder of this subchapter, the results of the experiment will be discussed, including runtime, the number of tests performed, and the length of generated inputs. Finally, the total number of tests executed, and the average tests per minute will be used to discuss the influence of the imbalance between the number of CPU cores available to the different tools.

The Results of the experiment are presented in Table 1. AFL was unable to solve any of the mazes. IJon successfully solved all small and large mazes. However, it solved the easy super large Maze only once within the time limit and failed to solve the hard super large Maze entirely. TimeFuzz Score solved all Maze configurations, with the exception of one failed run on the easy super large Maze. TimeFuzz Length was able to solve most of the mazes but struggled with both the easy large and easy super large variants.

The column `Time` in Table 1 shows the median runtime until either the best input was generated or the Maze was first solved by any input. For AFL, the median runtime was consistently below three minutes. This suggests that AFL not only failed to solve the mazes but also made no meaningful progress after the initial few minutes. In contrast, IJon did not exhibit this limitation. It either solved the Maze within the limit or continued making progress throughout the run. In the single run that did not make progress right until the end, IJon generated its best input at the 38 minute mark, a drastic improvement over AFL. TimeFuzz Score failed to solve only one Maze in a single run. Similar to IJon,

	AFL		IJon		TimeFuzz Score		TimeFuzz Length	
Maze	Solved	Time	Solved	Time	Solved	Time	Solved	Time
Small E	✗	00:08	✓	06:43	✓	00:02	✓	00:10
Small H	✗	01:29	✓	04:47	✓	00:01	✓	00:01
Large E	✗	02:01	✓	21:05	✓	00:07	1/3	59:52
Large H	✗	00:01	✓	18:17	✓	00:09	✓	04:43
Super E	✗	00:19	1/3	57:03	2/3	00:37	✗	59:43
Super H	✗	00:14	✗	57:58	✓	01:27	✓	17:12

Table 1: The Maze is being solved by AFL, IJon, TimeFuzz using score value, and TimeFuzz only using input length. Three runs were performed each, with the number of runs that were able to solve the Maze being shown in the columns "Solved". ✗ denotes that no run was able to find a solution while ✓ denotes that all three runs were successful. The column "Time" shows the median time in mm:ss until the best input was generated / until the Maze was solved first. The Maze being solve is shown with its size, and a letter marking difficulty of the Maze: H for the hard version, and E for the easy version where the player cannot walk into a wall.

	AFL	IJon	TimeFuzz Score	TimeFuzz Length
Maze	Tests	Tests	Tests	Tests
Small E	26896	900765	1506	12510
Small H	76740	426821	1207	871
Large E	298475	3097551	9794	137531
Large H	125256	1540261	24417	13701
Super E	264016	3947760	167861	262761
Super H	66745	942498	39458	28475

Table 2: The average number of tests performed until the best input was generated / until the Maze was first solved.

TimeFuzz Score run still showed progress until the 28-minute mark. TimeFuzz Length showed similar behavior as IJon. It either solved the Maze or continuous to make progress until the time limit. Overall, the results demonstrate that both IJon and TimeFuzz (in both configurations) significantly outperform AFL, both in terms of Maze completion and continuous progress over time.

While continuous progress and the ability to solve the mazes are important, the runtime of the tools and the number of executed tests is similarly insightful. When comparing IJon and TimeFuzz on mazes both tools successfully solved, the difference in runtime is clear. IJon typically requires several minutes whereas TimeFuzz Score succeeds within seconds. TimeFuzz Length is slower than TimeFuzz Score on nearly all cases. However, it still outperforms IJon on the mazes it successfully solved. For example, TimeFuzz Length solved the easy large Maze in 13 minutes and 48 seconds.

Table 2 shows the number of tests executed until the Maze was solved or the best input was generated. In both configurations, TimeFuzz requires significantly fewer tests to solve the mazes compared to IJon. This demonstrates that TimeFuzz's approach, extending good inputs rather than generating inputs from scratch, enabled it to outperform both IJon and AFL in terms of runtime and test efficiency.

TimeFuzz not only requires fewer tests to solve the Maze but also generates fewer tests overall. Table 3 shows the average total number of tests and the average number of

	AFL		IJon		TimeFuzz Score		TimeFuzz Length	
Maze	Total Tests	Execs per min	Total Tests	Execs per min	Total Tests	Execs per min	Total Tests	Execs per min
Small E	9716936	161948	6784806	113080	1705	40040	14329	46104
Small H	2529333	42155	5070345	84505	9504	57498	4438	53704
Large E	8185115	136418	7351904	122531	11533	60332	140312	3748
Large H	6881194	114686	4186955	69782	1818909	37048	23053	6412
Super E	2525477	42091	4592055	65796	224665	37174	262760	4377
Super H	4206880	70114	988731	16478	1341238	22354	94462	4296

Table 3: The average total amount of tests generated by the different fuzzers, and the average number of tests executed per minute.

tests executed per minute. The data shows that IJon executes fewer tests than AFL in the same time. While the total amount of tests executed by TimeFuzz is not directly comparable, since the tool exits after generating and minimizing at least one input, the levels for which TimeFuzz Score and TimeFuzz Length failed to solve the Maze still used significantly fewer tests than IJon. The only exception is the hard super large Maze, where the amount of tests generated by IJon is surprisingly low.

While the total number of tests run by TimeFuzz is not representative, the number of tests executed per minute is. This metric clearly shows that TimeFuzz executes significantly fewer tests than AFL and IJon across nearly all levels. This is despite the fact that AFL and IJon were restricted to a single CPU core for computation and test execution while TimeFuzz was permitted to use five cores solely for executing tests. TimeFuzz's comparatively low throughput in this regard can be attributed to two main factors. First, AFL and IJon dump all program input to the Maze's standard input. In contrast, TimeFuzz supplies user-events to the mazes one at a time. Since this requires TimeFuzz to perform additional computation, including the management of multiple tests simultaneously, a delay is introduced between the Maze reading the user-event and TimeFuzz providing the next user-event. For a small programs like the mazes used in this experiment, this introduces significant delay, scaling with input length. The second contributing factor is process creation. AFL (and by extension IJon) is heavily optimized for rapid test execution and process spawning. TimeFuzz does not implement similar optimizations, which results in overhead associated with spawning new processes. Although concurrent test execution introduces some delay, the more significant factor is the length of the test inputs, particularly for TimeFuzz Length on the harder mazes where inputs often exceed several thousand events.

Table 4 compares the lengths of generated passing inputs to the theoretical minimum required to solve the Maze. It reports both the shortest successful input generated and the average of these shortest inputs across the three runs per fuzzer. If a tool failed to solve the Maze, the entry is marked with ✗.

IJon achieves the minimal input length on the easy small and large mazes, but generates significantly longer inputs on the hard small and large mazes, as well as the easy super large Maze. While the shortest lengths are comparable, the average is much higher for all mazes (except for the super large Maze, where only one of three runs suceeded). In contrast, TimeFuzz Score generates minimal inputs for all 6 mazes. For the hard mazes,

		IJon		TimeFuzz Score		TimeFuzz Length	
Maze	Minimal	Shortest	Average	Shortest	Average	Shortest	Average
Small E	28	28	30.67	28	28.67	74	114
Small H	28	45	56	28	28	28	28.33
Large E	68	68	91	68	68.67	1526	1526
Large H	68	171	231.33	68	68	68	79.33
Super E	115	160	160	115	119	✗	✗
Super H	115	✗	✗	115	115	115	119

Table 4: The shortest respectively the average length of the inputs that solved the mazes. Only inputs from successful runs have been included. AFL is not shown as it did not solve any of the mazes. The column "Minimal" shows the minimal achievable input length.

the average minimal length matches the theoretical minimal length exactly. Only on the easy mazes does TimeFuzz Score occasionally fail to reach the minimal length. TimeFuzz Length produces minimal inputs for all hard mazes but fails to do so on the easy small and large mazes. It also struggles to consistently minimize inputs for the hard versions of the Maze.

This seemingly counterintuitive behavior of TimeFuzz on the easy mazes can be attributed to limitations of DD. DD works by recursively splitting inputs into equally sized subsets. In case of the easy mazes, any passing input longer than the minimal length must include pairs of steps that cancel each other out. For example, consider a case where the player is at position (1,3), with (1,1) as the starting point in the top-left corner and (1,5) the bottom-left corner. The only way to return to the same position is by taking an equal number of opposing steps, e.g. moving one step up followed by one step down, or two steps up and two down. If these steps are not perfectly aligned on even positions in the input sequence, DD may not be able to remove them, as removing a single event will result in the Maze terminating. Moreover, if a series of backtracking steps winds around the Maze's edges, all of them would need to be removed at once to preserve the validity of the input, something DD might not necessarily be able to do. This issue also exists in the hard Maze variants, but the ability to walk into walls without failing, significantly increases the number of removable events. This gives DD more flexibility in reducing input subsets. This demonstrates that, although DD is the most generally applicable input reduction algorithms, it struggles with highly interdependent program inputs.

7.2. Super Mario Bros C

The second and third experiments apply the fuzzers AFL, IJon and TimeFuzz to a reimplementation of the video game Super Mario Bros (SMB) written in C. This chapter will lay out the two experiments and their respective results, comparing the fuzzers in terms of runtime, number of executed tests, the distance Mario was able to traverse, and the length efficiency the generated inputs.

SMB contains 32 levels split into 8 worlds, each comprising 4 levels. The first three levels of each world feature various obstacles and enemy entities that Mario must overcome or avoid. The fourth level features similar challenges, but ends with a boss fight followed by a short non-interactive cutscene. An input is considered passing if Mario completes the level in normal levels, or passes the boss in boss levels and enters the cutscene, as it does not require any player interaction.

The original C implementation replicates the input scheme of the physical game console, including jump, backward, and forward movements. In contrast, the version used by Aschermann et al. to compare IJon and AFL replaces traditional button based input with a simplified scheme. The program is modified to simulate the right button being pressed, resulting in constant forward movement. Additionally, Mario is set to always sprint. The inputs generated by AFL or IJon only encode whether the jump button is pressed or not. Unlike physical controllers, the jump state is not specified per frame, but encoded as a sequence of duration based values. Each input is a series of bytes where each byte specifies the number of frames to maintain the current jump state. The bytes alternate between maintaining the jump button as pressed or released, until the input is exhausted.

In the version used by Aschermann et al., SMB terminates under a defined set of conditions. It terminates if Mario has died due to to an enemy or falls out of the world. It terminates if Mario has completed a level, when the input has been exhausted, and if Mario's position remains unchanged for more than four frames. The last condition was implemented to produce speedrun-like inputs [9], but is not necessary to generate winning strategies, as will be demonstrated. In this work, the only constraints imposed on any program shall be a restriction of the input given to said program. Therefore, in the following experiments, SMB does not terminate when Mario is idle.

The second experiment runs SMB in the configuration discussed above. Each byte in an input alternates Mario's jump state consistent with the scheme from the original experiment. The third experiment removes the input restrictions introduced in the original experiment. Instead, SMB processes one user-event per rendered frame. The input consists of a stream of bytes where bits 2 through 7 each represent that a different button has been pressed.

TimeFuzz was run on an openSUSE Leap 15.6 system running kernel version 6.4.0-150600.23.38-default. The runs were distributed across four servers, each equipped with 1 TB of RAM and an Intel Xeon E7-4880 v2 CPU operating at 2.5 GHz. AFL and IJon could not be run on this infrastructure and were instead run on the same setup as used in the Maze experiments. A virtual machine running Ubuntu-24.04 with kernel version 6.11.0-21-generic and a CPU clock speed of 2 GHz was used. This virtual machine ran

on ProxMox, with no overutilization of RAM nor physical CPU cores.

Each fuzzer was evaluated on all 32 levels of SMB for a maximum of 12h runtime. Each level was tested three times by each tool. While AFL and IJon were executed for the full duration, TimeFuzz was allowed to terminate once at least one passing input was generated. As in the Maze experiment, AFL and IJon were restricted to a single CPU core whereas TimeFuzz was allocated additional computational resources. Each instance of TimeFuzz utilized five CPU cores for computation and executed up to ten 10 concurrent tests. TimeFuzz was allocated additional resources for three main reasons. First, unlike AFL and IJon, which implement optimizations for process spawning, TimeFuzz does not currently incorporate such optimizations. Second, it was anticipated that TimeFuzz would need to execute a high number of DD tests, resulting in more overall test executions compared to AFL and IJon. Lastly, TimeFuzz cannot be run on a single core, as it requires both threads for computation and test execution, in addition to the tests itself.

In addition to the 12-hour internal runtime (i.e., the runtime excluding time spent on backups and exporting results), TimeFuzz is allocated an extra 1 hour of wallclock time to perform backups and export results. If TimeFuzz uses less than 1 hour for these tasks, it exits before reaching the full 13-hour wallclock limit. However, if it spends more than 1 hour on backups and had not yet completed the 12-hour internal runtime, the process is terminated early. In such cases, results are recovered by loading the last savefile and exporting all data generated up to that point.

While SMB does not expose Mario's position programmatically except for the visual output, IJon requires access to Mario's position in order to function. Therefore, TimeFuzz was permitted to use Mario's x-coordinate in a level as score value. This represents a restriction compared to IJon which utilizes both Mario's x- and y-coordinates. SMB was modified to print Mario's current position to the standard output on every frame. A test oracle was used to extract this information and to provide it to TimeFuzz. Because IJon only uses Mario's final position in its calculations, TimeFuzz was not permitted to apply the `Score Progress` DD algorithm for input minimization. Instead, it was run with the standard DD algorithm to ensure a fair comparison. Since DD requires a lot of tests to be computed and executed, TimeFuzz was granted additional resources compared to AFL and IJon.

The remainder of this chapter first presents the results of the SMB experiment with input restricted to jump states. Next, the results of SMB without input restrictions are presented. Finally the impact of the resource imbalance between TimeFuzz and AFL and IJon is analyzed.

7.2.1. Super Mario Bros Restricted Input

The results of AFL, IJon, and TimeFuzz on the input restricted version of SMB are shown in Table 5. The first column indicates how many times each tool was able to solve a level: ✗ denotes that no run was able to solve the level, while ✓ indicates all three runs were successful. AFL solved 7 levels, at least, once and completed 1 level in all three runs. IJon performed significantly better, solving 25 levels in all three runs and 3 additional levels at least once. In level 2-2, although Mario travels the full distance, he

	AFL						IJon						TimeFuzz					
Level	Solved	Time	% Distance	Execs per min	Execs best	Score per Length	Solved	Time	% Distance	Execs per min	Execs best	Score per Length	Solved	Time	% Distance	Execs per min	Execs best	Score per Length
1-1	2/3	04:08	100%	999	379633	51.62	✓	01:21	100%	400	44343	61.75	✓	00:01	100%	3554	8483	71.57
1-2	✗	01:23	57%	496	136415	36.16	✓	01:32	100%	340	36900	61.75	✓	00:03	100%	3358	11487	68.46
1-3	✗	00:03	63%	1037	6824	43.27	✓	01:23	100%	803	123701	40.95	2/3	01:58	100%	5941	468937	71.06
1-4	1/3	00:08	92%	607	19962	33.78	✓	00:37	100%	570	40992	80.77	✓*	02:28	95%	3327	643281	96.38
2-1	✗	07:09	61%	882	543753	18.83	✓	03:36	100%	300	85795	57.84	✓	00:13	100%	2017	24529	67.68
2-2	✗	05:25	59%	466	134605	24.83	✗	10:17	100%	116	69644	26.22	✗	06:05	82%	1612	425044	33.42
2-3	✗	01:05	42%	776	127198	46.06	✓	02:16	100%	402	62205	94.18	✓	00:02	100%	4094	9645	111.84
2-4	✗	00:18	59%	551	26227	59.67	✓	00:39	100%	449	32585	67.31	✓*	03:13	100%	3571	1364799	67.28
3-1	✗	00:09	56%	490	13224	47.63	✓	03:32	100%	353	88160	55.81	✓	00:03	100%	3245	11588	66.27
3-2	2/3	06:58	100%	1048	589268	30.02	✓	00:51	100%	457	50313	65.18	✓	00:04	100%	4154	20180	66.48
3-3	✗	00:25	63%	1256	46328	36.89	✓	00:38	100%	740	41018	57.14	✓	01:48	100%	7321	88975	80
3-4	✗	00:04	63%	1149	19890	40.45	✓	01:34	100%	570	89121	51.55	✓*	04:19	100%	3753	1059867	73.03
4-1	2/3	00:58	100%	806	109688	58.67	✓	00:32	100%	370	18736	94.18	✓	00:12	100%	4085	2893	105.26
4-2	✗	00:21	60%	714	63132	34.15	2/3	02:00	118%	431	78166	72.57	2/3	05:12	118%	1918	72557	86.73
4-3	✗	00:07	61%	1078	14977	26.52	✓	01:03	100%	620	62611	35.39	✓	05:10	100%	8024	114995	80.55
4-4	✗	00:31	66%	468	122731	51.08	✗	00:45	78%	578	53742	74.16	✗	01:20	78%	3352	1198985	25.82
5-1	1/3	08:34	87%	938	598926	33.67	✓	01:13	100%	550	63043	52.75	✓	00:04	100%	9685	57849	71.93
5-2	✗	00:10	51%	819	18921	40.73	✓	08:01	100%	261	126861	45.44	✓	00:03	100%	3071	11402	83.71
5-3	✗	00:34	59%	881	133311	22.98	✓	00:46	100%	792	51283	58.93	✓	00:38	100%	7038	356887	63.58
5-4	✗	00:33	59%	487	58452	49.34	✓	01:09	100%	372	39785	46.6	✓*	07:32	97%	3692	1558647	97.63
6-1	2/3	03:22	100%	857	308556	52.82	✓	00:46	100%	405	26671	55.81	✓	00:01	100%	4205	5745	70.43
6-2	✗	00:08	44%	431	14646	47.53	✓	01:54	100%	198	37273	49.09	✓	00:03	100%	2515	7236	64.83
6-3	1/3	10:33	67%	1857	942261	36.88	✓	00:51	100%	760	52220	53.1	✓	00:07	100%	8607	65341	91.55
6-4	✗	00:24	59%	930	103667	42.12	✓	00:44	100%	455	42646	39.72	✓*	03:57	95%	3113	1166338	95.88
7-1	✓	01:18	100%	795	150140	43.78	✓	00:53	100%	327	33277	51.75	✓	00:00	100%	5952	1066	61.87
7-2	✗	04:19	59%	502	152066	25.14	✗	10:54	79%	113	77169	22.77	✗	08:17	82%	1492	229317	24.49
7-3	✗	01:11	42%	571	65696	74.71	✓	00:42	100%	393	23252	81.34	✓	00:02	100%	5313	10328	96.73
7-4	✗	00:25	44%	697	78862	38.09	✓	00:33	100%	321	25142	71.73	✓*	04:28	100%	2655	7471	86.03
8-1	✗	00:01	28%	923	5068	59.68	✗	08:08	67%	331	192249	29.76	1/3	04:12	91%	1505	362100	86.81
8-2	✗	00:10	44%	965	27598	60.8	2/3	03:28	100%	299	86219	58.24	✓	00:05	100%	2692	13836	74.7
8-3	✗	00:04	45%	771	8110	53.08	✓	02:53	100%	289	52471	64.23	✓	01:21	100%	3778	9768	94.56
8-4	✗	00:00	29%	731	2896	37.5	2/3	06:04	100%	201	155585	81.42	2/3	09:16	100%	1330	883533	76.49

Table 5: Fuzzing results on SMB C with restricted input. In tests marked with * TimeFuzz was sometimes able to beat the boss fight, but was unable to complete the following cut-scene.

fails to enter the pipe at the end and thus cannot complete the level. TimeFuzz showed similar performance, solving 25 levels in all runs and an additional 4 levels at least once.

The second column reports the median runtime until either the best input was generated or the level was solved first. The fourth column shows the number of tests executed up to that point. On all levels AFL is able to solve, TimeFuzz is significantly faster and executes a fraction of the tests required by AFL. This in part also holds true for level 1-4 where AFL's median time is reported as 8 minutes. However, the successful run required 4 hours and 6 minutes with 118106 executed tests. TimeFuzz succeeds in less time but executes significantly more tests than AFL. Similarly, IJon outperforms AFL on all levels solved by AFL, including level 1-4. This holds true for both runtime and the number of executed tests.

TimeFuzz and IJon were able to solve a similar amount of levels. Excluding levels that either tools failed to solve, TimeFuzz found a solution faster than IJon in 17 levels whereas IJon was faster in 12. Both tools successfully found the warp zone in level 4-2, going above 100% score. While the runtime is not expressive due to the imbalance in resources, TimeFuzz required fewer tests than IJon in 16 levels, while IJon required fewer in 12. In almost all levels, the difference in the number of executed tests is significant. This is particularly true for boss levels completed by IJon.

The boss levels share two commonalities: (1) they include a cutscene at the end during which Mario's x position continues to increase, but player input is ignored; and (2) they do not terminate upon completion. IJon was able to consistently complete the cutscene whenever it solved a boss level. In contrast, TimeFuzz often struggled to complete the cutscene for reasons that remain unclear. In all affected cases, TimeFuzz successfully got past the boss and reached the start of the cutscene within a fraction of the reported time, typically between 6 and 60 minutes, and with a fraction of the tests executed. However, once the cutscene began, progress became extremely slow, resulting in high overall runtimes and test counts observed.

Tests that completed the cutscene, could not be classified as passing, since the levels did not cause SMB to exit as expected. Instead, the game continued into another level - one that was not the logically subsequent level in the game's progression. This behavior was unknown prior to the evaluation and could not be compensated for, leading to missclassified inputs and consequently inflating the number of tests executed by TimeFuzz. While IJon was similarly affected, it tended to produce longer inputs that sometimes extended well into the unintended next level. Comparing IJon and TimeFuzz only on non-boss levels that both tools successfully completed, TimeFuzz executed fewer tests in 15 levels, while IJon executed fewer tests in only 6 levels.

The comparison between IJon and TimeFuzz in terms of runtime and number of executed tests leaves room for interpretation. However, TimeFuzz clearly outperforms IJon with respect to the length of generated inputs. Specifically, TimeFuzz generated shorter inputs in 26 out of 28 levels that both fuzzers successfully solved. This is reflected in the last column of Table 5. It shows the score per length of the generated inputs. On average, TimeFuzz achieved a score per length of 20.4 points higher than that of IJon across the 26 levels. On one level, both tools performed equally well, and in the remaining levels, IJon beats TimeFuzz by 5 points. These results show that although TimeFuzz

may occasionally execute more tests than IJon, it consistently produces smaller inputs.

7.2.2. Super Mario Bros Unrestricted Input

While IJon and TimeFuzz solved a similar amount of levels for the input restricted version of SMB, this is no longer the case on the unrestricted version. The results of this experiment are shown in Table 6. AFL was unable to solve any level, and in all cases, its progress did not exceed 28% of the total distance. IJon managed to solve a single level once, showing substantially more progress than AFL on all levels. TimeFuzz was able to fully solve 5 levels and solve 9 levels at least once, demonstrating a clear advantage in this version.

Comparing the progress made across all levels, TimeFuzz outperforms IJon on 21 levels and matches IJon's progress on 4 additional levels. Furthermore, TimeFuzz requires fewer tests on 14 of these levels, sometimes achieving significantly greater progress while executing substantially fewer tests. For the remaining levels, the results align with expectations that for achieving greater progress typically a higher number of tests is required.

While TimeFuzz significantly outperforms both AFL and IJon in terms of distance covered, the best inputs it generates are often much longer than those generated by AFL and IJon. The column `lLength per Score` shows the number of user-events required for Mario to move one x-coordinate unit. Although the values for AFL are largely uninformative because of its poor overall performance, the differences between IJon and TimeFuzz are both relevant and significant. This poor performance of TimeFuzz can be explained by the structure of the inputs it produces. IJon generates inputs by mutating existing ones, often resulting in long sequences of repeated user-events. In contrast, TimeFuzz tends to produce inputs with evently distributed user-events, rarely generating extended subsequences of identical actions. This becomes problematic for one particular reason: In SMB, Mario only begins sprinting after the sprint button has been held down for a sustained period. While IJon can naturally exploit this feature because of the structure of generated inputs, TimeFuzz rarely triggers sprinting in its generated inputs. Although DD could, in principle, generate inputs that maintain sprinting behavior, this outcome was not observed in practice.

Even though the oracle used by TimeFuzz was not designed to detect bugs uncovered during fuzzing, TimeFuzz was nevertheless able to exploit at least one bug in SMB to its advantage. In level 4-4, where IJon fails to make any further progress, TimeFuzz generates an input that causes Mario to glitch through the wall, resulting in significantly greater progress. This behavior is illustrated in Figure 10.

	AFL						IJon						TimeFuzz					
Level	Solved	Time	% Distance	Execs per min	Execs best	Length per score	Solved	Time	% Distance	Execs per min	Execs best	Length per score	Solved	Time	% Distance	Execs per min	Execs best	Length per Score
1-1	✗	10:31	23%	2544	1694963	0.53	✗	11:52	66%	755	539934	0.45	✓	01:46	100%	2428	254521	1.09
1-2	✗	07:02	18%	2338	827499	0.7	✗	09:52	52%	704	417168	0.49	✓	04:09	100%	2939	714557	0.93
1-3	✗	06:30	25%	1614	1120032	0.75	✗	10:30	73%	978	614193	0.46	✗	03:09	50%	3142	366024	0.58
1-4	✗	07:19	25%	2763	1372188	0.57	✗	09:24	93%	883	609383	0.46	2/3	08:00	93%	1059	753328	0.69
2-1	✗	11:22	19%	2492	1732432	0.73	✗	08:23	37%	799	447971	0.55	✗	04:46	52%	1526	296562	3.45
2-2	✗	05:08	20%	2580	933791	0.82	✗	11:58	47%	500	358535	0.79	✗	00:12	1%	681	10410	7.19
2-3	✗	07:53	17%	860	397653	0.75	✗	09:44	53%	773	471600	0.48	1/3	02:02	66%	671	203381	0.55
2-4	✗	07:21	25%	2562	1112192	0.56	✗	11:20	72%	953	673963	0.48	1/3	02:29	92%	1427	363431	0.7
3-1	✗	09:25	20%	2406	1263821	0.68	✗	11:17	40%	828	593089	0.48	1/3	02:22	54%	737	472557	0.91
3-2	✗	06:58	18%	1361	671913	0.79	✗	10:50	62%	736	503387	0.45	✓	02:25	100%	1879	250866	1.48
3-3	✗	08:03	25%	1469	690951	0.52	✗	10:54	80%	974	687313	0.45	✗	04:58	76%	849	470983	0.5
3-4	✗	09:23	21%	1127	587535	0.65	✗	10:43	69%	784	527707	0.48	✗	03:15	92%	650	261076	0.64
4-1	✗	11:32	18%	2189	597451	0.58	✗	10:45	55%	620	389150	0.47	1/3	03:35	87%	763	374802	0.68
4-2	✗	08:14	22%	1622	843151	0.62	✗	11:30	39%	948	641464	0.53	2/3	04:46	118%	2073	1378092	2.72
4-3	✗	08:05	28%	2873	1562960	0.56	✗	10:20	59%	983	644675	0.47	✗	02:09	68%	1644	498704	0.47
4-4	✗	09:00	22%	2574	864727	0.73	✗	10:46	69%	864	595552	0.46	1/3	02:42	79%	2184	1622335	1.03
5-1	✗	07:08	22%	2474	1205870	0.66	✗	10:17	45%	784	503562	0.5	✗	01:20	78%	1777	369207	0.84
5-2	✗	03:10	20%	2376	547732	0.58	✗	11:13	50%	816	547386	0.47	1/3	05:06	93%	1915	1096650	0.95
5-3	✗	07:06	24%	2750	1154093	0.53	1/3	11:08	72%	972	685717	0.46	✗	01:13	49%	3607	418617	0.54
5-4	✗	08:50	22%	2600	1525188	0.58	✗	07:49	61%	897	461617	0.47	✗	04:34	92%	1570	872325	1.14
6-1	✗	06:49	22%	2610	1252365	0.57	✗	10:29	62%	849	561148	0.44	✓	01:43	100%	1834	307560	0.68
6-2	✗	03:26	17%	2313	721521	0.69	✗	10:45	40%	800	538418	0.57	✗	06:23	59%	2145	1570360	1.81
6-3	✗	09:36	22%	2687	1867773	0.56	✗	10:35	68%	885	559751	0.48	✗	05:26	68%	3128	784570	0.62
6-4	✗	11:00	26%	1605	1065115	0.59	✗	08:50	63%	1061	594849	0.46	✓	04:00	93%	2717	668771	0.54
7-1	✗	09:10	21%	2642	1039795	0.7	✗	11:43	64%	849	601581	0.45	2/3	01:22	100%	1669	358092	2.25
7-2	✗	04:53	23%	2450	957114	0.66	✗	11:46	48%	471	336030	1.21	✗	00:09	1%	856	36573	11.9
7-3	✗	10:55	17%	2621	1423708	0.75	✗	11:10	56%	856	565149	0.45	✗	02:55	56%	763	263454	0.47
7-4	✗	10:11	16%	1042	706810	0.7	✗	11:56	64%	788	557572	0.45	✗	02:28	95%	2411	452620	0.75
8-1	✗	07:26	10%	1341	768736	0.67	✗	10:38	25%	800	528928	0.47	✗	05:53	25%	911	402311	0.82
8-2	✗	08:41	16%	2503	1486800	0.53	✗	10:54	50%	856	580185	0.48	✗	03:26	72%	1608	846929	0.87
8-3	✗	10:53	20%	791	521713	0.6	✗	10:06	44%	995	646988	0.5	✗	01:10	40%	1019	347677	0.97
8-4	✗	08:35	12%	2482	1430132	0.81	✗	08:25	32%	880	511532	0.46	✗	01:45	24%	2197	597874	0.5

Table 6: Fuzzing results on SMB C with unrestricted input.

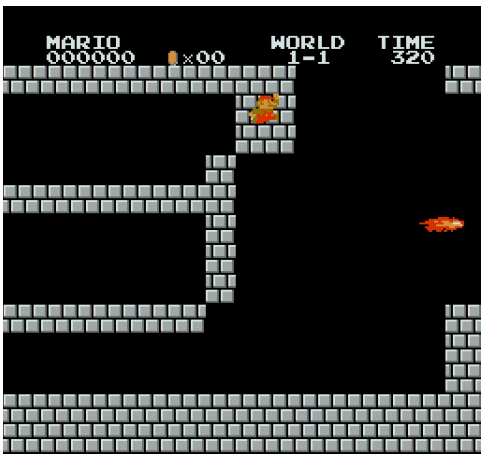


Figure 10: Mario passes through the wall in level 4-4 due to rapidly alternating input.

7.2.3. Conclusion and discussion of resources

While TimeFuzz does not outperform IJon in all aspects, the results from both experiments demonstrate that it consistently outperforms AFL across all evaluated metrics. Furthermore, TimeFuzz achieves results comparable to IJon in many scenarios, surpassing it in terms of input length efficiency in the restricted SMB version, and outperforming it in both distance covered, tests executed and levels solved in the unrestricted SMB version.

TimeFuzz was granted significantly more computational resources than IJon. However, in SMB's restricted mode, it did not utilize this advantage in 16 of 32 levels, as it generated its best or first passing input using fewer tests than IJon. Based on a 720-minute runtime, and considering IJon's average execution rate of 424 tests per minute and AFL's 812, an upper bound between 305 000 and 584 500 executed tests is reasonable for fair comparison, particularly since TimeFuzz was intentionally allocated more resources to accommodate the high number of tests expected from DD and the non-existence of optimization for process spawning. The exact number of tests generated by DD is shown in Table 9 in chaper 7.4. Under this threshold, all levels except the boss levels meet the criteria for fair comparison. As previously discussed, the boss levels were effectively solved much earlier (i.e., the boss was passed), and the large number of test executions resulted only after the cutscene began.

A similar argument can be made for the unrestricted version of SMB. Based on IJon's observed average test execution rate of 832 tests per minute, and AFL's much higher rate, any number of tests below 800 000 should be acceptable (equivalent to approximately 16 hours of execution at IJon's rate, but significantly lower than what AFL would be able to execute in 12 hours). TimeFuzz exceeds this threshold on only four levels. Since TimeFuzz outperforms IJon in terms of distance covered on 21 levels, this does not affect the overall conclusions of the experiment.

The fact that TimeFuzz was afforded a significantly greater computational resources also highlights a key weakness of its approach. In the unrestricted SMB experiment,

only 10 levels exhibit median runtimes exceeding 4 hours, while none exceeded 8 hours. This indicates that, although TimeFuzz can efficiently leverage available computational resources, it is equally prone to getting stuck. The core issue is found in TimeFuzz's dependence on Mario's position: if Mario hits a dead-end and TimeFuzz is not able to backtrack from this position, TimeFuzz fails to make any further progress.

7.3. TimeFuzz on slow PUT

In the first three experiments, TimeFuzz was compared to AFL and IJon on PUTs with relatively short execution times. Both PUTs were implemented in C and could be instrumented by a grey-box fuzzer. In this fourth experiment, TimeFuzz is evaluated on a PUT that cannot be instrumented by AFL or IJon. This PUT also has a significantly longer execution time. It will be used to assess whether TimeFuzz scales well to more demanding applications.

In the experiment, TimeFuzz is applied to the Gym-Retro framework developed by OpenAI. Gym-Retro is a Python based emulation framework capable of executing a wide variety of retro video games, including Super Mario Bros, Sonic the Hedgehog, and others. As in the previous experiment Super Mario Bros is used once again. The key difference between this version and the C based implementation is that individual levels cannot be selected. Instead, Mario can only start at the beginning of one of the eight worlds.

Gym-retro allows retrieval of both Mario's in-game score[10] and his x-coordinates in the world at every frame. TimeFuzz uses Mario's x-coordinate as the primary score and the in-game score as secondary score. Due to time constraints, only the primary score will be used to steer input generation.

TimeFuzz is evaluated in two configurations. In the first configuration, TimeFuzz generates inputs randomly (referred to as `TimeFuzz Random`). Inputs are neither extended nor minimized using DD. In the second configuration, inputs are extended and and minimized using DD. Since test execution is expensive and a score value is available for each user-event, TimeFuzz uses the `Score Progress` DD algorithm to minimize inputs while executing as few tests as possible. The two configurations are compared in terms of distance covered, the number of tests required to generate the inputs, the total number of tests executed, and the input length per score. To ensure continuous progress within the time-limit, the user-events have been restricted to right, down, jump, and sprint.

Each configuration was executed for a total of 12 hours. Each world was tested three times. Both versions of TimeFuzz were run on an openSUSE Leap 15.6 running kernel version 6.4.0-150600.23.38-default, equipped with 1 TB of RAM and an Intel Xeon E7-4880 v2 CPU with a speed of 2.5 GHz. Each instance of TimeFuzz was permitted to use up to 60 CPU cores, divided between input generation and test execution. TimeFuzz Random was assigned 25 thread for input generation and 35 concurrent test executions, while standard TimeFuzz used 20 threads for input generation and 40 for test execution. This configuration was chosen to balance the computational cost of input generation: TimeFuzz Random was set to generate significantly longer inputs to guarantee that inputs would have the length to complete multiple levels at once if enough hurdles are passed by the inputs. Although there is an imbalance in the number of concurrent tests, it will be shown that this does not affect the validity of the results. The exclusion tree was disabled in both configurations to reduce memory consumption, as preliminary experiments indicated high RAM usage[11].

[10] This refers to the actual in-game score which can be earned for defeating enemy entities, consuming mushrooms, completing a level, etc.

[11] Subsequent analysis revealed that the high RAM usage observed during preliminary experiments

	TimeFuzz			TimeFuzz Random		
World	Time	Distance	Execs best	Time	Distance	Execs best
1	10:27	207%	291091	02:44	41%	110481
2	06:39	137%	230181	02:30	34%	328393
3	07:26	155%	260076	05:19	40%	326859
4	08:30	189%	222814	06:28	50%	316725
5	10:50	185%	327059	06:23	47%	237867
6	09:40	156%	322993	07:41	90%	451676
7	10:47	136%	323509	02:02	38%	138553
8	06:36	58%	228504	05:28	20%	170704

Table 7: The time after which the best inputs were generated, as well as the median distance traveled by Mario and the number of tests executed until the best input was generated.

	TimeFuzz			TimeFuzz Random		
World	Execs per min	Total Tests	Length per Score	Execs per min	Total Tests	Length per Score
1	449	323425	0.95	306	225772	4.28
2	496	357773	1.93	1288	928522	7.57
3	466	335925	1.40	835	599985	4.67
4	511	285325	0.87	911	652863	1.88
5	498	359063	0.96	924	663279	3.3
6	546	393254	0.89	1256	902109	1.60
7	493	355150	1.12	1384	1019074	1.07
8	640	461284	0.70	534	383788	2.48

Table 8: The average test per minutes as well as the total number of tests executed, and the length efficiency of generated inputs.

Table 7 presents the results of the experiment. Neither version of TimeFuzz was able to complete an entire world within the time limit. The table reports the median runtime until the best input was generated. Although TimeFuzz requires significantly more time to generate its best inputs, these inputs cover substantially greater distances than those generated by TimeFuzz Random. A distance of 100% corresponds to the completion of a level within the respective world. While TimeFuzz Random does not complete any level, it manages to produce inputs that reach 90% of the distance for level 6-1. TimeFuzz successfully completes at least one level in every world except for world 8. Notably, TimeFuzz completes two levels in world 1 and achieves the same in world 4 in one out of three runs. These results demonstrate that TimeFuzz consistently outperforms random input generation across all test instances. In addition, it generates significantly better inputs while executing fewer tests for 4 out of 8 worlds.

TimeFuzz outperforms random generation with respect to the length per score of the generated inputs, as shown in Table 8. This result is expected, given that TimeFuzz Random does not apply DD to reduce input length. However, despite input restrictions in this version of SMB, the length per score remains comparable to that of the unrestricted

was not caused by the exclusion tree. Instead, it was attributed to memory management issues in other components of TimeFuzz. These issues were later resolved, and the SMB C experiments were conducted with the exclusion tree enabled. However, it was not possible to repeat this experiment due to time constraints.

SMB C which results are shown in Table 6. This can be attributed to the fact that, although Mario is able to sprint more frequently in this experiment, he still does not maintain this behavior throughout the entire world. Additionally, there are numerous instances where DD failed to effectively remove scenarios in which Mario got stuck in front of obstacles. Furthermore, specific levels, such as 2-2, pose unique challenges. Levels 2-2 and 7-2 are water levels. In these levels, Mario must sink underwater to bypass certain obstacles. Since TimeFuzz frequently generates jump events that propel Mario to the water's surface, the tool's is unable to complete these levels. As a result, runs for worlds 2 and 7 show a significant increase in the length per score, while in the unrestricted SMB C experiment this resulted in TimeFuzz being unable to make any progress beyond 1% on levels 2-2 and 7-2.

Table 8 also reports the total number of tests executed and the test executions per minute. For all worlds except worlds 1 and 8, TimeFuzz Random executes between two to three times more tests than TimeFuzz, despite being limited to 35 concurrent test executions compared to TimeFuzz's 40. This discrepancy is due to the significantly longer average runtime of a test in TimeFuzz, as its inputs cover greater distances, resulting in longer overall execution times. For instance, in world 6, TimeFuzz starts with an average execution time of just over 1 second in the first generation, which increases to around 5 seconds by the 2.5-hour mark. In contrast, the average test runtime for TimeFuzz Random remains around 1.4 seconds over the full 12-hour runs. This holds true for all worlds except for worlds 1 and 8. In most worlds, Mario frequently dies to enemy entities. In contrast, in worlds 1 and 7, Mario tends to get stuck in front of obstacles and only dies when the level timer expires, resulting in very long test executions for TimeFuzz Random. While this also occurs with standard TimeFuzz, many of its inputs lead to Mario's death by falling out of the world, or by dying to enemies, thereby shortening the execution time. Overall, allocating more computational resources to input generation rather than test execution allowed TimeFuzz Random to generate inputs more quickly and execute significantly more tests in most worlds but did not impact the results of the comparison.

In conclusion, TimeFuzz demonstrates strong scalability to high-runtime PUTs and is capable of consistently producing high-quality results.

7.4. Comparing Delta Debugging Algorithms

This section of the evaluation uses general statistics from the previous experiments to demonstrate the importance of DD in the context of this works approach and includes a focused experiment comparing the two implemented DD strategies.

A substantial amount of data was collected during the execution of TimeFuzz. Table 9 presents a selection of key important metrics from the experiments conducted. Across all experiments, approximately 183 million tests were executed, 97 million of those were generated by DD. Although the proportion varies between experiments, DD consistently accounts for a significant share of total tests executed. This shows the substantial impact of DD on the overall number of executed tests, and consequently, on the overall runtime.

To better understand the implications of the selected DD algorithm, the use of exclusion trees, and the execution mode (batch or non-batch mode), a small supplementary

Metric	SMB C Restricted	SMB C Unrestricted	SMB Python	SMB Python Random	Maze	TOTAL
Generated Tests	41 778 572	24 157 905	2 586 251	16 126 178	2 063 305	86 712 211
Delta Debugging Tests	10 683 486	63 927 579	6 027 349	0	16 441 135	97 079 549
Skipped Delta Debugging	198 408	53 913 562	460 485	0	138	54 572 593
Excluded Delta Debugging	15 773 426	55 671 327	0	0	38 772	71 483 525
Approximated Delta Debugging	15 835 767	126 471 077	0	0	0	142 306 844
Excluded Generation	22 749 693	102 181 085	0	0	34 937 830	159 868 608
Excluded Total	38 523 119	157 852 412	0	0	34976602	231 352 133
Total Executed	52 462 058	88 085 484	8 613 600	16 126 178	18 504 440	183 791 760
Total Not Executed	54 557 294	338 237 051	460 485	0	34 976 740	428 231 570

Table 9: The overall statistics of test execution and exclusion for each of the experiments, and the total over the respective row.

	Score Progress				Standard DD			
Metric	None	Batch	EXCL	EXCL + Batch	None	Batch	EXCL	EXCL + Batch
Reduced Length	26%	32%	33%	28%	42%	57%	38%	51%
New Score	115%	117%	125%	114%	138%	142%	167%	144%
DD Tests	36331	30243	37371	26975	150279	39329	56887	7761
Skipped	0	7439	649	7623	0	21061	0	15229
Prefix	0	0	4720	5133	0	0	21332	4966
Approx	0	0	0	0	0	0	60978	20575

Table 10: The overall statistics of the reduced length, the new score, the number of tests executed and those excluded for each of the experiments.

experiment is performed. For this experiment, 10 program inputs generated by TimeFuzz Random on the Python based SMB were selected. These inputs were reduced using both `Score Progress` and standard DD algorithms. Each algorithm was executed in four configurations: (1) exclusion tree only, (2) exclusion tree and batch mode, (3) neither exclusion tree nor batch mode, and (4) batch mode only. All configurations were run with an acceptable loss of 0. Although the main experiments allowed for non-zero acceptable loss, using a strict threshold here enables a direct comparison of the core algorithms.

It is expected that `Score Progress` will outperform standard DD in terms of the number of tests executed across all configurations. Furthermore, both algorithms are expected to produce minimized inputs of comparable length, as `Score Progress` simply restricts the regions of the input to which DD is applied.

The expectations were only met to a certain degree. The results are presented in Table 10, which shows the reduced length as a percentage of the original input length and the new score as a percentage of the original score. The `New Score` value is higher than the original score for both algorithms across all four configurations. As previously discussed in Chapter 2, a small change in the middle of an input sequence can significantly affect how the PUT processes the remainder of the input sequence. This phenomenon is observed here. While `Score Progress` shows an average increase in score value of 15%-25%, the standard DD algorithm results in average increases of 38%-67%. This difference arises because `Score Progress` applied DD only to selected segments of the input whereas standard DD applied reductions across the entire sequence, creating more opportunities

for score increases.

The results clearly show that `Score Progress` consistently produces further minimized inputs. The original length of all inputs reduced in this experiment ranged between 5000 and 8000 events; therefore, a 10% reduction in input length is significant. While reduced lengths for `Score Progress` and standard DD are similar when the exclusion tree is enabled, substantial differences appear accross the other three configurations. Although inputs reduced by standard DD tend to achieve higher score values, an average increase of 30% may not be particularly meaningful, given that all original score values ranged between 500 and 700.

Chapter 6.2 briefly explained that the choice between batch mode and non-batch mode is not straightforward, as batch-mode may miss inputs that could lead to substantial score increases. The results show that when the exclusion tree is disabled, batch mode generally outperforms non-batch mode in terms of the new score. However, when the exclusion tree is enabled, non-batch mode performs better.

Lastly, the expectation that `Score Progress` would require fewer test executions than standard DD was only partially confirmed. `Score Progress` performs significantly better when both batch mode and exclusion tree are disabled. However, this advantage diminishes when either feature is enabled, and disappear entirely when both are enabled. In this final configuration `Score Progress` requires significantly more tests than standard DD.

8. Discussion

In this chapter, the results of the evaluation will be interpreted and the research questions of this thesis are addressed.

RQ1: How does TimeFuzz compare to the popular grey-box fuzzing tool AFL and its extension IJon, focusing on deep state fuzzing? That is, can TimeFuzz solve similar problems in comparable time, despite being a black-box fuzzer without access to internal program data?

TimeFuzz is able to solve problems similar to those solved by IJon. It not only generates winning strategies for problems IJon can solve, but also generates winning strategies for problems that IJon struggles with. This is demonstrated in the Maze experiment and in SMB C without input restrictions.

In the Maze experiment, it does so in less time and with significantly fewer executed tests. In SMB C, it generates winning strategies with fewer executed tests on most levels but requires significantly more on others. Due to a resource imbalance in the SMB C experiments, runtime cannot be directly compared. However, if the average number of tests had been similar, it is reasonable to assume that TimeFuzz would still outperform IJon in terms of runtime, as it requires fewer tests for most levels.

Compared to AFL, TimeFuzz performs significantly better in all experiments. TimeFuzz solves more problems in less time while executing fewer tests.

TimeFuzz is a black-box approach that does not have access to internal program data and, therefore, cannot use state-of-the-art techniques like code-coverage. Nevertheless, TimeFuzz outperforms both IJon and AFL in all conducted experiments.

RQ2: To what extend are winning strategies generated by TimeFuzz minimal, and how do they compare to those generated by AFL and IJon?

In the experiments conducted, TimeFuzz generated a significant number of winning strategies for both the mazes and Super Mario Bros, in multiple configurations. TimeFuzz successfully generated minimal inputs for all mazes whereas IJon failed to do so in several cases. AFL was unable to generate winning strategies for any of the mazes.

For Super Mario Bros C with restricted input, TimeFuzz generated shorter winning strategies than AFL and IJon. However, these inputs were not minimal. All inputs generated by TimeFuzz either get stuck at least once or slow down due to hitting a wall or ceiling. A minimal input would need to complete the levels without getting stuck or slowing down.

The verdict is even worse for Super Mario Bros C with unrestricted input and the Python based version. In these experiments, TimeFuzz was able to generate winning strategies for SMB C and complete multiple levels per world in the Python based version, but the resulting inputs were not minimal. This can be attributed to the randomness in TimeFuzz's input generation. Since SMB requires long, continuous sequences of user-events to both use the sprint feature and consequently to generate minimal inputs, and to the fact that DD does not yield globally minimal inputs, but rather 1-minimal

ones. Additionally, the Python based SMB experiment used `ScoreProgress` for input reduction. `ScoreProgress` only applies DD on a subset of a generated input. Although the experiment that compared the DD algorithms showed that `ScoreProgress` generates shorter inputs, it is not clear whether this method yields 1-minimal results,

TimeFuzz is unable to use the game's sprinting feature consistently, and in some cases at all, resulting in long and relatively slow inputs. In contrast, IJon often used sprinting both in levels it solves and in those it failed to solve. As a result, IJon was able to generate shorter inputs than TimeFuzz.

While TimeFuzz generates more winning strategies than IJon, its ability to generate shorter inputs varies significantly across experiments. Based on the available data, no definite conclusion can be drawn regarding which tool produces shorter inputs. A reliable conclusion would require further evaluation.

RQ3: How effective and efficient is the Score Progress delta debugging algorithm and how does it compare to the standard DD algorithm?

To evaluate and compare the proposed `Score Progress` algorithm with the standard DD algorithm, a small-scale experiment was conducted. Its results were already discussed in Chapter 7.4. The experiment was expected to demonstrate that `Score Progress` would outperform standard DD on the specific test cases it was designed for. However, the results are mixed. While `Score Progress` can reduce inputs further, standard DD achieves higher score increases. Additionally, `Score Progress` quickly looses its advantage in terms of the number of executed tests when other optimizations are enabled. Nonetheless, the experiment shows that DD can not only reduce input sequences, but also actively improve progress. Unlike random input generation which produces highly varied inputs, DD creates refined variations of already promising results. This resembles the generation strategies of mutation based fuzzers, which creates subtle variations through iterative refinements, a technique proven effective in numerous prior studies, including AFL and IJon.

The experimental results are largely inconclusive, which each algorithm excelling in different areas. However, it demonstrates that `Score Progress` performs better than standard DD in the most critical aspect of input reduction being the size of the minimized inputs. In conclusion, the `Score Progress` algorithm is effective at reducing input size and efficient in the number of tests executions.

That said, the current evaluation of DD does not address all relevant aspects. `Score Progress` implements an additional mode in which inputs extended from an already minimized input are not fully re-minimized. Instead, DD is applied selectively to stagnant subsequences in the newly generation portion of the input. This optimization, not covered in the experiment, was applied in the Python based SMB experiment and is expected to further reduce the number of required test executions compared to standard DD.

Furthermore, the experiment only considered ten test cases, all of which were relatively long inputs with low progress. Expanding the experiment to a broader range and larger set of test cases could yield different results and potentially alter the conclusions drawn here.

Metric	Depth	Nodes	Leaves	Min Size MB
Small E	34	2296	1058	0.14
Small H	175	171985	1981	10.5
Large E	82	18684	8805	1.14
Large H	376	52518043	378087	3205.45
Super E	139	526096	218723	32.11
Super H	673	58531576	243036	3572.48

Table 11: Average values for the exclusion tree on the Maze using TimeFuzz Score.[12]

Metric	Depth	Nodes	Leaves	Min Size MB
Small E	192	110348	11931	6.74
Small H	265	85830	668	5.24
Large E	2695	525205	132457	32.06
Large H	4384	1405922	2706	3205.45
Super E	1864	593173	255629	36.2
Super H	5777	9092468	12127	554.96

Table 12: Average values for the exclusion tree on the Maze using TimeFuzz Length.[13]

RQ4: How effective is the usage of an exclusion tree to reduce the number of executed tests and how much memory is needed to maintain it?

The exclusion tree has proven highly effective. Table 9 presents the total number of executed tests and excluded tests across all experiments. In total, 183 million tests were executed, while 428 million were generated but not executed. Of the 428 million not executed tests, 373.6 million tests were excluded by the exclusion tree, either by prefix or approximation. The majority of these exclusions, 284 million, occurred in the SMB C experiment with unrestricted input.

Overall, enabling the exclusion tree drastically reduces the number of executed tests. However, this comes at a cost. The current implementation requires approximately 64 Bytes of RAM per node. Additional memory is required for connections between nodes and tree management.

Tables 11 and 12 present the average depth, size, and node count of exclusion trees for the various mazes using TimeFuzz Score and TimeFuzz Length respectively. In the easy versions of the mazes, the leaves contain both failing and passing inputs. In the hard versions, only passing inputs are represented. The results are in line with expectations. Easy mazes exhibit significantly shorter maximum lengths of generated inputs and fewer nodes, since many inputs are excluded. In contrast, harder mazes, where no inputs fail, produce much larger exclusion trees.

The minimal RAM required, shown in the tables, is estimated as 64 Bytes times the number of nodes. This estimate excludes memory for connections between nodes, management overhead, and OS-related usage. Nevertheless, the estimate exceeds 3 GB for the large hard mazes and the super hard Maze for TimeFuzz Score.

[12]TimeFuzz Score generated a significant number of passing inputs for the hard large Maze. As each of input was minimized using DD, the exclusion tree grew exponential in size.

[13]The hard small Maze was solved more quickly than its easier counterpart, resulting in a smaller exclusion tree.

In the unrestricted mode of SMB C, the values are even higher, averaging 69.7 million nodes and requiring approximately 4.2 GB of RAM. The largest exclusion tree observed across all experiments consists of 188.7 million nodes with an estimated size of 11.5 GB. Appendix A provides additional statistics, including the average number of nodes, depth, and leaf nodes, as well as memory estimates for the restricted and unrestricted SMB C experiments. For larger experiments, this could limit the usability.

The problem of such large exclusion trees extends beyond memory consumption. Because TimeFuzz periodically saves all state data to disk, including the full exclusion tree, large trees result in significantly larger savefiles. Although these files are compressed, the increased size still leads to substantial disk space requirements.

Although the usage of an exclusion tree requires substantial memory and disk space, it has proven to be highly effective.

RQ5: To what extent does TimeFuzz adhere to the core principles outlined in the introduction?

Generality. The experiments demonstrate that TimeFuzz is applicable to varied interactive applications. As shown in the Maze experiment, TimeFuzz can communicate directly with a PUT, monitor whether the PUT consumes user-events, measure reaction times, and determine the point of failure in user-event sequences. It can dump the full generated input to a PUT if needed and can extract point of failure information via a user defined oracle. Furthermore, TimeFuzz supports intermediate APIs for user-event dispatching and can rely on API-reported outcomes evaluated by the oracle, instead of collecting all data itself. Additionally, the framework offers cross-platform support for both Linux based systems and Windows. Although individual user-event dispatchment is unavailable on Windows, all other features remain functional. This makes TimeFuzz suitable for testing a wide variety of PUTs, regardless of programming langue.

Scalability In the evaluation TimeFuzz was executed on different numbers of computation threads and concurrent tests. In all experiments TimeFuzz performed well, using the resources available to its maximum. It was shown that TimeFuzz scales at least to 25 computation threads and 40 concurrent tests without issues. However, individual dispatching user-events to the PUT results in increased execution times for PUTs with very fast response times. This overhead can be attributed to the execution handler which becomes the bottleneck for test execution times.

Modularity and Extensibility The design presented in 5.3 has been fully implemented and is both modular and extensible. Although adding new algorithms generally requires little effort, certain components of the framework still require manual adaptation depending on the specific algorithm being integrated. One such component is the user interface, which requires direct access to some of the framework's internal classes and therefore must be updated accordingly.

Customizability TimeFuzz offers extensive customization options for configuring the PUT, managing communication with the PUT, and defining test oracles. Additionally, it provides a wide range of settings for input generation, delta debugging, save files, termination conditions (i.e., when TimeFuzz should stop), and allocation of computational

resources. Overall, these features allow TimeFuzz to be effectively adapted to a variety of individual use cases.

Robustness TimeFuzz regularly creates backups of all data generated up to the point of saving, allowing users to pause and resume sessions without data loss. While this enhances robustness, it also requires a significant amount of disk space and time to write the backup itself.

Overall, TimeFuzz adheres well to its design principles, though there remain opportunities to improve efficiency, particularly in savefile handling and the implementation of the execution handler.

9. Threats to Validity

The evaluation of this work has the following threads to validity.

9.1. Internal Threats

There are several threads to internal validity (correctness of the evaluation).

Some test cases were executed using different versions of TimeFuzz. TimeFuzz comprises over 32 000 lines of code and includes a number of complex algorithms. While most bugs were fixed prior to the experiments, some issues only manifested during the evaluation phase. Consequently, certain experiments, particularly individual levels of both SMB C and the Python based SMB, were run using different versions of TimeFuzz.

The identified bugs did not affect input generations or DD. Instead, they were limited to issues such as deadlocks and crashes to desktop (CTDs), which could prevent a test run from completing. Nonetheless, these fixes might still have had an effect on overall performance. One additional fix involved memory management and significantly reduced memory consumption. This specific fix only affected the Python based SMB experiments, as they were the first to run. While it may have had a minor impact on memory allocation speed, fuzzing on Python based SMB is bottle-necked by test execution time, not memory allocation, and thus the overall performance impact should be negligible.

Aside from the memory management issue, none of the bug fixes altered TimeFuzz's algorithms or scheduling mechanisms. They solely ensured that test runs could complete successfully. Therefore, it is unlikely that these fixes affected the experimental results.

All TimeFuzz SMB experiments were distributed across four physical servers. Although the servers feature identical hardware, variations in installed or concurrently running software may have introduced differences in performance across runs. Furthermore, the servers were not used exclusively for this work's experiments; they were accessible to a significant number of users.

During occasional observation, no excessive system load was observed on any of the servers. However, scheduling and user activity could still have impacted the performance. To mitigate the potential effects of interference, each experiment was repeated three times.

AFL and IJon, in the versions used in this work, are based on outdated libraries and compilation tools. Their llvm mode was originally compiled with Clang 6.0 and LLVM 6.0 whereas the currently available version is 19.0. Both tools do not compile with modern versions of LLVM and could only be compiled with LLVM 6.0. Since this version is no longer available on modern systems and the compilation failed with manually built versions, both AFL and IJon had to be compiled on Ubuntu-18.04. However, they were executed on Ubuntu-24.04, which may have introduced performance variances and could affect the reproducibility of results.

A similar situation applies to TimeFuzz. Written in modern C++, TimeFuzz requires Clang 18.0 for compilation. As Clang 18.0 is not available on openSUSE and downgrading the codebase to Clang 17.0 was not feasible, TimeFuzz was compiled on Kali GNU/Linux

2024.4. Any performance impact resulting from these platform differences should be mitigated by the evaluations focus on the number of tests executed rather than runtime.

Finally, TimeFuzz was run distributed on four physical servers for the SMB C experiments. In contrast, AFL and IJon were run on a virtual machine with a different operating system and a different underlying SPU speed. This setup was necessary as AFL and IJon could not be executed on the physical servers. The virtual machine was assigned CPU cores and RAM exclusively, and no resource overcommitment occurred. While this introduces a degree of heterogeneity in the test environment, the impact can be considered negligible due to greater overall resources allocated to TimeFuzz. Additionally, differences in server architecture and resource allocation are mitigated by prioritizing comparisons based on the number of tests executed, over raw runtime comparisons.

9.2. External Threats

In addition to internal threads, TimeFuzz's evaluation is also affected by a number of external threads influencing generalizability.

The evaluation targets selected for this work are limited in scope. Effectively, TimeFuzz was only evaluated on two distinct programs. Although the experimental goals and the behavior of the programs varied across different configurations, because of the differences in input and world structures in the experiments, the underlying software remained essentially the same. This introduced a potential selection bias, as these programs may not be representative. While TimeFuzz has demonstrated effectiveness when optimizing based on input length in the Maze experiments, the Maze experiment is an artificial use case. As such, no general conclusions can be drawn from it regarding TimeFuzz's applicability to real-world applications.

Moreover, for both the mazes and SMB, a score value can be easily obtained that increases continuously with valid progress toward the goal. Such a score may not be available for other applications. As TimeFuzz has not been evaluated on programs without a continuously increasing score, no conclusions can be drawn about its effectiveness in such scenarios.

Another factor limiting generalizability is TimeFuzz's behavior when encountering dead-ends. While TimeFuzz is able to backtrack on generated inputs, the experiment performed for SMB with unrestricted input showed that TimeFuzz can get stuck and fail to make further progress. This might affect the applicability of TimeFuzz to other kinds of PUTs.

TimeFuzz also requires PUTs to be deterministic. Both the exclusion tree and the DD algorithms (including Score Progress) rely on deterministic test results to function correctly. Consequently, TimeFuzz is not applicable to non-deterministic applications.

The algorithms used for subsequence extraction and input extension utilize the early parser for general regular and context-free input grammars. However, due to the parsers cubic worst-case time complexity, the applicability of TimeFuzz may be limited when targeting PUTs with complex input grammars.

10. Limitations

TimeFuzz offers a comprehensive implementation of a wide range of functionalities and custom algorithms. However, some limitations have to be acknowledged.

The extensive scope of the implementation inherently required a substantial investment of time in both development and bug fixing. The experimental evaluation alone accumulated a combined runtime of approximately 7254 hours, excluding preliminary testing and experiments that had to be repeated due to the unforeseen issues. This significant demand on time and computational resources imposed practical constraints on the number of experiments that could be performed within the available timeframe. These limitations were exacerbated by critical bugs in TimeFuzz that were only discovered during the evaluation phase.

Additional experiments have been prepared, featuring different inputs and structures of the score values available to TimeFuzz. Preliminary testing suggests that TimeFuzz is effective on these additional experiments, while also highlighting limitations in the design already discovered during the evaluation. An example is TimeFuzz's difficulty in generating long sequences of identical or similar user-events. These results are promising in supporting TimeFuzz's applicability to a wider range of PUTs while clearly establishing avenues for improvement of the approach. However, due to time constraints, a broader evaluation was not feasible within the scope of this work. The evaluation remains limited to specific PUTs and a large scale evaluation remains as future work.

The framework developed in this work is highly customizable, offering a range of parameters that affect its behavior and performance. The effects of these parameters are complex and often interdependent. Different settings may lead to varying outcomes depending on the characteristics of the PUT. In the conducted experiments, parameter values are selected based on results of preliminary testing aimed at identifying configurations that yielded reasonable performance and results. These choices may not reflect the objectively best settings for the individual experiments.

Fully understanding the impact of each parameter and the consequences on the performance of input generation and the experimental results, would require a large scale experimental setup. It would require a significant amount of time and computational resources to set up and conduct. Such an exhaustive parameter study was not within the scope of this work and remains for future work.

TimeFuzz focuses on parallel execution, modularity, and extensibility. The design of these features required considerable development effort. The primary focus during development was the correctness of the implemented algorithms. Consequently, the algorithms used for derivation, extraction, and extension have not been fully optimized for performance. While they reliably produce inputs, certain components may introduce overhead, both in terms of memory and performance. In practice, acceptable performance for extraction and extension can only be guaranteed for simple grammars; significant performance issues arise when applied to general regular and context-free grammars. Although some of this can be compensated for by increasing the amount of available computational resources, it ultimately imposes limits on the scalability of TimeFuzz.

The current version of TimeFuzz implements a relatively simple source selection strategy

used at the end of each generation. While this approach has the advantage of being straightforward and simple to implement, it can lead to TimeFuzz running into dead ends without being able to recover. This basic strategy may serve as a useful baseline for future enhancements but it currently restricts TimeFuzz's ability to find winning strategies in some instances. Over time, it significantly narrows the explorable search space, potentially preventing TimeFuzz from generating multiple distinct winning strategies.

Finally, the current savefile system implemented in TimeFuzz exhibits low disk space efficiency. Although compression is used to drastically reduce the size of savefiles, storing information about millions of inputs and a large exclusion tree still results in large savefiles. The implemented system reliably preserves relevant data, allows a user to continue a prior session, and allows for additional post-execution analysis to be performed. However, it also introduces significant redundancy and storage overhead. For long and test intensive fuzzing sessions with expansive exclusion trees, savefiles may take up several gigabytes of disk space, imposing considerable limitations for environments with low storage capacities.

11. Future work

The current state of the framework opens several avenues of improvement that could be explored in future work.

The scope of the evaluation performed in this work was limited due to time and resource constraints. While further preliminary tests show promising results, while highlighting shortcomings of TimeFuzz, a systematic evaluation of TimeFuzz on a larger set of experiments is required to draw definite conclusions about the approach's overall success of varied PUTs. Future work might focus on conducting an extended evaluation, incorporating the additional experiments that were not run within the scope of this work. Additionally, future work might focus on diversifying the evaluated PUTs and on performing experiments evaluating the `Score Progress` algorithm, and DD in general, on a variety of interactive programs.

The parameter configurations used during the experiments were based on the results of preliminary tests. They may not reflect the objectively best settings. Future work could evaluate the effects of different parameter configurations on the performance of TimeFuzz and the results of fuzzing sessions. This could not only improve the overall performance, but may provide insight into the interplay of different parameters, such as generation sizes and number of sources chosen in each generation.

The input generation algorithms, while functionally correct, are not fully optimized for performance. The derivation algorithm is generally applicable and performs reliably, but the extraction and expansion algorithms can only be reasonably applied to simple grammars. Future work could address the performance issues of these algorithms, by broadening the definition of simple grammars and thereby extending the range of PUTs to which TimeFuzz can be effectively applied. Additional improvements may include replacing the Earley parser with a more efficient parsing algorithm or changing the way TimeFuzz extracts sub-sequences from inputs to reduce the number of steps a parser has to complete.

Moreover, the approach to input generation used in TimeFuzz has drawbacks. While fuzzers like AFL and IJon are able to generate long subsequences of similar or identical user-events, TimeFuzz struggles to do so. Future work could research the effectiveness of combining grammar based input generation with mutation based generation on TimeFuzz's approach to generational fuzzing.

The selection of inputs to minimize and source inputs for the next generation implemented in TimeFuzz, is relatively simple. To enhance the effectiveness of the selection, future work could explore the integration of more advanced heuristics. By integrating techniques that better estimate the potential of inputs or that diversify the inputs selected, the drastic constriction of the search space cold be avoided. For instance, an approach could choose to maintain a set of inputs that are sufficiently dissimilar at all times. Such improvements may enable TimeFuzz to recover from dead ends or even avoid them entirely.

TimeFuzz implements an adaptable and scalable computation architecture that allows it to save the current status at any time and later resume the work from a saved checkpoint. This allows TimeFuzz to create a history of checkpoints that represent different stages in

the fuzzing process. For instance, TimeFuzz can save after each generation, which allows a researcher to view and analyze the current state after each generation. Expanding this system to include an GUI that can not only display the data but also allows the user to adapt parameters, change algorithms to be used, and choose the inputs to be minimized and the sources for the next generation manually, would be a straightforward task. This would allow TimeFuzz to support the human-in-the-loop paradigm, allowing a user to use their domain specific knowledge to steer the fuzzing process, potentially decreasing testing time, and reducing path constrictions to undesired paths. As the actual workload for making these changes should be considerably low based on TimeFuzz's modular and adaptable architecture, this could be an interesting prospect for future work.

While the TimeFuzz is able to save the current status at any time, the disk space efficiency of the implemented system is not optimal. Save files grow large, especially in long sessions with many inputs and large exclusion tress. Future improvements to the framework could include advanced storage solutions. Such solutions could include only saving newly generated and executed tests as well as changes to the exclusion tree, instead of creating a full save of the entire current state. Alternatively, TimeFuzz could transition to a central database that only stores the up-to-date data, and additionally metadata for backtracking to earlier generations. Implementing an advanced solution could increase TimeFuzz's disk space efficiency and its applicability on platforms with low overall disk space.

The current implementation of DD can either be applied to reproduce a test result or to reproduce a score value. Obtaining a score might not always be possible. An alternative future work may investigate the possibility of using an oracle that is able to decide whether the result of two tests is similar or identical. For example, the oracle could compare execution traces, or the PUT's visual output to determine whether the final state of two tests is sufficiently similar. This would enable the use of DD to minimized inputs at the end of a generation in broader contexts were score values are not available.

The applicability of the approach to non-deterministic programs might be theme of future work. The implemented algorithms could be extended to account for non-deterministic results by incorporating repeated executions and statistical analysis of test results. The exclusion tree could be modified to predict the statistical likelihood of test results instead of the absolute result, while maintaining its ability to decide whether inputs should be excluded by prefix or by approximation. This would significantly increase TimeFuzz's applicability to a wider range of interactive programs, as non-determinism is inherently unavoidable in any application working with more than one thread or random events. One algorithm that could be adapted for general event sequence minimization is the algorithms by Clapp et al. [19] which apply DD to non-deterministic Android applications.

The current implementation of time grammars in TimeFuzz is limited to event generation. Future work could expand this concept by integrating explicit timing constraints into the grammar definitions. Such enhancements might include specifying the minimum delay before the next user event can be provided to the PUT, or defining maximum response times for individual events. Incorporating timing information directly into the grammar would enable more fine-grained response time analysis and offer increased

customizability for timeout conditions, thereby improving the precision and adaptability of fuzzing sessions.

The exclusion tree currently employed by TimeFuzz can grow substantially during extended fuzzing sessions. Future work could address this issue by implementing pruning strategies that retain only relevant tree paths while discarding those that are no longer necessary. One possible approach is to preserve only the paths that correspond to inputs similar to the current generation's source inputs, as well as inputs generated by the most recent DD instances. This pruning strategy could help control the size of the exclusion tree without compromising its effectiveness.

12. Conclusions

This work introduced TimeFuzz, a modular and extensible fuzzing framework designed to address the lack of general purpose fuzzing tools for general interactive programs. TimeFuzz adopts a generational black-box approach, representing program inputs as time series of user-events, enabling the collection of response times and determine the point of failure within the event sequence. A central objective was to develop a scalable framework capable of interacting with a wide variety of PUTs, generating targeted and minimal program inputs, while being unaffected by common restrictions such as the programming language of the PUT.

The core contributions of this work include the design and implementation of a fully functioning fuzzing framework, the integration of user defined parameters for input generation, PUT communication, and test evaluation. TimeFuzz includes a backup mechanism, in the form of savefiles, to support long running fuzzing sessions and reanalysis of data. To implement TimeFuzz, a number of constructs and algorithms have been designed and formalized. Time grammars have been defined allowing TimeFuzz to easily generate input sequences. This work presented algorithms for input derivation, extraction of subsequences, and the extension of generated inputs. A novel DD algorithm was introduced combining the strengths of DD while focusing it on relevant parts of an input. All the algorithms were implemented with focus on parallelization and memory consumption of the framework, allowing for the regeneration of inputs on-demand.

Over 7000 hours of experiments have been performed, demonstrating the frameworks success in generating winning strategies for versions of the Maze and SMB. TimeFuzz is able to outperform the traditional grey-box fuzzing tool AFL in all test cases. It also outperforms IJon in two experiments while outperforming it slightly on the last experiment. The framework performs well on PUTs with low execution times, scales well with slow PUTs, and substantially outperforms random input generation.

Although the evaluation was constrained to a specific set of PUTs due to time and resource constraints, further preliminary tests suggest that it may be applicable to a broader range of PUTs. However, the evaluation also exposed flaws in the approach. These include path constriction, the inability to recover from dead-ends, low disk space efficiency, and relatively high memory consumption. Those need to be addressed in future iterations of the framework. Additionally, TimeFuzz is not able to generate minimal inputs because of the input generation approach used and the shortcomings of DD in generating minimized program inputs.

TimeFuzz's modular structure and inherent support for concurrent test execution and parallelization of input generation and test evaluation is a suitable base for the implementation of more advanced generation and selection algorithms as well as human-in-the-loop approaches. While the evaluation could not conclusively determine whether TimeFuzz's approach would be applicable to general interactive applications, its ability to handle a variety of PUTs while incorporating any user defined information into its evaluation positions it as a good starting point for the implementation of fuzzing approaches for interactive programs.

In conclusion, this work provides a technically sound and extensible foundation for

advancing fuzzing techniques in an area of applications where traditional fuzzers fall short: interactive programs. TimeFuzz demonstrates that a black-box approach is able to outperform traditional grey-box fuzzing with the help of simple metrics that could be obtained from external analysis such as the output of the program or displayed content. By focusing on extensibility, scalability, and customizability, TimeFuzz allows for a wide range of potential improvements and lays a foundation for future research into targeted fuzzing strategies applicable to generic interactive programs.

References

[1] ANTLR. URL https://www.antlr.org/.

[2] Lua. URL https://www.lua.org.

[3] NEAT. URL https://neat-python.readthedocs.io/en/latest/neat_overview.html.

[4] OneNote. URL https://www.microsoft.com/de-de/microsoft-365/onenote/digital-note-taking-app.

[5] QEMU. URL https://github.com/qemu/qemu.

[6] Selenium. URL https://www.selenium.dev/.

[7] S. Ariyurek, A. Betin-Can, and E. Surer. Automated Video Game Testing Using Synthetic and Human-Like Agents. URL https://arxiv.org/abs/1906.00317.

[8] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert. NAUTILUS: Fishing for Deep Bugs with Grammars. In *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, . ISBN 978-1-891562-55-6. doi: 10.14722/ndss.2019.23412. URL https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_04A-3_Aschermann_paper.pdf.

[9] C. Aschermann, S. Schumilo, A. Abbasi, and T. Holz. Ijon: Exploring Deep State Spaces via Fuzzing. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1597–1612, . doi: 10.1109/SP40000.2020.00117.

[10] Y.-M. Baek and D.-H. Bae. Automated model-based Android GUI testing using multi-level GUI comparison criteria. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ASE '16, pages 238–249. Association for Computing Machinery. ISBN 978-1-4503-3845-5. doi: 10.1145/2970276.2970313. URL https://doi.org/10.1145/2970276.2970313.

[11] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi. A Survey of Symbolic Execution Techniques. 51(3):1–39. ISSN 0360-0300, 1557-7341. doi: 10.1145/3182657. URL https://dl.acm.org/doi/10.1145/3182657.

[12] I. Banerjee, B. Nguyen, V. Garousi, and A. Memon. Graphical user interface (GUI) testing: Systematic mapping and repository. 55(10):1679–1694. ISSN 0950-5849. doi: 10.1016/j.infsof.2013.03.004. URL https://doi.org/10.1016/j.infsof.2013.03.004.

[13] G. Banks, M. Cova, V. Felmetsger, K. Almeroth, R. Kemmerer, and G. Vigna. SNOOZE: Toward a Stateful NetwOrk prOtocol fuzZEr. In S. K. Katsikas, J. López, M. Backes, S. Gritzalis, and B. Preneel, editors, *Information Security*, volume 4176, pages 343–358. Springer Berlin Heidelberg. ISBN 978-3-540-38341-3 978-3-540-38343-7. doi: 10.1007/11836810_25. URL http://link.springer.com/10.1007/11836810_25.

[14] O. Bastani, R. Sharma, A. Aiken, and P. Liang. Synthesizing program input grammars. 52(6): 95–110. ISSN 0362-1340, 1558-1160. doi: 10.1145/3140587.3062349. URL https://dl.acm.org/doi/10.1145/3140587.3062349.

[15] P. Becker. Programming Languages — C++. URL https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2010/n3092.pdf.

[16] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2329–2344. ACM. ISBN 978-1-4503-4946-8. doi: 10.1145/3133956.3134020. URL https://dl.acm.org/doi/10.1145/3133956.3134020.

[17] A. Canny, P. Palanque, and D. Navarre. Model-Based Testing of GUI Applications Featuring Dynamic Instanciation of Widgets. In *IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW 2020)*, pages 95–104. IEEE. doi: 10.1109/ICSTW50294.2020.00029. URL https://hal.science/hal-03099199.

[18] N. Chomsky. On certain formal properties of grammars. 2(2):137–167. ISSN 0019-9958. doi: 10.1016/S0019-9958(59)90362-6. URL https://www.sciencedirect.com/science/article/pii/S0019995859903626.

[19] L. Clapp, O. Bastani, S. Anand, and A. Aiken. Minimizing GUI event traces. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 422–434. ACM. ISBN 978-1-4503-4218-6. doi: 10.1145/2950290.2950342. URL https:

//dl.acm.org/doi/10.1145/2950290.2950342.
[20] M. Corporation. VCPKG. URL https://github.com/microsoft/vcpkg.
[21] J. De Ruiter and E. Poll. Protocol state fuzzing of TLS implementations. In *Proceedings of the 24th USENIX Conference on Security Symposium*, SEC'15, pages 193–206. USENIX Association. ISBN 978-1-931971-23-2.
[22] J. Deng, X. Zhu, X. Xiao, S. Wen, Q. Li, and S. Xia. Fuzzing With Optimized Grammar-Aware Mutation Strategies. 9:95061–95071. ISSN 2169-3536. doi: 10.1109/ACCESS.2021.3093904. URL https://ieeexplore.ieee.org/document/9469897/.
[23] J. Earley. An efficient context-free parsing algorithm. 13(2):94–102. ISSN 0001-0782, 1557-7317. doi: 10.1145/362007.362035. URL https://dl.acm.org/doi/10.1145/362007.362035.
[24] M. Eberlein, M. Smytzek, D. Steinhöfel, L. Grunske, and A. Zeller. Semantic Debugging. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, pages 438–449. Association for Computing Machinery. ISBN 979-8-4007-0327-0. doi: 10.1145/3611643.3616296. URL https://doi.org/10.1145/3611643.3616296.
[25] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++: Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association. URL https://www.usenix.org/conference/woot20/presentation/fioraldi.
[26] F. S. Foundation. Grep. URL https://www.gnu.org/software/grep/.
[27] E. Fredericks. Towards fuzz testing a procedurally-generated video game. In *2024 ASEE North Central Section Conference*, number 10.18260/1-2–45643. ASEE Conferences.
[28] H. Gascon, C. Wressnegger, F. Yamaguchi, D. Arp, and K. Rieck. Pulsar: Stateful Black-Box Fuzzing of Proprietary Network Protocols. In B. Thuraisingham, X. Wang, and V. Yegneswaran, editors, *Security and Privacy in Communication Networks*, volume 164, pages 330–347. Springer International Publishing. ISBN 978-3-319-28864-2 978-3-319-28865-9. doi: 10.1007/978-3-319-28865-9_18. URL http://link.springer.com/10.1007/978-3-319-28865-9_18.
[29] P. Godefroid. Fuzzing: Hack, art, and science. 63(2):70–76. ISSN 0001-0782, 1557-7317. doi: 10.1145/3363824. URL https://dl.acm.org/doi/10.1145/3363824.
[30] R. Gopinath and A. Zeller. Building Fast Fuzzers. URL https://arxiv.org/abs/1911.07707.
[31] R. Gopinath, A. Kampmann, N. Havrikov, E. O. Soremekun, and A. Zeller. Abstracting failure-inducing inputs. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2020, pages 237–248. Association for Computing Machinery. ISBN 978-1-4503-8008-9. doi: 10.1145/3395363.3397349. URL https://doi.org/10.1145/3395363.3397349.
[32] G. Grieco, M. Ceresa, and P. Buiras. QuickFuzz: An automatic random fuzzer for common file formats. In *Proceedings of the 9th International Symposium on Haskell*, pages 13–20. ACM. ISBN 978-1-4503-4434-0. doi: 10.1145/2976002.2976017. URL https://dl.acm.org/doi/10.1145/2976002.2976017.
[33] F. Gross, G. Fraser, and A. Zeller. EXSYST: Search-based gui testing. doi: 10.1109/ICSE.2012.6227232.
[34] L. D. Group. libFuzzer. URL https://llvm.org/docs/LibFuzzer.html.
[35] D. Grune and C. J. H. Jacobs. *Parsing Techniques: A Practical Guide*. Monographs in Computer Science. Springer, 2nd ed edition. ISBN 978-0-387-20248-8 978-0-387-68954-8.
[36] T. Gu, C. Sun, X. Ma, C. Cao, C. Xu, Y. Yao, Q. Zhang, J. Lu, and Z. Su. Practical GUI Testing of Android Applications Via Model Abstraction and Refinement. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 269–280. IEEE. ISBN 978-1-7281-0869-8. doi: 10.1109/ICSE.2019.00042. URL https://ieeexplore.ieee.org/document/8812132/.
[37] M. J. Harrold. Testing: A roadmap. In *Proceedings of the Conference on The Future of Software Engineering*, ICSE '00, pages 61–72. Association for Computing Machinery. ISBN 1-58113-253-0. doi: 10.1145/336512.336532. URL https://doi.org/10.1145/336512.336532.
[38] R. Hildebrandt and A. Zeller. Simplifying failure-inducing input. In *Proceedings of the 2000*

ACM SIGSOFT International Symposium on Software Testing and Analysis, pages 135–145. ACM. ISBN 978-1-58113-266-3. doi: 10.1145/347324.348938. URL https://dl.acm.org/doi/10.1145/347324.348938.

[39] R. Hodovan and Kiss. Practical Improvements to the Minimizing Delta Debugging Algorithm. In *Proceedings of the 11th International Joint Conference on Software Technologies*, volume 0ICSOFT, pages 241–248. doi: 10.5220/0005988602410248.

[40] R. Hodován and Kiss. Modernizing hierarchical delta debugging. In *Proceedings of the 7th International Workshop on Automating Test Case Design, Selection, and Evaluation*, pages 31–37. ACM. ISBN 978-1-4503-4401-2. doi: 10.1145/2994291.2994296. URL https://dl.acm.org/doi/10.1145/2994291.2994296.

[41] R. Hodován, Kiss, and T. Gyimóthy. Grammarinator: A grammar-based open source fuzzer. In *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, pages 45–48. ACM. ISBN 978-1-4503-6053-1. doi: 10.1145/3278186.3278193. URL https://dl.acm.org/doi/10.1145/3278186.3278193.

[42] C. Holler, K. Herzig, and A. Zeller. Fuzzing with Code Fragments. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 445–458. ISBN 978-931971-95-9. URL https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler.

[43] Z. Hu, J. Shi, Y. Huang, J. Xiong, and X. Bu. GANFuzz: A GAN-based industrial network protocol fuzzing framework. In *Proceedings of the 15th ACM International Conference on Computing Frontiers*, pages 138–145. ACM. ISBN 978-1-4503-5761-6. doi: 10.1145/3203217.3203241. URL https://dl.acm.org/doi/10.1145/3203217.3203241.

[44] H. J. H. *Adaption in Natural and Artifical Systems*. MIT Press.

[45] S. Jarzabek and T. Krawczyk. LL-regular grammars. 4(2):31–37. ISSN 00200190. doi: 10.1016/0020-0190(75)90009-5. URL https://linkinghub.elsevier.com/retrieve/pii/0020019075900095.

[46] B. Jiang, Y. Wu, and T. Li. SimplyDroid. In *ASE '17: Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ACM Digital Library, pages 297–307. IEEE Press, . ISBN 978-1-5386-2684-9.

[47] T. Jiang, M. Li, B. Ravikumar, and K. Regan. Formal grammars and languages. pages 20–20, .

[48] Z.-M. Jiang, J.-J. Bai, and Z. Su. DynSQL: Stateful fuzzing for database management systems with complex and valid SQL query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4949–4965. USENIX Association, . ISBN 978-1-939133-37-3. URL https://www.usenix.org/conference/usenixsecurity23/presentation/jiang-zu-ming.

[49] A. Kampmann, N. Havrikov, E. O. Soremekun, and A. Zeller. When does my program do this? learning circumstances of software behavior. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, pages 1228–1239. Association for Computing Machinery. ISBN 978-1-4503-7043-1. doi: 10.1145/3368089.3409687. URL https://doi.org/10.1145/3368089.3409687.

[50] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. 220 (4598):671–680. ISSN 0036-8075, 1095-9203. doi: 10.1126/science.220.4598.671. URL https://www.science.org/doi/10.1126/science.220.4598.671.

[51] Kiss, R. Hodován, and T. Gyimóthy. HDDr: A recursive variant of the hierarchical Delta debugging algorithm. In *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, pages 16–22. ACM. ISBN 978-1-4503-6053-1. doi: 10.1145/3278186.3278189. URL https://dl.acm.org/doi/10.1145/3278186.3278189.

[52] N. K.K. Super Mario Bros. URL https://www.nintendo.com/de-de/Spiele/NES/Super-Mario-Bros-803853.html.

[53] C. Kreitz. Theoretische Informatik I.

[54] C. Lemieux, R. Padhye, K. Sen, and D. Song. PerfFuzz: Automatically generating pathological inputs. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 254–265. ACM. ISBN 978-1-4503-5699-2. doi: 10.1145/3213846.3213874. URL https://dl.acm.org/doi/10.1145/3213846.3213874.

[55] J. Li, B. Zhao, and C. Zhang. Fuzzing: A survey. 1(1), . doi: 10.1186/s42400-018-0002-y.

[56] P. Li, W. Meng, and C. Zhang. SDFuzz: Target states driven directed fuzzing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 2441–2457. USENIX Association, . ISBN 978-1-939133-44-1. URL https://www.usenix.org/conference/usenixsecurity24/presentation/li-penghui.

[57] Y. Li, Z. Yang, Y. Guo, and X. Chen. Humanoid: A deep learning-based approach to automated black-box Android app testing. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*, ASE '19, pages 1070–1073. IEEE Press, . ISBN 978-1-7281-2508-4. doi: 10.1109/ASE.2019.00104. URL https://doi.org/10.1109/ASE.2019.00104.

[58] G. LLC. American Fuzzy Loop. URL https://github.com/google/AFL.

[59] G. LLC and Others. UI/Application Exerciser Monkey. URL https://developer.android.com/studio/test/other-testing-tools/monkey.

[60] A. Machiry, R. Tahiliani, and M. Naik. Dynodroid: An input generation system for Android apps. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2013, pages 224–234. Association for Computing Machinery. ISBN 978-1-4503-2237-9. doi: 10.1145/2491411.2491450. URL https://doi.org/10.1145/2491411.2491450.

[61] R. Mahmood, N. Mirzaei, and S. Malek. EvoDroid: Segmented evolutionary testing of Android apps. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 599–609. ACM. ISBN 978-1-4503-3056-5. doi: 10.1145/2635868.2635896. URL https://dl.acm.org/doi/10.1145/2635868.2635896.

[62] K. Mao, M. Harman, and Y. Jia. Sapienz: Multi-objective automated testing for Android applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ISSTA 2016, pages 94–105. Association for Computing Machinery. ISBN 978-1-4503-4390-9. doi: 10.1145/2931037.2931054. URL https://doi.org/10.1145/2931037.2931054.

[63] L. Mariani, D. Hao, R. Subramanyan, and H. Zhu. The central role of test automation in software quality assurance. 25(3):797–802. ISSN 0963-9314, 1573-1367. doi: 10.1007/s11219-017-9383-5. URL http://link.springer.com/10.1007/s11219-017-9383-5.

[64] B. Mathis, R. Gopinath, M. Mera, A. Kampmann, M. Höschele, and A. Zeller. Parser-directed fuzzing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, pages 548–560. Association for Computing Machinery. ISBN 978-1-4503-6712-7. doi: 10.1145/3314221.3314651. URL https://doi.org/10.1145/3314221.3314651.

[65] P. McMinn. Search-based software test data generation: A survey. 14(2):105–156. ISSN 0960-0833, 1099-1689. doi: 10.1002/stvr.294. URL https://onlinelibrary.wiley.com/doi/10.1002/stvr.294.

[66] A. Memon, I. Banerjee, and A. Nagarajan. GUI Ripping: Reverse Engineering of Graphical User Interfaces for Testing. In *Proceedings of the 10th Working Conference on Reverse Engineering*, WCRE '03, page 260. IEEE Computer Society, . ISBN 0-7695-2027-8.

[67] A. M. Memon, M. E. Pollack, and M. L. Soffa. Hierarchical GUI Test Case Generation Using Automated Planning. 27(2):144–155, . ISSN 0098-5589. doi: 10.1109/32.908959. URL https://doi.org/10.1109/32.908959.

[68] R. Meng, Z. Dong, J. Li, I. Beschastnikh, and A. Roychoudhury. Linear-time Temporal Logic guided Greybox Fuzzing. URL https://arxiv.org/abs/2109.02312.

[69] B. P. Miller, L. Fredriksen, and B. So. An empirical study of the reliability of UNIX utilities. 33 (12):32–44. ISSN 0001-0782, 1557-7317. doi: 10.1145/96267.96279. URL https://dl.acm.org/doi/10.1145/96267.96279.

[70] G. Misherghi and Z. Su. HDD: Hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering*, ICSE '06, pages 142–151. Association for Computing Machinery. ISBN 1-59593-375-1. doi: 10.1145/1134285.1134307. URL https://doi.org/10.1145/1134285.1134307.

[71] R. Natella. StateAFL: Greybox fuzzing for stateful network servers. 27(7):191. ISSN 1382-3256, 1573-7616. doi: 10.1007/s10664-022-10233-3. URL https://link.springer.com/10.1007/s10664-022-10233-3.

[72] M.-D. Nguyen, S. Bardin, R. Bonichon, R. Groz, and M. Lemerre. Binary-level directed fuzzing for Use-After-Free vulnerabilities. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pages 47–62. USENIX Association. ISBN 978-1-939133-18-2. URL https://www.usenix.org/conference/raid2020/presentation/nguyen.

[73] OpenAI. Gym Retro. URL https://openai.com/index/gym-retro/.

[74] V. Paliath, E. Trickel, T. Bao, R. Wang, A. Doupé, and Y. Shoshitaishvili. SandPuppy: Deep-State Fuzzing Guided by Automatic Detection of State-Representative Variables. In F. Maggi, M. Egele, M. Payer, and M. Carminati, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 227–250. Springer Nature Switzerland. ISBN 978-3-031-64171-8.

[75] D. Parygina, T. Mezhuev, and D. Kuts. LibAFL-DiFuzz: Advanced Architecture Enabling Directed Fuzzing. URL https://arxiv.org/abs/2412.19143.

[76] T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana. SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2155–2168. ACM. ISBN 978-1-4503-4946-8. doi: 10.1145/3133956.3134073. URL https://dl.acm.org/doi/10.1145/3133956.3134073.

[77] J. Pfau. Progression Balancing × Baldur's Gate 3: Insights, Terms and Tools for Multi-Dimensional Video Game Balance. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–12. ACM. ISBN 979-8-4007-1394-1. doi: 10.1145/3706598.3713162. URL https://dl.acm.org/doi/10.1145/3706598.3713162.

[78] J. Pfau, J. D. Smeddinck, and R. Malaka. Automated Game Testing with ICARUS: Intelligent Completion of Adventure Riddles via Unsupervised Solving. In *Extended Abstracts Publication of the Annual Symposium on Computer-Human Interaction in Play*, pages 153–164. ACM. ISBN 978-1-4503-5111-9. doi: 10.1145/3130859.3131439. URL https://dl.acm.org/doi/10.1145/3130859.3131439.

[79] V.-T. Pham, M. Bohme, and A. Roychoudhury. AFLNET: A Greybox Fuzzer for Network Protocols. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 460–465. IEEE. ISBN 978-1-7281-5778-8. doi: 10.1109/ICST46399.2020.00062. URL https://ieeexplore.ieee.org/document/9159093/.

[80] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang. Test-case reduction for C compiler bugs. 47(6):335–346. ISSN 0362-1340. doi: 10.1145/2345156.2254104. URL https://doi.org/10.1145/2345156.2254104.

[81] K. S. Said, L. Nie, A. A. Ajibode, and X. Zhou. GUI testing for mobile applications: Objectives, approaches and challenges. In *Proceedings of the 12th Asia-Pacific Symposium on Internetware*, Internetware '20, pages 51–60. Association for Computing Machinery. ISBN 978-1-4503-8819-1. doi: 10.1145/3457913.3457931. URL https://doi.org/10.1145/3457913.3457931.

[82] W. Samek, T. Wiegand, and K.-R. Müller. Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models. URL https://arxiv.org/abs/1708.08296.

[83] SEGA. Sonic the Hedgehog. URL https://www.sega.com/de/sonic-the-hedgehog.

[84] R. A. Silva, S. D. R. Senger De Souza, and P. S. Lopes De Souza. A systematic review on search based mutation testing. 81:19–35. ISSN 09505849. doi: 10.1016/j.infsof.2016.01.017. URL https://linkinghub.elsevier.com/retrieve/pii/S0950584916300167.

[85] C. Sun, Y. Li, Q. Zhang, T. Gu, and Z. Su. Perses: Syntax-guided program reduction. In *Proceedings of the 40th International Conference on Software Engineering*, ICSE '18, pages 361–371. Association for Computing Machinery. ISBN 978-1-4503-5638-1. doi: 10.1145/3180155.3180236. URL https://doi.org/10.1145/3180155.3180236.

[86] S. Thomas et al. Zork: THe Underground Empire. URL https://github.com/devshane/zork.

[87] G. Wang, R. Shen, J. Chen, Y. Xiong, and L. Zhang. Probabilistic Delta debugging. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, pages 881–892. Association for Computing Machinery, . ISBN 978-1-4503-8562-6. doi: 10.1145/3468264.3468625. URL https://doi.org/10.1145/3468264.3468625.

[88] J. Wang, B. Chen, L. Wei, and Y. Liu. Skyfire: Data-Driven Seed Generation for Fuzzing. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 579–594. IEEE, . ISBN 978-1-5090-5533-3. doi: 10.1109/SP.2017.23. URL http://ieeexplore.ieee.org/document/7958599/.

[89] J. Wang, B. Chen, L. Wei, and Y. Liu. Superion: Grammar-Aware Greybox Fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 724–735. IEEE, . ISBN 978-1-7281-0869-8. doi: 10.1109/ICSE.2019.00081. URL https://ieeexplore.ieee.org/document/8811923/.

[90] J. Wang, Y. Jiang, C. Xu, C. Cao, X. Ma, and J. Lu. ComboDroid: Generating high-quality test inputs for Android apps via use case combinations. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, pages 469–480. Association for Computing Machinery, . ISBN 978-1-4503-7121-6. doi: 10.1145/3377811.3380382. URL https://doi.org/10.1145/3377811.3380382.

[91] P. Wang, X. Zhou, T. Yue, P. Lin, Y. Liu, and K. Lu. The Progress, Challenges, and Perspectives of Directed Greybox Fuzzing. . doi: 10.48550/ARXIV.2005.11907. URL https://arxiv.org/abs/2005.11907.

[92] T. Xie, N. Tillmann, J. De Halleux, and W. Schulte. Fitness-guided path exploration in dynamic symbolic execution. In *2009 IEEE/IFIP International Conference on Dependable Systems & Networks*, pages 359–368. IEEE. ISBN 978-1-4244-4422-9. doi: 10.1109/DSN.2009.5270315. URL http://ieeexplore.ieee.org/document/5270315/.

[93] M. Xu, S. Kashyap, H. Zhao, and T. Kim. Krace: Data Race Fuzzing for Kernel File Systems. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1643–1660. IEEE, . ISBN 978-1-7281-3497-0. doi: 10.1109/SP40000.2020.00078. URL https://ieeexplore.ieee.org/document/9152693/.

[94] Z. Xu, Y. Tian, M. Zhang, G. Zhao, Y. Jiang, and C. Sun. Pushing the Limit of 1-Minimality of Language-Agnostic Program Reduction. 7, . doi: 10.1145/3586049. URL https://doi.org/10.1145/3586049.

[95] J. Yan, H. Zhou, X. Deng, P. Wang, R. Yan, J. Yan, and J. Zhang. Efficient testing of GUI applications by event sequence reduction. 201:102522. doi: 10.1016/j.scico.2020.102522.

[96] D. Yang, Y. Zhang, and Q. Liu. BlendFuzz: A Model-Based Framework for Fuzz Testing Programs with Grammatical Inputs. In *2012 IEEE 11th International Conference on Trust, Security and Privacy in Computing and Communications*, pages 1070–1076. IEEE. ISBN 978-1-4673-2172-3 978-0-7695-4745-9. doi: 10.1109/TrustCom.2012.99. URL http://ieeexplore.ieee.org/document/6296093/.

[97] A. Zeller. Yesterday, my program worked. Today, it does not. Why? 24(6):253–267. ISSN 0163-5948. doi: 10.1145/318774.318946. URL https://doi.org/10.1145/318774.318946.

[98] A. Zeller and R. Hildebrandt. Simplifying and Isolating Failure-Inducing Input. 28(2):183–200. ISSN 0098-5589. doi: 10.1109/32.988498. URL https://doi.org/10.1109/32.988498.

[99] Y. Zhang, Z. Chen, J. Wang, W. Dong, and Z. Liu. Regular property guided dynamic symbolic execution. ICSE '15, pages 643–653. IEEE Press. ISBN 978-1-4799-1934-5.

[100] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang. Fuzzing: A Survey for Roadmap. 54:1–36. ISSN 0360-0300, 1557-7341. doi: 10.1145/3512345. URL https://dl.acm.org/doi/10.1145/3512345.

Appendix

A. Exclusion Tree Sizes

This appendix shows the Tables 13 and 14. The tables show the average values for the exclusion trees of each individual level. All values are the averages over the three performed runs with the with the memory estimate being shown in the last column.

Metric	Depth	Nodes	Leaves	Min Size MB
1-1	50	11609	521	0.71
1-2	98	18643	188	1.14
1-3	36	85735	34271	5.23
1-4	65	6110878	80537	372.98
2-1	100	42588	693	2.60
2-2	100	5321805	119996	324.82
2-3	37	32736	4585	2.00
2-4	62	2539758	66115	155.01
3-1	50	18826	1038	1.15
3-2	47	11855	105	0.72
3-3	30	66634	19819	4.07
3-4	67	3107645	17732	189.68
4-1	39	7224	538	0.44
4-2	99	2231648	273227	136.21
4-3	29	101446	32825	6.19
4-4	100	4535344	108531	276.82
5-1	44	59988	732	3.66
5-2	40	35831	3371	2.19
5-3	38	68164	26734	4.16
5-4	66	5111691	147613	311.99
6-1	46	12814	1840	0.78
6-2	99	27089	260	1.65
6-3	29	29521	10108	1.80
6-4	67	7861331	75685	479.82
7-1	53	8062	189	0.49
7-2	100	7983474	199606	487.27
7-3	35	27734	2561	1.69
7-4	72	3882691	45504	236.98
8-1	67	745724	152818	45.52
8-2	48	25740	2088	1.57
8-3	37	24263	1005	1.48
8-4	94	4502950	37800	274.84

Table 13: Average values for the exclusion tree on the restricted SMB C.

Metric	Depth	Nodes	Leaves	Min Size MB
1-1	1000	25349832	23090	1547.23
1-2	999	8986816	46506	548.51
1-3	1000	134263706	997670	8194.81
1-4	1000	52243623	221704	3188.7
2-1	1000	125507601	320927	7660.38
2-2	1000	203966	6320	12.45
2-3	1000	114015409	306059	6958.95
2-4	1000	41295164	184858	2520.46
3-1	1000	12690930	31182	774.59
3-2	998	5253386	30405	320.64
3-3	1000	83886910	412754	5120.05
3-4	1000	90966615	292285	5552.16
4-1	1000	33606775	73439	2051.19
4-2	1000	38831470	184057	2370.08
4-3	1000	156912898	670430	9577.2
4-4	1000	77187464	260915	4711.15
5-1	1000	113546683	281360	6930.34
5-2	1000	13055548	46201	796.85
5-3	1000	129768753	1088008	7920.46
5-4	1000	33341633	148604	2035.01
6-1	1000	79642490	253765	4860.99
6-2	1000	82239693	62650	5019.51
6-3	1000	188770156	955957	11521.62
6-4	1000	51612771	230611	3150.19
7-1	1000	29474911	220102	1799.01
7-2	1000	223134	7025	13.62
7-3	1000	132221534	333307	8070.16
7-4	1000	35652485	142341	2176.05
8-1	1000	78050435	175533	4763.82
8-2	1000	65482288	271391	3996.72
8-3	1000	110562933	230711	6748.23
8-4	1000	88515593	310623	5402.56

Table 14: Average values for the exclusion tree on the unrestricted SMB C.

Selbständigkeitserklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig verfasst und noch nicht für andere Prüfungen eingereicht habe. Sämtliche Quellen einschließlich Internetquellen, die unverändert oder abgewandelt wiedergegeben werden, insbesondere Quellen für Texte, Grafiken, Tabellen und Bilder, sind als solche kenntlich gemacht. Mir ist bekannt, dass bei Verstößen gegen diese Grundsätze ein Verfahren wegen Täuschungsversuchs bzw. Täuschung eingeleitet wird.

Berlin, den June 5, 2025 Kai Münch
(The original handwritten signature has been removed for privacy reasons)